

The screenshot shows a web browser window with the title 'Newspaper App'. The address bar displays '127.0.0.1:8000/articles/new/'. The page has a header with 'Newspaper' and 'Home' links, and 'Log in' and 'Sign up' buttons. The main content area is titled 'New article' and contains two input fields: 'Title*' and 'Body*'. The 'Body*' field is a large text area. A green 'Save' button is located at the bottom left of the form.

Newspaper App

127.0.0.1:8000/articles/new/

Guest

Newspaper Home

Log in Sign up

New article

Title*

Body*

Save

Logged Out New

Now, try to create a new article with a title and body. Then, click on the “Save” button.

Newspaper App x +

← → ↻ ⓘ 127.0.0.1:8000/articles/new/ Guest ⋮

Newspaper Home Log in Sign up

New article

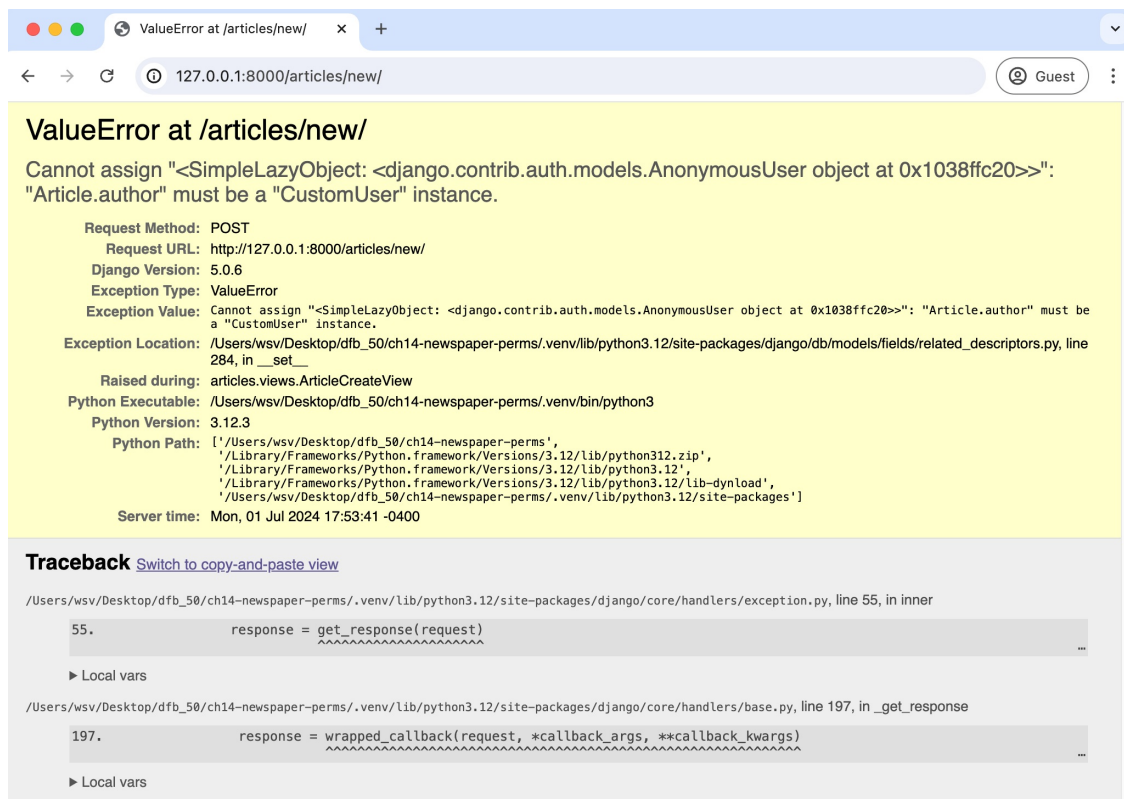
Title*

Body*

Save

로그아웃 새로 만들기

이제 제목과 본문이 있는 새 기사를 작성해 보세요. 그런 다음 "저장" 버튼을 클릭하세요.



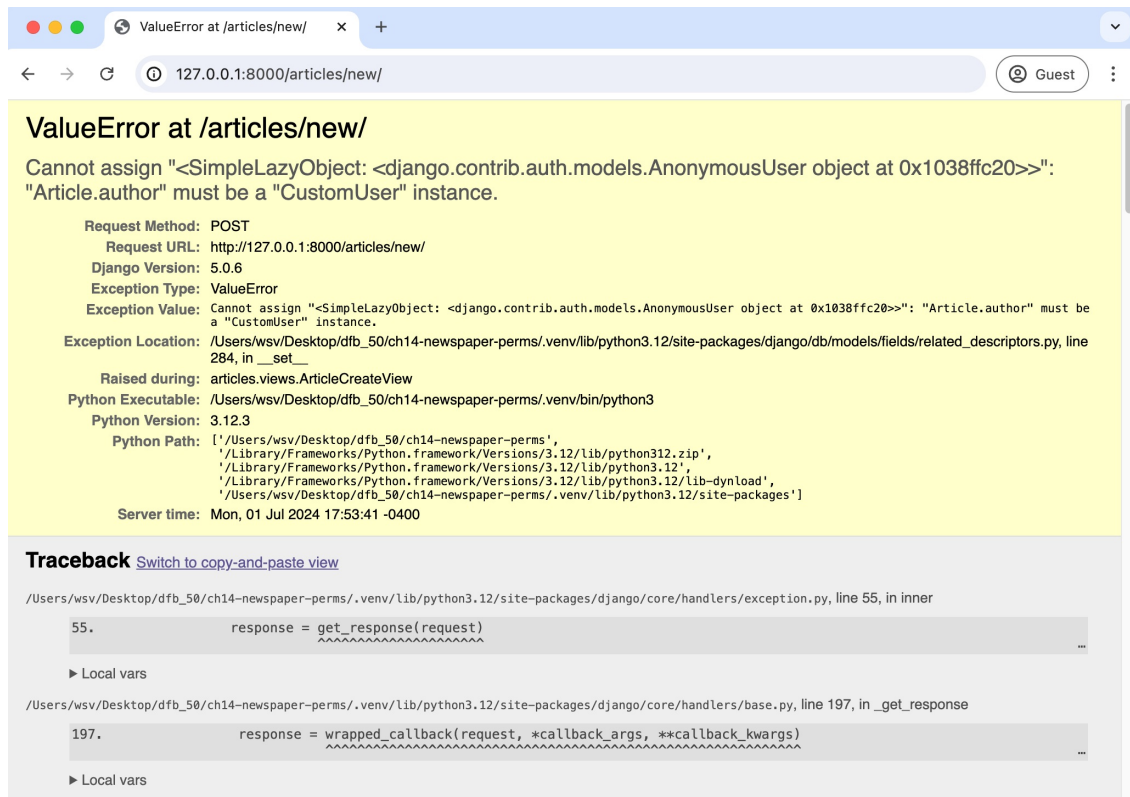
Create Page Error

An error! This is because our model **expects** an author field that is linked to the currently logged-in user. But since we are not logged in, there's no author, so the submission fails. What to do?

Mixins

We want to set some authorizations so only logged-in users can access specific URLs. To do this, we can use a *mixin*, a special kind of multiple inheritance that Django uses to avoid duplicate code and still allow customization. For example, the built-in generic [ListView](#) needs a way to return a template. But so does [DetailView](#) and almost every other view. Rather than repeat the same code in each big generic view, Django breaks out this functionality into a mixin known as [TemplateResponseMixin](#). Both `ListView` and `DetailView` use this mixin to render the proper template.

You'll see mixins used everywhere if you read the Django source code, which is freely available [on Github](#). To restrict view access to only logged-in users,



페이지 생성 오류

오류! 이는 우리의 모델이 현재 로그인한 사용자와 연결된 작성자 필드를 기대하기 때문입니다. 그러나 로그인하지 않았기 때문에 작성자가 없어서 제출이 실패합니다. 어떻게 해야 하나요?

믹스인

우리는 특정 URL에 로그인한 사용자만 접근할 수 있도록 일부 권한을 설정하고 싶습니다. 이를 위해 우리는 믹스인, 즉 Django가 중복 코드를 피하고 여전히 사용자 지정을 허용하기 위해 사용하는 특별한 종류의 다중 상속을 사용할 수 있습니다. 예를 들어, 내장된 일반 ListView는 템플릿을 반환하는 방법이 필요합니다. 그러나 DetailView와 거의 모든 다른 뷰도 마찬가지입니다. 각 큰 일반 뷰에서 동일한 코드를 반복하기보다는 Django는 이 기능을 TemplateResponseMixin으로 알려진 믹스인으로 분리합니다. ListView와 DetailView 모두 이 믹스인을 사용하여 적절한 템플릿을 렌더링합니다.

Django 소스 코드를 읽으면 믹스인이 어디에나 사용되는 것을 볼 수 있습니다. 이 코드는 Github에서 자유롭게 사용할 수 있습니다. 뷰 접근을 로그인한 사용자로 제한하려면,

Django has a [LoginRequired mixin](#) that we can use. It's powerful and extremely concise.

In the `articles/views.py` file, import `LoginRequiredMixin` and add it to `ArticleCreateView`. Make sure that the mixin is to the left of `CreateView` so it will be read first. We want the `CreateView` to know we intend to restrict access.

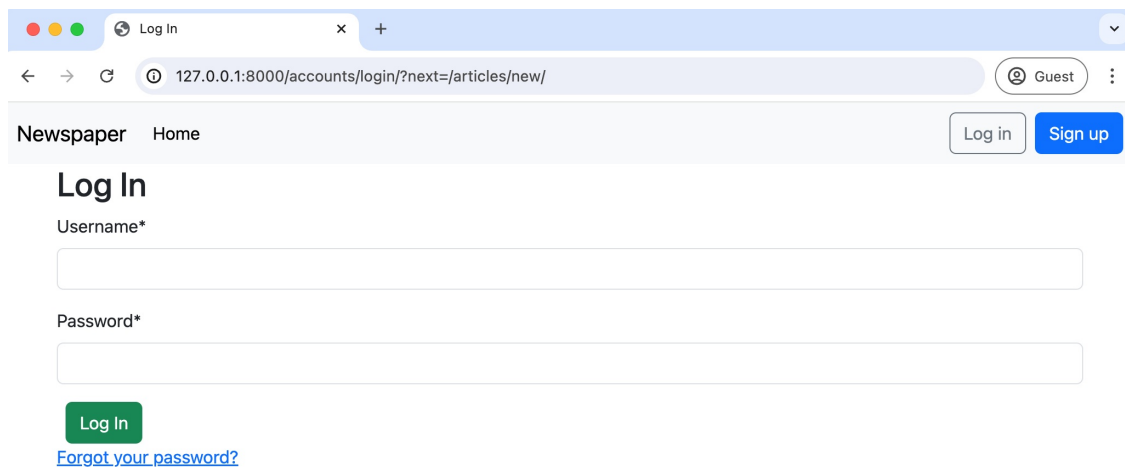
And that's it! We're done.

Code

```
# articles/views.py
from django.contrib.auth.mixins import LoginRequiredMixin # new
from django.views.generic import ListView, DetailView
...

class ArticleCreateView(LoginRequiredMixin, CreateView): # new
    ...
```

Return to the homepage at `http://127.0.0.1:8000/` to avoid resubmitting the form. Navigate to `http://127.0.0.1:8000/articles/new/` again to access the URL route for a new article.



The screenshot shows a web browser window with the title 'Log In'. The address bar displays the URL `127.0.0.1:8000/accounts/login/?next=/articles/new/`. The page layout includes a header with 'Newspaper' and 'Home' links, and 'Log in' and 'Sign up' buttons. The main content area is titled 'Log In' and contains two input fields: 'Username*' and 'Password*'. Below the fields is a green 'Log In' button and a blue link 'Forgot your password?'.

Log In Redirect Page

What's happening? Django automatically redirected users to the login page! If you look closely, the URL is `http://127.0.0.1:8000/accounts/login/?next=/articles/new/`, which shows we *tried* to go to `articles/new/` but were instead redirected to log in.

장고에는 우리가 사용할 수 있는 LoginRequired 믹스인이 있습니다. 강력하고 매우 간결합니다.

articles/views.py 파일에서 LoginRequiredMixin을 가져오고 ArticleCreateView에 추가합니다. 믹스인이 CreateView의 왼쪽에 있도록 하여 먼저 읽히도록 하세요. CreateView가 접근을 제한할 의도가 있음을 알 수 있도록 하기를 원합니다.

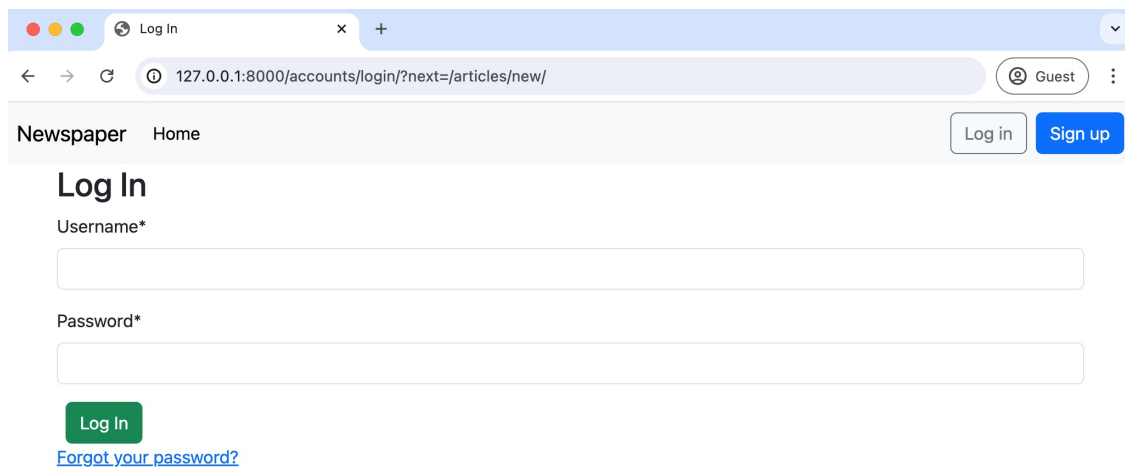
그게 전부입니다! 우리는 끝났습니다.

코드

```
# articles/views.py
from django.contrib.auth.mixins import LoginRequiredMixin # new
from django.views.generic import ListView, DetailView...

class ArticleCreateView(LoginRequiredMixin, CreateView): # new
    ...
```

http://127.0.0.1:8000/에서 홈페이지로 돌아가서 양식을 다시 제출하지 않도록 하세요. http://127.0.0.1:8000/articles/new/로 다시 이동하여 새 기사의 URL 경로에 접근하세요.



로그인 리디렉션 페이지

무슨 일이죠? Django가 사용자를 로그인 페이지로 자동으로 리디렉션했습니다! 자세히 살펴보면, URL은 http://127.0.0.1:8000/accounts/login/?next=/articles/new/로, articles/new/로 가려고 했지만 대신 로그인하라는 페이지로 리디렉션되었음을 보여줍니다.

LoginRequiredMixin

Restricting view access requires adding `LoginRequiredMixin` at the beginning of all existing views. Let's update the rest of our articles views since we don't want a user to be able to create, read, update, or delete an article if they aren't logged in.

The complete `views.py` file should now look like this:

Code

```
# articles/views.py
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView, DetailView
from django.views.generic.edit import CreateView, UpdateView, DeleteView
from django.urls import reverse_lazy

from .models import Article

class ArticleListView(LoginRequiredMixin, ListView): # new
    model = Article
    template_name = "article_list.html"

class ArticleDetailView(LoginRequiredMixin, DetailView): # new
    model = Article
    template_name = "article_detail.html"

class ArticleUpdateView(LoginRequiredMixin, UpdateView): # new
    model = Article
    fields = (
        "title",
        "body",
    )
    template_name = "article_edit.html"

class ArticleDeleteView(LoginRequiredMixin, DeleteView): # new
    model = Article
    template_name = "article_delete.html"
    success_url = reverse_lazy("article_list")

class ArticleCreateView(LoginRequiredMixin, CreateView):
    model = Article
    template_name = "article_new.html"
    fields = ("title", "body",)

    def form_valid(self, form):
        form.instance.author = self.request.user
        return super().form_valid(form)
```

LoginRequiredMixin

뷰 접근을 제한하려면 기존 모든 뷰의 시작 부분에 LoginRequiredMixin을 추가해야 합니다. 사용자가 로그인하지 않은 경우 기사를 생성, 읽기, 업데이트 또는 삭제할 수 없도록 나머지 기사 뷰를 업데이트합니다.

완전한 views.py 파일은 이제 다음과 같아야 합니다:코드

```
# articles/views.py
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView, DetailView
from django.views.generic.edit import CreateView, UpdateView, DeleteView
from django.urls import reverse_lazy

from .models import Article

class ArticleListView(LoginRequiredMixin, ListView): # new
    model = Article
    template_name = "article_list.html"

class ArticleDetailView(LoginRequiredMixin, DetailView): # new
    model = Article
    template_name = "article_detail.html"

class ArticleUpdateView(LoginRequiredMixin, UpdateView): # new
    model = Article
    fields = ("title", "body", )
    template_name = "article_edit.html"

class ArticleDeleteView(LoginRequiredMixin, DeleteView): # new
    model = Article
    template_name = "article_delete.html"
    success_url = reverse_lazy("article_list")

class ArticleCreateView(LoginRequiredMixin, CreateView):
    model = Article
    template_name = "article_new.html"
    fields = ("title", "body", )

    def form_valid(self, form):
        form.instance.author = self.request.user
        return super().form_valid(form)
```

Play around with the site to confirm that redirecting to the login works as expected. If you need help recalling the proper URLs, log in first and write down the URLs for each route to create, edit, delete, and list all articles.

UpdateView and DeleteView

We're progressing, but our edit and delete views are still an issue. Any logged-in user can change any article, but we want to restrict this access so that only the article's author has this permission.

We could add permissions logic to each view for this, but a more elegant solution is to create a dedicated mixin, a class with a particular feature we want to reuse in our Django code. And better yet, Django ships with a built-in mixin, [UserPassesTestMixin](#), just for this purpose!

To use `UserPassesTestMixin`, first, import it at the top of the `articles/views.py` file and then add it to both the update and delete views where we want this restriction. The `test_func` method is used by `UserPassesTestMixin` for our logic; we need to override it. In this case, we set the variable `obj` to the current object returned by the view using `get_object()`. Then we say, if the author on the current object matches the current user on the webpage (whoever is logged in and trying to make the change), then allow it. If false, an error will automatically be thrown.

The code looks like this:

Code

```
# articles/views.py
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    UserPassesTestMixin # new
)
from django.views.generic import ListView, DetailView
from django.views.generic.edit import UpdateView, DeleteView, CreateView
from django.urls import reverse_lazy

from .models import Article
...
class ArticleUpdateView(
    LoginRequiredMixin, UserPassesTestMixin, UpdateView): # new
    model = Article
    fields = (
        "title",
        "body",
    )
    template_name = "article_edit.html"
```

사이트를 사용해 보면서 로그인으로 리디렉션이 예상대로 작동하는지 확인하세요. 적절한 URL을 기억하는 데 도움이 필요하다면 먼저 로그인하고 각 경로에 대한 URL을 작성하세요: 생성, 편집, 삭제 및 모든 기사를 나열하는 URL.

UpdateView 및 DeleteView

우리는 진행 중이지만, 편집 및 삭제 뷰는 여전히 문제입니다. 로그인한 사용자는 어떤 기사든 변경할 수 있지만, 우리는 이 접근을 제한하여 오직 기사의 저자만 이 권한을 가지도록 하고 싶습니다.

각 뷰에 권한 로직을 추가할 수 있지만, 더 우아한 해결책은 재사용하고자 하는 특정 기능을 가진 전용 믹스인을 만드는 것입니다. 그리고 더 좋은 점은 Django에는 이 목적을 위해 내장 믹스인인 [UserPassesTestMixin](#)이 제공됩니다!

UserPassesTestMixin을 사용하려면 먼저 `articles/views.py` 파일의 맨 위에 가져오고, 그런 다음 이 제한을 적용하고자 하는 업데이트 및 삭제 뷰에 추가하세요. `test_func` 메서드는 UserPassesTestMixin에서 우리의 로직을 위해 사용되며, 우리는 이를 재정의해야 합니다. 이 경우, 변수 `obj`를 `get_object()`를 사용하여 뷰에서 반환된 현재 객체로 설정합니다. 그런 다음 현재 객체의 저자가 웹페이지의 현재 사용자(로그인하여 변경을 시도하는 사용자)와 일치하면 허용합니다. 그렇지 않으면 오류가 자동으로 발생합니다.

코드는 다음과 같습니다:

코드

```
# articles/views.py
from django.contrib.auth.mixins import
    (LoginRequiredMixin,
     UserPassesTestMixin # new)

from django.views.generic import ListView, DetailView
from django.views.generic.edit import UpdateView, DeleteView, CreateView
from django.urls import reverse_lazy

from .models import Article
...
class ArticleUpdateView(
    LoginRequiredMixin, UserPassesTestMixin, UpdateView): # new
    model = Article
    fields = (
        "title",
        "body", )
    template_name = "article_edit.html"
```

```
def test_func(self): # new
    obj = self.get_object()
    return obj.author == self.request.user

class ArticleDeleteView(
    LoginRequiredMixin, UserPassesTestMixin, DeleteView): # new
    model = Article
    template_name = "article_delete.html"
    success_url = reverse_lazy("article_list")

    def test_func(self): # new
        obj = self.get_object()
        return obj.author == self.request.user
```

The order is critical when using mixins with class-based views. `LoginRequiredMixin` comes first so that we force login, then add `UserPassesTestMixin` for an additional layer of functionality, and finally, either `UpdateView` or `DeleteView`. The code will only work properly if you have this order.

Log in with your testuser account and go to the articles list page. If the code works, you should not be able to edit or delete any posts written by your superuser account; instead, you will see a Permission Denied 403 error page.



403 Forbidden

403 Error Page

However, if you create a new article with testuser, you *will* be able to edit and delete it. And if you log in with your superuser account instead, you can edit and delete posts written by that author.

Template Logic

Although we have successfully restricted access to the “edit” and “delete” pages for each article, they are still on the list all articles page and the individual article page. It would be better not to display them to users who cannot access them. In other words, we want to restrict their display to only the author of an article.

```

def test_func(self): # new
    obj = self.get_object()
    return obj.author == self.request.user

class ArticleDeleteView(
    LoginRequiredMixin, UserPassesTestMixin, DeleteView): model = # new
    Article
    template_name = "article_delete.html" success_url =
    reverse_lazy("article_list")

def test_func(self): # new
    obj = self.get_object()
    return obj.author == self.request.user

```

믹스인과 클래스 기반 뷰를 사용할 때 순서가 중요합니다. LoginRequiredMixin이 먼저 와서 로그인을 강제한 다음, 추가 기능을 위해 addUserPassesTestMixin을 추가하고 마지막으로 UpdateView 또는 DeleteView를 추가합니다. 이 순서가 맞아야 코드가 제대로 작동합니다.

테스트 사용자 계정으로 로그인하고 기사 목록 페이지로 이동하세요. 코드가 작동하면 슈퍼유저 계정으로 작성된 게시물을 편집하거나 삭제할 수 없어야 하며, 대신 권한 거부 403 오류 페이지가 표시됩니다.



403 Forbidden

403 오류 페이지

하지만 테스트 사용자로 새 기사를 작성하면 편집하고 삭제할 수 있습니다. 그리고 슈퍼유저 계정으로 로그인하면 해당 작성자가 작성한 게시물을 편집하고 삭제할 수 있습니다.

템플릿 로직

각 기사의 '편집' 및 '삭제' 페이지에 대한 접근을 성공적으로 제한했지만, 여전히 모든 기사 목록 페이지와 개별 기사 페이지에 표시됩니다. 접근할 수 없는 사용자에게는 표시하지 않는 것이 좋습니다. 다시 말해, 기사의 작성자에게만 표시되도록 제한하고 싶습니다.

We can add simple logic to our `article_list` and `article_detail` template files by using the built-in `if` filter so that only the article author can see the edit and delete links.

Code

```
<!-- templates/article_list.html -->
...
<div class="card-footer text-center text-muted">
  <!-- new code here -->
  {% if article.author.pk == request.user.pk %}
  <a href="{% url 'article_edit' article.pk %}">Edit</a>
  <a href="{% url 'article_delete' article.pk %}">Delete</a>
  {% endif %} <!-- new code here -->
</div>
...
```

Code

```
<!-- templates/article_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="article-entry">
  <h2>{{ object.title }}</h2>
  <p>by {{ object.author }} | {{ object.date }}</p>
  <p>{{ object.body }}</p>
</div>
<div>
  <!-- new code here -->
  {% if article.author.pk == request.user.pk %}
  <p><a href="{% url 'article_edit' article.pk %}">Edit</a>
    <a href="{% url 'article_delete' article.pk %}">Delete</a>
  </p>
  {% endif %} <!-- new code here -->
  <p>Back to <a href="{% url 'article_list' %}">All Articles</a>.</p>
</div>
{% endblock content %}
```

To make sure that our new edit/delete logic works as intended, make sure you are logged in as `testuser` and create a new article using the “+ New” button in the top navbar. If you refresh the all articles webpage, only the article authored by `testuser` should have the edit and delete links visible.

우리는 내장된 if 필터를 사용하여 article_list 및 article_detail 템플릿 파일에 간단한 논리를 추가할 수 있습니다. 이렇게 하면 오직 기사 작성자만 편집 및 삭제 링크를 볼 수 있습니다.

코드

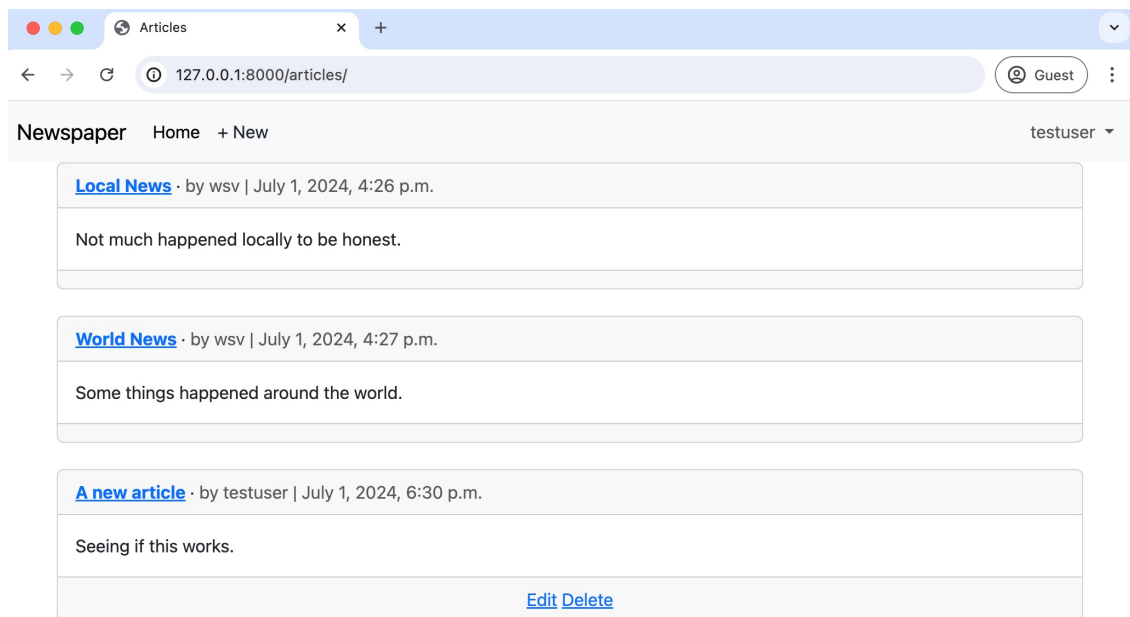
```
<!-- templates/article_list.html -->
...
<div class="card-footer text-center text-muted">
  <!-- new code here -->
  {% if article.author.pk == request.user.pk %}
  <a href="{% url 'article_edit' article.pk %}">편집</a><a href="{% url
'article_delete' article.pk %}">삭제</a>{% endif %} <!-- new code here
-->
</div>
...
```

코드

```
<!-- templates/article_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="article-entry"><h2>
  {{ object.title }}</h2>
  <p>작성자 {{ object.author }} | {{ object.date }}</p>
  <p>{{ object.body }}</p>
</div>
<div>
  <!-- new code here -->
  {% if article.author.pk == request.user.pk %}
  <p><a href="{% url 'article_edit' article.pk %}">수정</a><a href="{% url
'article_delete' article.pk %}">삭제</a>
</p>
  {% endif %} <!-- new code here -->
  <p>모든 기사로 돌아가기 <a href="{% url 'article_list' %}">All Articles</a>.</p>
</div>
{% endblock content %}
```

새로운 편집/삭제 로직이 의도한 대로 작동하는지 확인하려면, testuser로 로그인하고 상단 내비게이션 바의 “+ 새로 만들기” 버튼을 사용하여 새 기사를 작성하세요. 모든 기사 웹페이지를 새로 고치면, testuser가 작성한 기사만 편집 및 삭제 링크가 표시되어야 합니다.



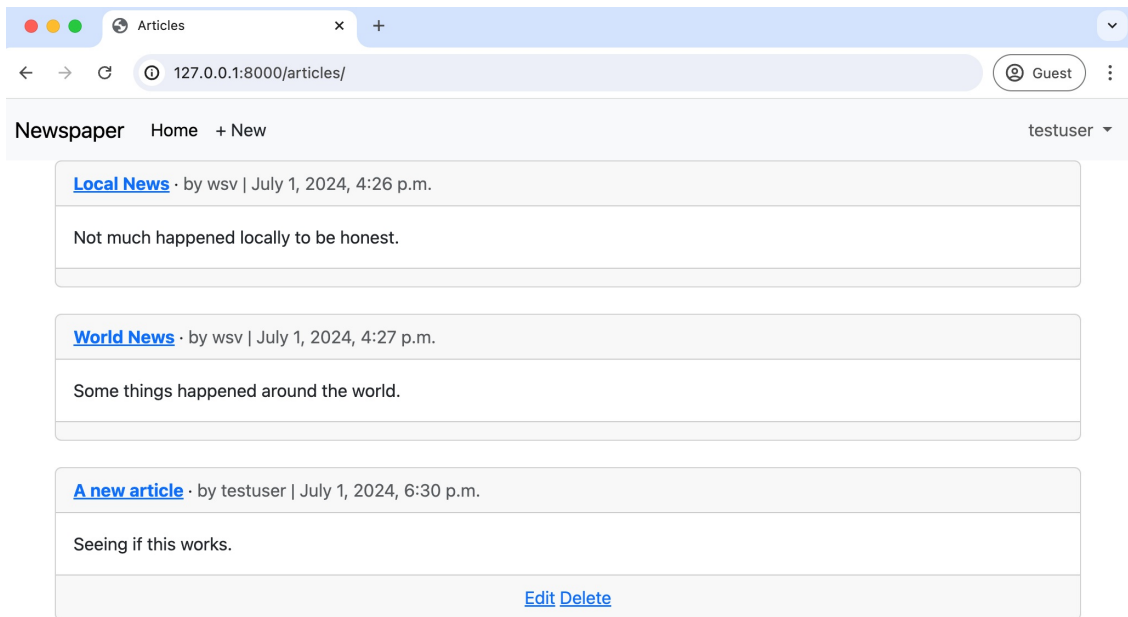
Edit/Delete Links Not Shown

Click on the article name to navigate to its detail page. The edit and delete links are visible.



Edit/Delete Links Shown for testuser

However, if you navigate to the detail page of an article created by superuser, the links are gone.



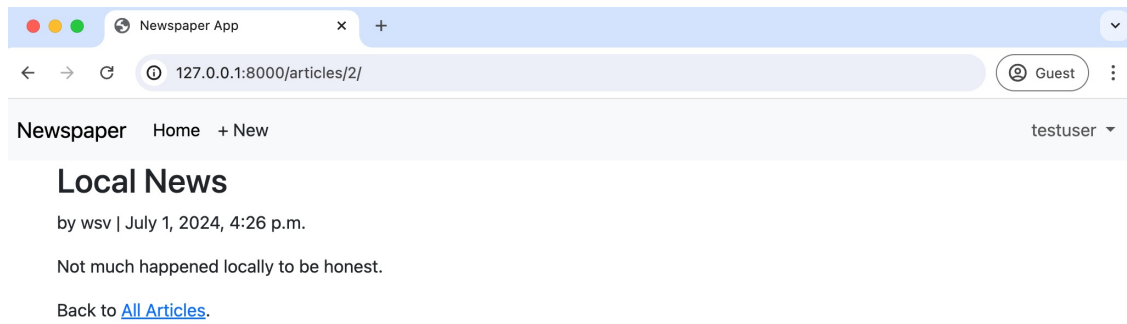
편집/삭제 링크가 표시되지 않음

기사 이름을 클릭하여 세부 페이지로 이동합니다. 편집 및 삭제 링크가 표시됩니다.



테스트 사용자에게 대한 편집/삭제 링크 표시됨

그러나 슈퍼유저가 생성한 기사의 세부 페이지로 이동하면 링크가 사라집니다.



Edit/Delete Links Not Shown

Git

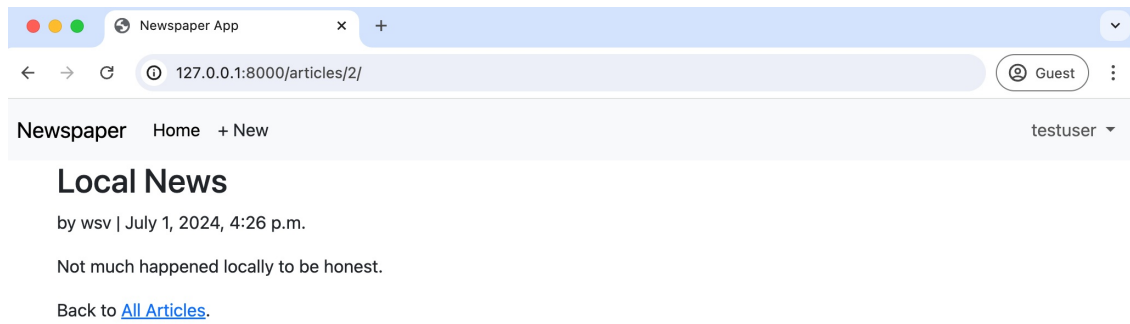
A quick save with Git is in order as we finish this chapter.

Shell

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "permissions and authorizations"
(.venv) $ git push origin main
```

Conclusion

Our newspaper app is almost done. We could take further steps at this point, such as only displaying edit and delete links to the appropriate users, which would involve [custom template tags](#), but overall, the app is in good shape. Our articles are correctly configured, set permissions and authorizations, and have a working user authentication flow. The last item needed is the ability for fellow logged-in users to leave comments, which we'll cover in the next chapter.



편집/삭제 링크가 표시되지 않음

Git

이 장을 마치면서 Git으로 빠른 저장이 필요합니다.

셸

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "permissions and authorizations"
(.venv) $ git push origin main
```

결론

우리의 신문 앱은 거의 완성되었습니다. 이 시점에서 적절한 사용자에게만 편집 및 삭제 링크를 표시하는 것과 같은 추가 단계를 진행할 수 있으며, 이는 사용자 정의 템플릿 태그를 포함할 것입니다. 하지만 전반적으로 앱은 좋은 상태입니다. 우리의 기사들은 올바르게 구성되어 있으며, 권한과 인증이 설정되어 있고, 작동하는 사용자 인증 흐름이 있습니다. 마지막으로 필요한 항목은 로그인한 사용자들이 댓글을 남길 수 있는 기능이며, 이는 다음 장에서 다룰 것입니다.

Chapter 15: Comments

We could add comments to our *Newspaper* site in two ways. The first is to create a dedicated comments app and link it to articles; however, that seems like over-engineering. Instead, we can add a model called `Comment` to our articles app and link it to the `Article` model through a foreign key. We will take the more straightforward approach since adding more complexity later is always possible.

The whole Django structure of one project containing multiple smaller apps is designed to help the developer reason about the website. The computer doesn't care how the code is structured. Breaking functionality into smaller pieces helps us—and future teammates—understand the logic in a web application. But you don't need to optimize prematurely. If your eventual comments logic becomes lengthy, then yes, by all means, spin it off into its own comments app. But the first thing to do is make the code work, make sure it is performant, and structure it, so it's understandable to you or someone else months later.

What do we need to add comments functionality to our website? We already know it will involve models, URLs, views, templates, and in this case, forms. We need all four for the final solution, but the order in which we tackle them is largely up to us. Many Django developers find that going in this order—models -> URLs -> views -> templates/forms—works best, so that is what we will use here. By the end of this chapter, users will be able to add comments to any existing article on our website.

Model

Let's begin by adding another table to our existing database called `Comment`. This model will have a many-to-one foreign key relationship to `Article`: one article can have many comments, but not vice versa. Traditionally the name of the foreign key field is simply the model it links to, so this field will be called `article`. The other two fields will be `comment` and `author`.

Open up the file `articles/models.py` and underneath the existing code, add the following. Note that we include `__str__` and `get_absolute_url` methods as best practices.

Code

15장: 댓글

신문 사이트에 댓글을 추가하는 방법은 두 가지가 있습니다. 첫 번째는 전용 댓글 앱을 만들고 이를 기사에 연결하는 것이지만, 이는 과도한 설계처럼 보입니다. 대신, 우리는 기사 앱에 Comment라는 모델을 추가하고 외래 키를 통해 Article 모델에 연결할 수 있습니다. 더 복잡한 구조를 나중에 추가하는 것이 항상 가능하므로, 우리는 더 간단한 접근 방식을 취할 것입니다.

하나의 프로젝트에 여러 개의 작은 앱이 포함된 전체 Django 구조는 개발자가 웹사이트에 대해 논리적으로 생각할 수 있도록 설계되었습니다. 컴퓨터는 코드가 어떻게 구조화되어 있는지 신경 쓰지 않습니다. 기능을 더 작은 조각으로 나누는 것은 우리와 미래의 팀원들이 웹 애플리케이션의 논리를 이해하는 데 도움이 됩니다. 그러나 초기에 최적화할 필요는 없습니다. 최종 댓글 로직이 길어지면, 그때는 물론, 별도의 댓글 앱으로 분리할 수 있습니다. 하지만 가장 먼저 해야 할 일은 코드를 작동하게 하고, 성능이 좋은지 확인하며, 몇 달 후에 당신이나 다른 사람이 이해할 수 있도록 구조화하는 것입니다.

웹사이트에 댓글 기능을 추가하려면 무엇이 필요할까요? 우리는 이미 모델, URL, 뷰, 템플릿, 그리고 이 경우에는 폼이 포함될 것이라는 것을 알고 있습니다. 최종 솔루션을 위해 이 네 가지가 필요하지만, 우리가 이를 처리하는 순서는 대부분 우리에게 달려 있습니다. 많은 Django 개발자들은 이 순서로 진행하는 것이 가장 잘 작동한다고 생각합니다-모델 -> URL -> 뷰 -> 템플릿/폼-그래서 우리는 여기서 이 순서를 사용할 것입니다. 이 장이 끝날 무렵, 사용자들은 우리 웹사이트의 기존 기사에 댓글을 추가할 수 있게 될 것입니다.

모델

기존 데이터베이스에 Comment라는 또 다른 테이블을 추가하는 것으로 시작하겠습니다. 이 모델은 Article에 대해 다대일 외래 키 관계를 가집니다: 하나의 기사는 여러 개의 댓글을 가질 수 있지만 그 반대는 아닙니다. 전통적으로 외래 키 필드의 이름은 연결된 모델의 이름이므로, 이 필드는 article이라고 불릴 것입니다. 나머지 두 필드는 comment와 author가 될 것입니다.

파일 `articles/models.py`를 열고 기존 코드 아래에 다음을 추가합니다. 최선의 관행으로 `__str__` 및 `get_absolute_url` 메서드를 포함한다는 점에 유의하세요.

코드

```
# articles/models.py
...
class Comment(models.Model): # new
    article = models.ForeignKey(Article, on_delete=models.CASCADE)
    comment = models.CharField(max_length=140)
    author = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
    )

    def __str__(self):
        return self.comment

    def get_absolute_url(self):
        return reverse("article_list")
```

Since we've updated our models, it's time to make a new migration file and apply it. Note that by adding articles at the end of the makemigrations command—which is optional—we are specifying we want to use just the articles app here. This is a good habit because consider what would happen if we changed models in two different apps? If we **did not** specify an app, then both apps' changes would be incorporated in the same migrations file, making it harder to debug errors in the future. Keep each migration as small and contained as possible.

Shell

```
(.venv) $ python manage.py makemigrations articles
Migrations for 'articles':
  articles/migrations/0002_comment.py
    - Create model Comment
(.venv) $ python manage.py migrate
Operations to perform:
  Apply all migrations: accounts, admin, articles, auth, contenttypes, sessions
Running migrations:
  Applying articles.0002_comment... OK
```

Admin

After making a new model, it's a good idea to play around with it in the admin app before displaying it on our website. Add comment to our admin.py file so it will be visible.

Code

```
# articles/admin.py
from django.contrib import admin
from .models import Article, Comment # new

class ArticleAdmin(admin.ModelAdmin):
```

```
# articles/models.py
...
class Comment(models.Model): # new
    article = models.ForeignKey(Article, on_delete=models.CASCADE)
    comment = models.CharField(max_length=140)
    author = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE, )

    def __str__(self): return
        self.comment

    def get_absolute_url(self): return
        reverse("article_list")
```

모델을 업데이트했으므로 새로운 마이그레이션 파일을 만들고 적용할 시간입니다. makemigrations 명령의 끝에 articles를 추가하는 것은 선택 사항이며, 여기서 articles 앱만 사용하고 싶다는 것을 지정하고 있습니다. 이는 좋은 습관입니다. 왜냐하면 두 개의 다른 앱에서 모델을 변경하면 어떻게 될지 고려해 보세요. 앱을 지정하지 않으면 두 앱의 변경 사항이 동일한 마이그레이션 파일에 포함되어 나중에 오류를 디버깅하기가 더 어려워집니다. 각 마이그레이션을 가능한 한 작고 독립적으로 유지하세요.

Shell

```
(.venv) $ python manage.py makemigrations articles
'articles'에 대한 마이그레이션:
  articles/migrations/0002_comment.py
  - 모델 Comment 생성(.venv) $ python
manage.py migrate 수행할 작업:
```

```
모든 마이그레이션 적용: accounts, admin, articles, auth, contenttypes, sessions 마이그레이션
실행 중:
  articles.0002_comment 적용 중... OK
```

관리자

새 모델을 만든 후, 웹사이트에 표시하기 전에 adminapp에서 다루어 보는 것이 좋습니다. Comment를 admin.py 파일에 추가하여 보이도록 하세요.

코드

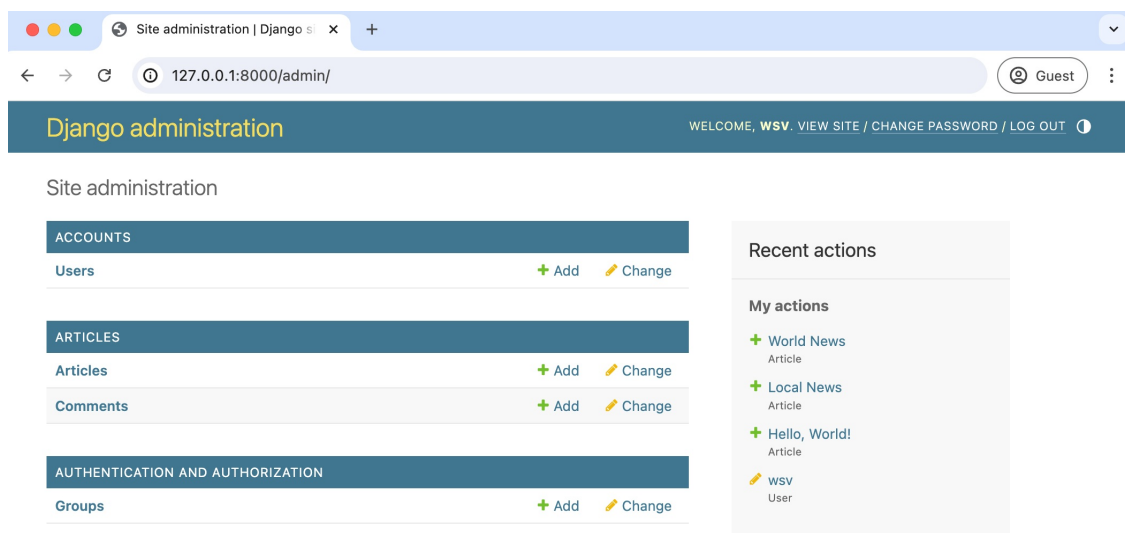
```
# articles/admin.py
from django.contrib import admin
from .models import Article, Comment # new

class ArticleAdmin(admin.ModelAdmin):
```

```
list_display = [
    "title",
    "body",
    "author",
]
```

```
admin.site.register(Article, ArticleAdmin)
admin.site.register(Comment) # new
```

Then start the server with `python manage.py runserver` and navigate to our main page `http://127.0.0.1:8000/admin/`.



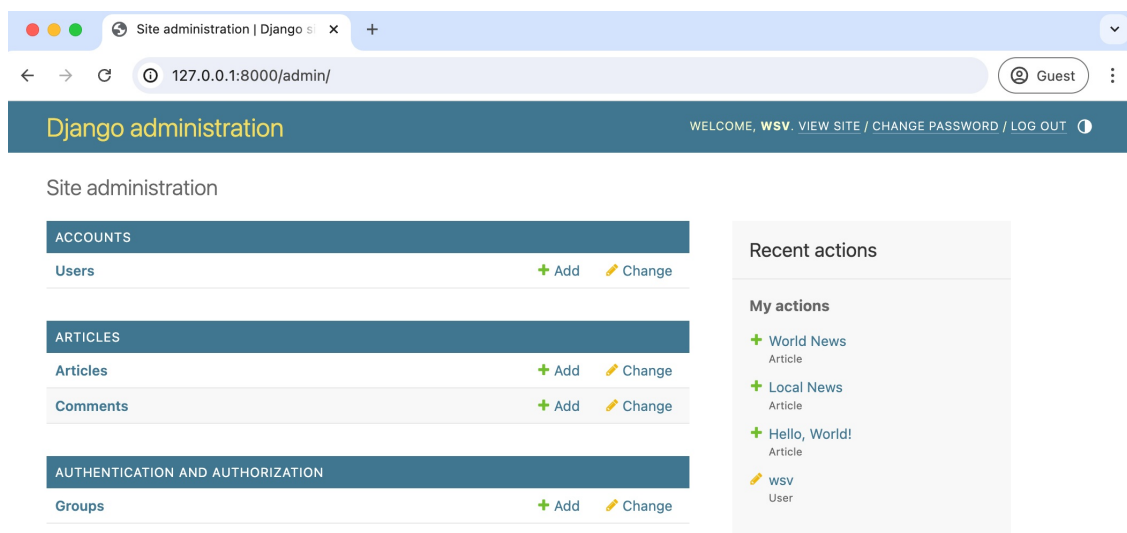
Admin Homepage with Comments

Under the “Articles” app, you’ll see our two tables: Comments and Articles. Click on the “+ Add” next to Comments. There are dropdowns for Article, Author, and a text field next to Comment.

```
list_display = [
    "title", "
    body", "
    author",
]
```

```
admin.site.register(Article, ArticleAdmin)
admin.site.register(Comment) # new
```

그런 다음 `python manage.py runserver`로 서버를 시작하고 우리 메인 페이지 `http://127.0.0.1:8000/admin/`로 이동합니다.



댓글이 있는 관리자 홈페이지

“Articles” 앱 아래에서 두 개의 테이블: Comments와 Articles를 볼 수 있습니다. Comments 옆의 “+ Add”를 클릭하세요. Article, Author에 대한 드롭다운과 Comment 옆에 텍스트 필드가 있습니다.

Django administration

WELCOME, wsv. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > Articles > Comments > Add comment

Start typing to filter...

ACCOUNTS

Users [+ Add](#)

ARTICLES

Articles [+ Add](#)

Comments [+ Add](#)

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Add comment

Article: [+](#) [👁](#)

Comment:

Author: [+](#) [👁](#)

[SAVE](#) [Save and add another](#) [Save and continue editing](#)

Admin Comments

Select an article, write a comment, and then choose an author that is not your superuser, perhaps testuser as I've done in the picture. Then click on the "Save" button.

Django administration

WELCOME, wsv. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > Articles > Comments > Add comment

Start typing to filter...

ACCOUNTS

Users [+ Add](#)

ARTICLES

Articles [+ Add](#)

Comments [+ Add](#)

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Add comment

Article: [+](#) [👁](#)

Comment:

Author: [+](#) [👁](#)

[SAVE](#) [Save and add another](#) [Save and continue editing](#)

Admin testuser Comment

You should next see your comment on the admin "Comments" page.

Django administration

WELCOME, wsv. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > Articles > Comments > Add comment

Start typing to filter...

ACCOUNTS

Users [+ Add](#)

ARTICLES

Articles [+ Add](#)

Comments [+ Add](#)

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Add comment

Article: [+](#) [👁](#)

Comment:

Author: [+](#) [👁](#)

[SAVE](#) [Save and add another](#) [Save and continue editing](#)

관리자 댓글

기사 선택, 댓글 작성 후, 내가 사진에서 한 것처럼 superuser가 아닌 작성자를 선택하세요, 아마 testuser일 것입니다. 그런 다음 "저장" 버튼을 클릭하세요.

Django administration

WELCOME, wsv. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > Articles > Comments > Add comment

Start typing to filter...

ACCOUNTS

Users [+ Add](#)

ARTICLES

Articles [+ Add](#)

Comments [+ Add](#)

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Add comment

Article: [+](#) [👁](#)

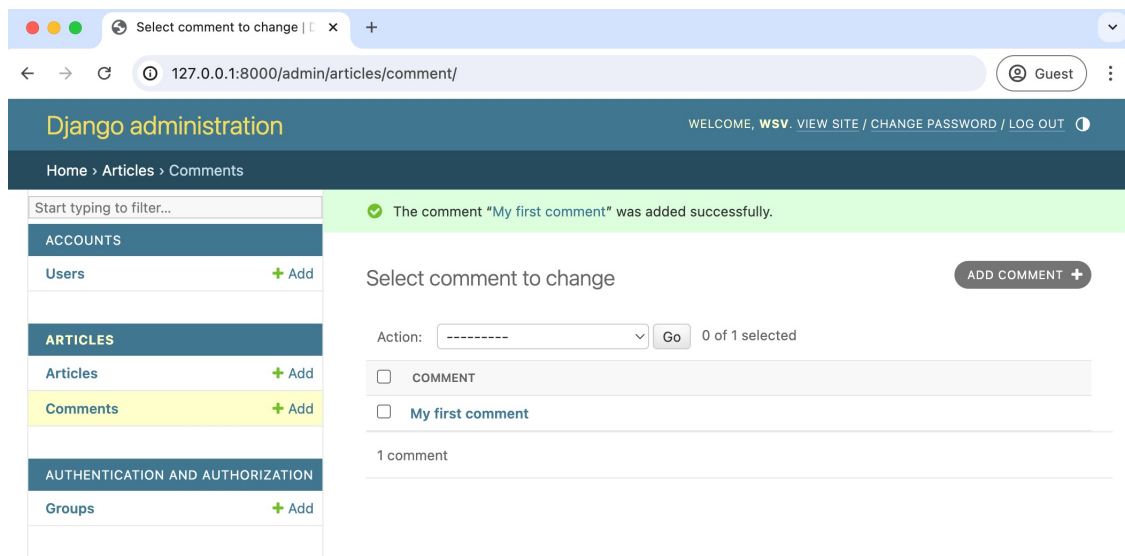
Comment:

Author: [+](#) [👁](#)

[SAVE](#) [Save and add another](#) [Save and continue editing](#)

관리자 testuser 댓글

다음으로 관리자 "댓글" 페이지에서 귀하의 댓글을 볼 수 있어야 합니다.



Admin Comment One

At this point, we could add an additional admin field to see the comment and the article on this page. But wouldn't it be better to see all Comment models related to a single Article model? We can with a Django admin feature called inlines, which displays foreign key relationships in a visual way.

There are two main inline views used: [TabularInline](#) and [StackedInline](#). The only difference between the two is the template for displaying information. In a [TabularInline](#), all model fields appear on one line; in a [StackedInline](#), each field has its own line. We'll implement both so you can decide which one you prefer.

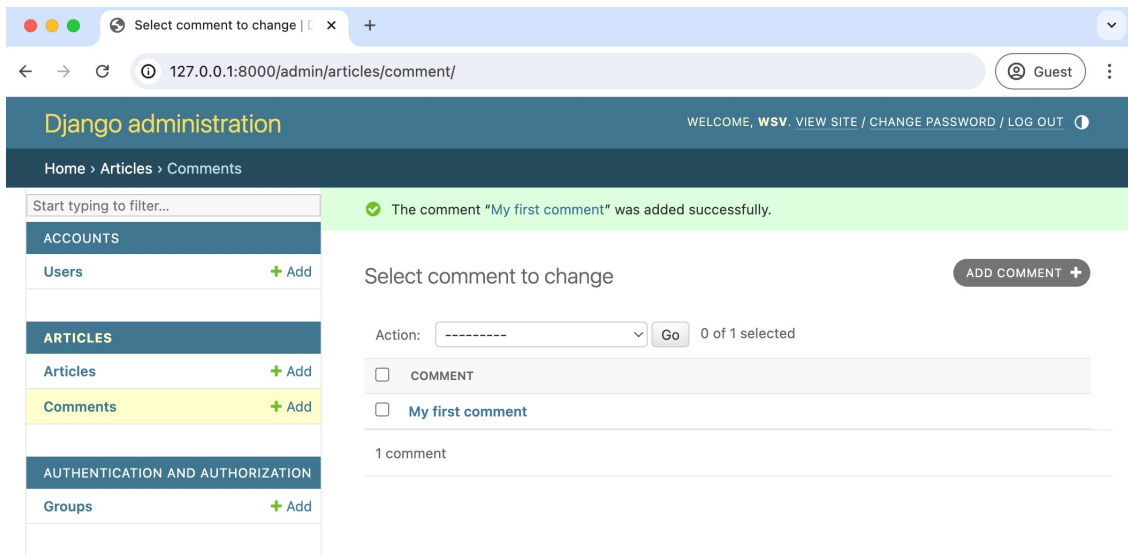
Update `articles/admin.py` in your text editor to add the `StackedInline` view.

Code

```
# articles/admin.py
from django.contrib import admin
from .models import Article, Comment

class CommentInline(admin.StackedInline): # new
    model = Comment

class ArticleAdmin(admin.ModelAdmin): # new
    inlines = [
        CommentInline,
    ]
    list_display = [
        "title",
        "body",
        "author",
    ]
```



관리자 댓글 하나

이 시점에서 이 페이지에서 댓글과 기사를 볼 수 있는 추가 관리자 필드를 추가할 수 있습니다. 하지만 단일 기사 모델과 관련된 모든 댓글 모델을 보는 것이 더 좋지 않을까요? 우리는 인라인이라는 Django 관리자 기능을 사용하여 외래 키 관계를 시각적으로 표시할 수 있습니다.

사용되는 두 가지 주요 인라인 뷰가 있습니다: [TabularInline](#)과 [StackedInline](#). 두 가지의 유일한 차이점은 정보를 표시하는 템플릿입니다. [TabularInline](#)에서는 모든 모델 필드가 한 줄에 나타나고, [StackedInline](#)에서는 각 필드가 자신의 줄에 나타납니다. 우리는 둘 다 구현할 것이므로 어떤 것을 선호하는지 결정할 수 있습니다.

텍스트 편집기에서 `articles/admin.py`를 업데이트하여 `StackedInline` 뷰를 추가합니다. 코드

```
# articles/admin.py
from django.contrib import admin from .models
import Article, Comment

class CommentInline(admin.StackedInline): # new model =
    Comment

class ArticleAdmin(admin.ModelAdmin): # new inlines =
    [
        CommentInline, ]

    list_display = [
        "제목", "
        본문", "저
        자",
```


]

```
admin.site.register(Article, ArticleAdmin) # new
admin.site.register(Comment)
```

Now go to the main admin page at `http://127.0.0.1:8000/admin/` and click “Articles.” Select the article you just commented on.

The screenshot shows the Django administration interface for a 'World News' article. The browser address bar indicates the URL `127.0.0.1:8000/admin/articles/article/3/change/`. The page title is 'Django administration' with a welcome message for 'WSV'. The sidebar on the left contains navigation links for 'ACCOUNTS' (Users), 'ARTICLES' (Articles, Comments), and 'AUTHENTICATION AND AUTHORIZATION' (Groups). The main content area is titled 'Change article' and includes buttons for 'HISTORY' and 'VIEW ON SITE'. The form fields are: 'Title' (World News), 'Body' (Some things happened around the world.), and 'Author' (WSV). Below the form is a 'COMMENTS' section with a table of existing comments and a form to add new ones. The table shows a comment by 'testuser' with the text 'My first comment'. The form for adding a new comment has fields for 'Comment:', 'Author:', and a 'Delete' checkbox.

Admin Change Page

Better, right? We can see and modify all our related articles and comments in one place. Note that, by default, the Django admin will display three empty rows here. You can change the default number that appears with the extra field. So if you wanted no extra fields by default, the code would look like this:

Code

```
# articles/admin.py
...
```

"]

```
admin.site.register(Article, ArticleAdmin)
admin.site.register(Comment)
```

새로 만들
기

이제 `http://127.0.0.1:8000/admin/`의 메인 관리자 페이지로 가서 "Articles."를 클릭하세요. 방금 댓글을 단 기사를 선택하세요.

The screenshot shows the Django administration interface. The browser address bar indicates the URL `127.0.0.1:8000/admin/articles/article/3/change/`. The page title is "Django administration". The sidebar on the left contains navigation links: "Start typing to filter...", "ACCOUNTS" (Users), "ARTICLES" (Articles, Comments), and "AUTHENTICATION AND AUTHORIZATION" (Groups). The main content area is titled "Change article" and shows the "World News" article. The form fields are: Title (World News), Body (Some things happened around the world.), and Author (WSV). Below the article form is a "COMMENTS" section with a table of comments. The first comment is "My first comment" by "testuser". The second comment is "#2" by "-----".

관리자 변경 페이지

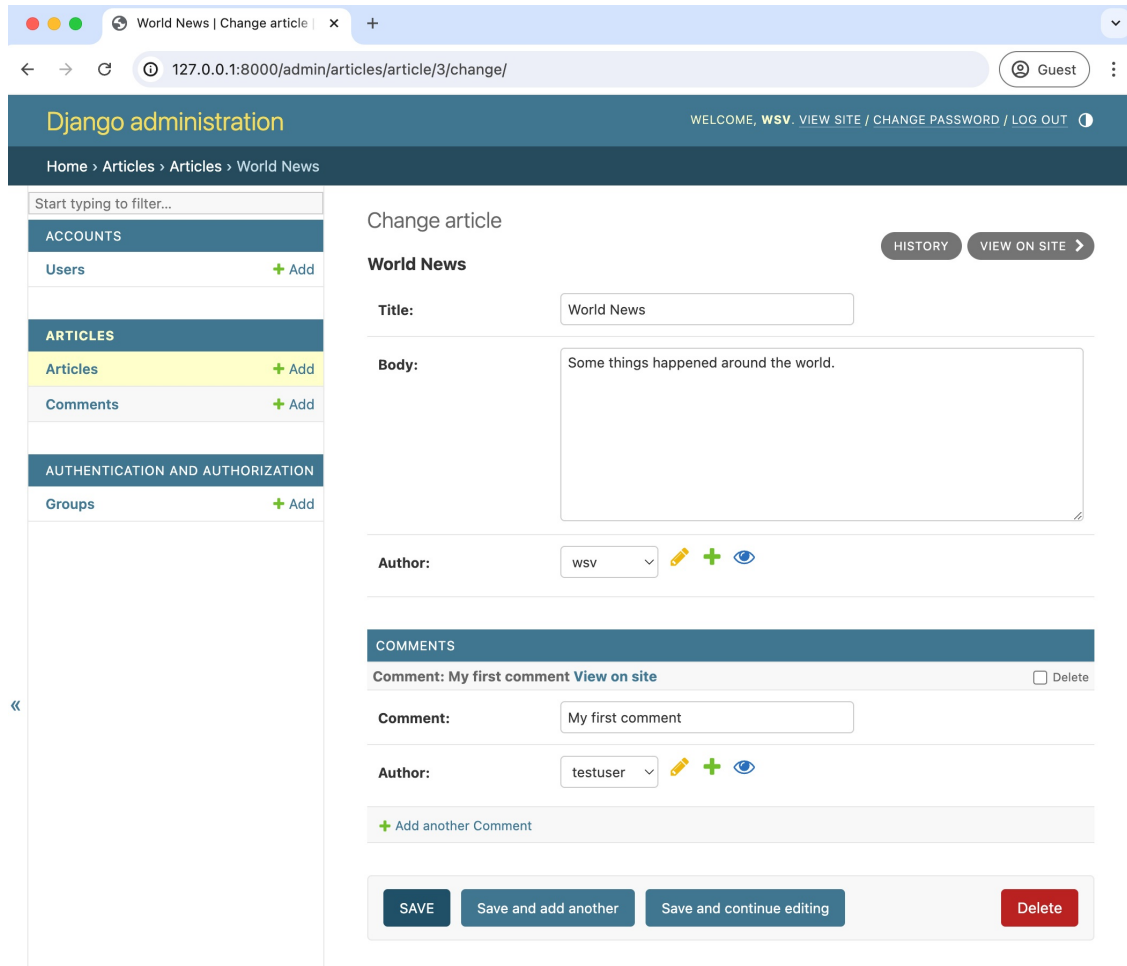
더 나아졌죠? 우리는 모든 관련 기사와 댓글을 한 곳에서 보고 수정할 수 있습니다. 기본적으로 Django 관리자는 여기에서 세 개의 빈 행을 표시합니다. 추가 필드로 표시되는 기본 숫자를 변경할 수 있습니다. 기본적으로 추가 필드가 없기를 원한다면, 코드는 다음과 같이 보일 것입니다:

코드

```
# articles/admin.py
```

...

```
class CommentInline(admin.StackedInline):
    model = Comment
    extra = 0 # new
```



Admin No Extra Comments

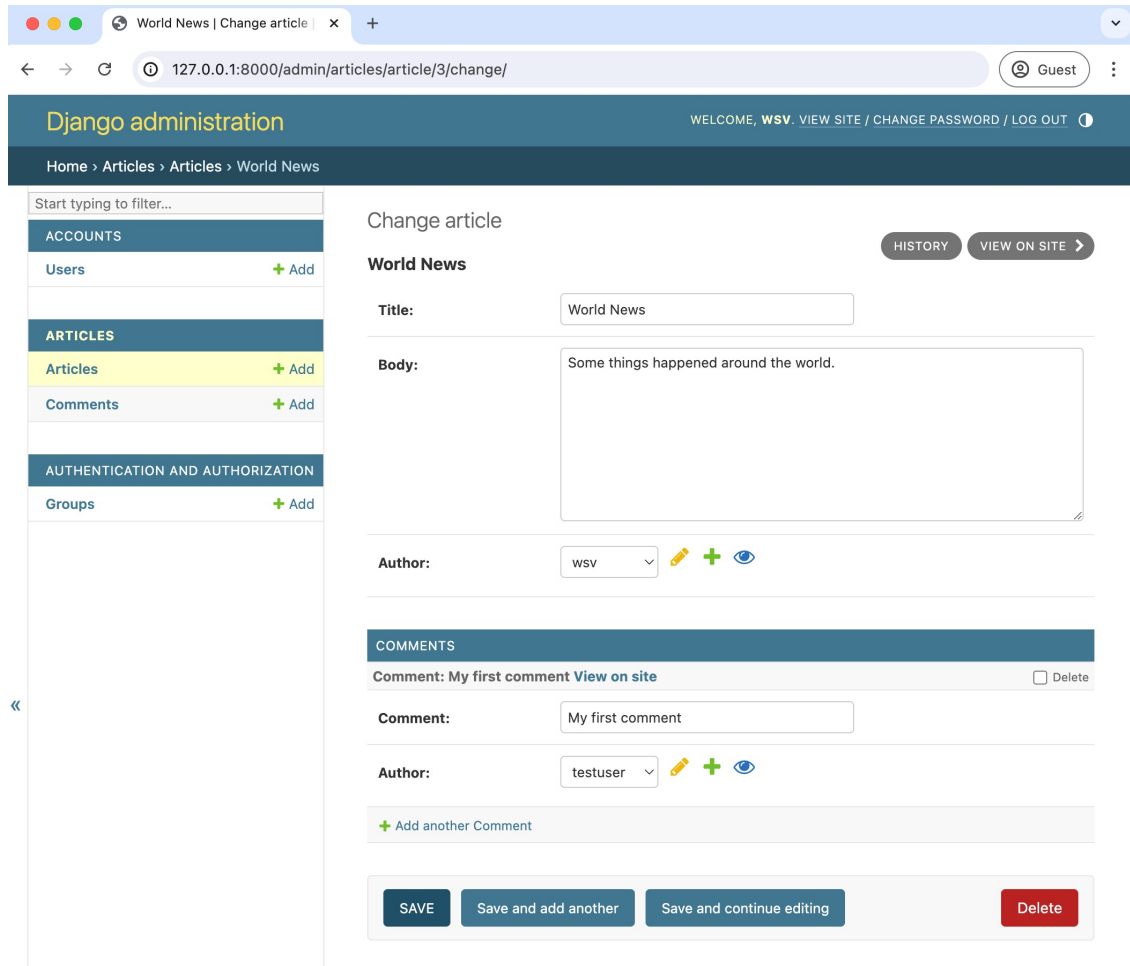
Personally, though, I prefer using `TabularInline` as it shows more information in less space: the comment, author, and more on one single line. To switch to it, we only need to change our `CommentInline` from `admin.StackedInline` to `admin.TabularInline`.

Code

```
# articles/admin.py
from django.contrib import admin
from .models import Article, Comment

class CommentInline(admin.TabularInline): # new
    model = Comment
    extra = 0 #
```

```
class CommentInline(admin.StackedInline):
    model = Comment
    extra = 0 # new
```



관리자 추가 댓글 없음

개인적으로, 나는 TabularInline을 사용하는 것을 선호하는데, 이는 더 적은 공간에 더 많은 정보를 보여주기 때문입니다: 댓글, 작성자, 그리고 한 줄에 더 많은 정보. 이를 전환하기 위해서는, CommentInline을 admin.StackedInline에서 admin.TabularInline으로 변경하기만 하면 됩니다.

코드

```
# articles/admin.py
from django.contrib import admin
from .models import Article, Comment
```

```
class CommentInline(admin.TabularInline):
    model = Comment
    extra = 0 # 새로 만들기
```

```
class ArticleAdmin(admin.ModelAdmin):
    inlines = [
        CommentInline,
    ]
    list_display = [
        "title",
        "body",
        "author",
    ]

admin.site.register(Article, ArticleAdmin)
admin.site.register(Comment)
```

Refresh the current admin page for Articles and you'll see the new change: all fields for each model are displayed on the same line.

The screenshot shows the Django administration interface for a 'World News' article. The page title is 'Django administration' and the user is 'WSV'. The breadcrumb trail is 'Home > Articles > Articles > World News'. The left sidebar shows the navigation menu with 'Articles' selected. The main content area is titled 'Change article' and contains the following form fields:

- Title:** World News
- Body:** Some things happened around the world.
- Author:** WSV

Below the form, there is a 'COMMENTS' section with a table of comments. The table has columns for 'COMMENT', 'AUTHOR', and 'DELETE?'. The first comment is 'My first comment' by 'testuser'. There is a button to 'Add another Comment'.

At the bottom of the page, there are four buttons: 'SAVE', 'Save and add another', 'Save and continue editing', and 'Delete'.

TabularInline Page

```

class ArticleAdmin(admin.ModelAdmin):
    inlines = [CommentInline,]
    list_display = ("title", "body", "author",)

admin.site.register(Article, ArticleAdmin)
admin.site.register(Comment)

```

현재 관리 페이지를 새로 고치면 다음과 같은 변경 사항이 표시됩니다: 각 모델의 모든 필드가 같은 줄에 표시됩니다.

The screenshot shows the Django administration interface for a 'World News' article. The page is titled 'Change article' and includes a sidebar with navigation links for ACCOUNTS, ARTICLES, and AUTHENTICATION AND AUTHORIZATION. The main content area contains form fields for Title, Body, and Author, and a table for comments.

Change article

World News

Title: World News

Body: Some things happened around the world.

Author: WSV

COMMENTS

COMMENT	AUTHOR	DELETE?
My first comment View on site	testuser	☐

[+ Add another Comment](#)

Buttons: SAVE, Save and add another, Save and continue editing, Delete

TabularInline 페이지

Much better. Now we need to display the comments on our website by updating our template.

Template

We want comments to appear on the articles list page and allow logged-in users to add a comment on the detail page for an article. That means updating the template files `article_list.html` and `article_detail.html`.

Let's start with `article_list.html`. If you look at the `articles/models.py` file again, it is clear that `Comment` has a foreign key relationship to the article. To display **all** comments related to a specific article, we will [follow the relationship backward](#) via a “query,” which is a way to ask the database for a specific bit of information. Django has a built-in syntax known as `F00_set` where `F00` is the lowercase source model name. So for our `Article` model, we use the syntax `{% for comment in article.comment_set.all %}` to view all related comments. And then, within this for loop, we can specify what to display, such as the comment itself and author.

Here is the updated `article_list.html` file –the changes start after the “card-body” div class.

Code

```
<!-- templates/article_list.html -->
...
<div class="card-body">
    {{ article.body }}
    {% if article.author.pk == request.user.pk %}
    <a href="{% url 'article_edit' article.pk %}">Edit</a>
    <a href="{% url 'article_delete' article.pk %}">Delete</a>
    {% endif %}
</div>
<div class="card-footer">
    {% for comment in article.comment_set.all %}
    <p>
        <span class="fw-bold">
            {{ comment.author }} &middot;
        </span>
        {{ comment }}
    </p>
    {% endfor %}
</div>
</div>
<br/>
{% endfor %}
{% endblock content %}
```

훨씬 나아졌습니다. 이제 템플릿을 업데이트하여 웹사이트에 댓글을 표시해야 합니다.

템플릿

우리는 댓글이 기사 목록 페이지에 나타나고 로그인한 사용자가 기사 세부 정보 페이지에서 댓글을 추가할 수 있도록 하기를 원합니다. 이는 `article_list.html` 및 `article_detail.html` 템플릿 파일을 업데이트해야 함을 의미합니다.

`article_list.html`부터 시작하겠습니다. `articles/models.py` 파일을 다시 살펴보면 `Comment`가 `article`에 대한 외래 키 관계를 가지고 있다는 것이 분명합니다. 특정 기사와 관련된 모든 댓글을 표시하기 위해 우리는 데이터베이스에 특정 정보를 요청하는 방법인 "쿼리"를 통해 관계를 거슬러 올라갈 것입니다. Django에는 `FOO_set`이라는 내장 구문이 있으며, 여기서 `FOO`는 소문자 소스 모델 이름입니다. 따라서 우리의 `Article` 모델에 대해 우리는 `{%for comment in article.comment_set.all %}` 구문을 사용하여 모든 관련 댓글을 볼 수 있습니다. 그리고 이 `for` 루프 내에서 댓글 자체와 작성자와 같은 표시할 내용을 지정할 수 있습니다.

업데이트된 `article_list.html` 파일입니다 - 변경 사항은 `"cardbody"` div 클래스 이후에 시작됩니다.

코드

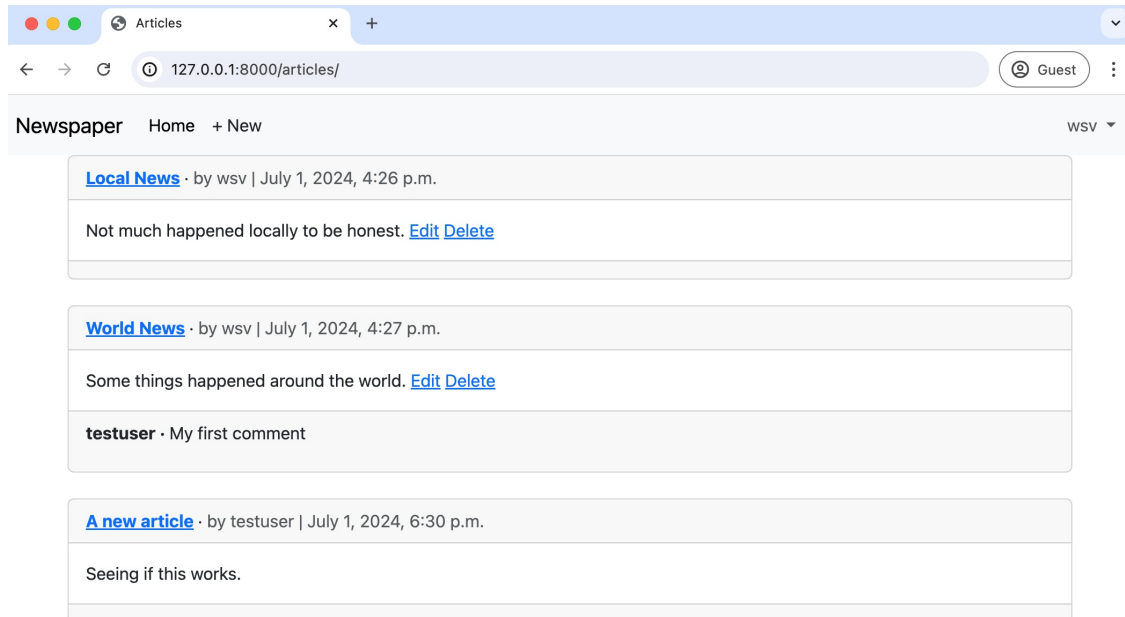
```
<!-- templates/article_list.html -->
...
<div class="card-body">
    {{ article.body }}
    {% if article.author.pk == request.user.pk %}
    <a href="{% url 'article_edit' article.pk %}">수정</a><a href="{% url
    'article_delete' article.pk %}">삭제</a>{% endif %}

</div>
<div class="card-footer">
    {% for comment in article.comment_set.all %}<p>

        <span class="fw-bold">
            {{ comment.author }} &middot;
        </span>
        {{ comment }}
    </p>{% endfor
    %}
</div>
</div><br/>{%
endfor %}

{% endblock content %}
```


If you refresh the articles page at `http://127.0.0.1:8000/articles/`, we can see our new comment on the page.



Articles Page with Comments

Let's also add comments to the detail page for each article. We'll use the same technique of following the relationship backward to access comments as a foreign key of the article model.

Code

```
<!-- templates/article_detail.html -->
{% extends "base.html" %}

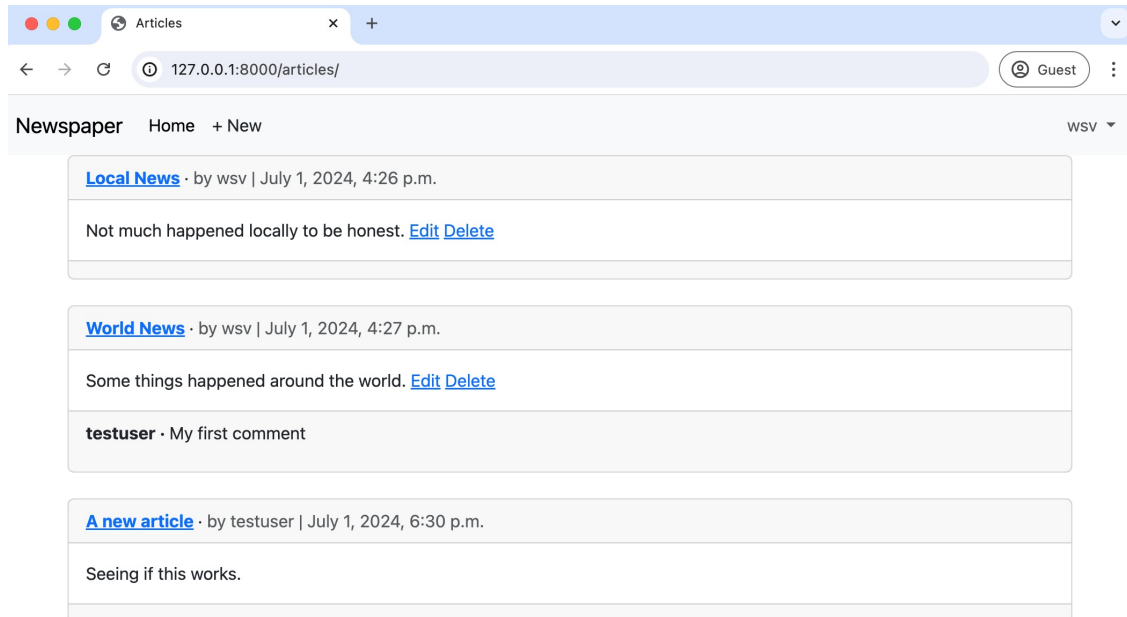
{% block content %}
<div class="article-entry">
  <h2>{{ object.title }}</h2>
  <p>by {{ object.author }} | {{ object.date }}</p>
  <p>{{ object.body }}</p>
</div>

<!-- Changes start here! -->
<hr>
<h4>Comments</h4>
{% for comment in article.comment_set.all %}
<p>{{ comment.author }} &middot; {{ comment }}</p>
{% endfor %}
<hr>
<!-- Changes end here! -->

{% if article.author.pk == request.user.pk %}
<p><a href="{% url 'article_edit' article.pk %}">Edit</a>

```

http://127.0.0.1:8000/articles/에서 기사 페이지를 새로 고치면, 페이지에서 우리의 새로운 댓글을 볼 수 있습니다.



댓글이 있는 기사 페이지

각 기사의 상세 페이지에도 댓글을 추가해 보겠습니다. 우리는 기사 모델의 외래 키로 댓글에 접근하기 위해 관계를 거슬러 올라가는 동일한 기술을 사용할 것입니다.

코드

```
<!-- templates/article_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="article-entry"><h2>
  {{ object.title }}</h2>
  <p>작성자: {{ object.author }} | {{ object.date }}</p>
  <p>{{ object.body }}</p>
</div>

<!-- 변경 사항은 여기서 시작합니다! -->
<hr><h4>댓글</h4>

{% for comment in article.comment_set.all %}<p>
  {{ comment.author }} &middot; {{ comment }}</p>{% endfor %}
<hr>
<!-- 변경 사항은 여기서 끝납니다! -->

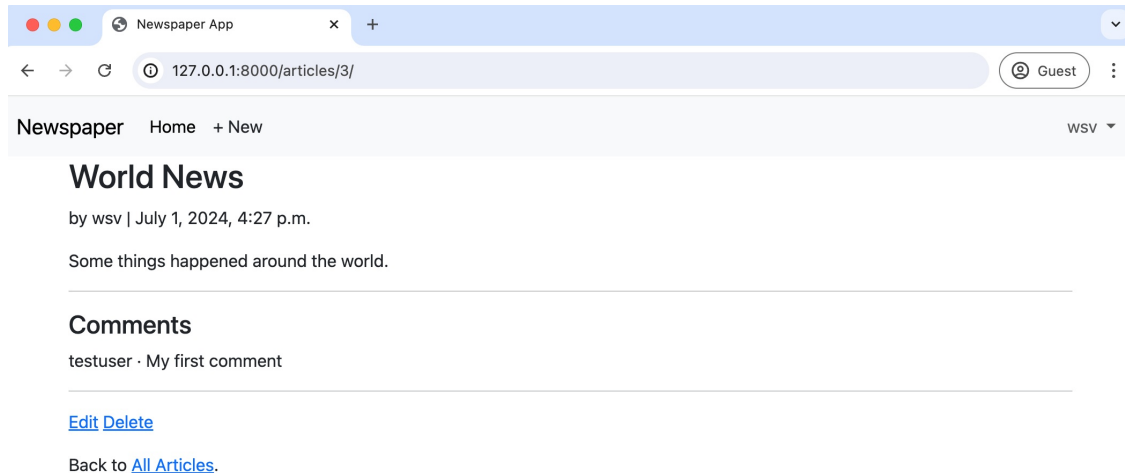
{% if article.author.pk == request.user.pk %}<p><a href="{% url
  'article_edit' article.pk %}">편집</a>
```

```

    <a href="{% url 'article_delete' article.pk %}">Delete</a>
  </p>
{% endif %}
<p>Back to <a href="{% url 'article_list' %}">All Articles</a>.</p>
{% endblock content %}

```

Navigate to the detail page of your article with a comment, and any comments will be visible.



Article Details Page with Comments

We won't win any design awards for this layout, but this is a book on Django, so our goal is to output the correct content.

Comment Form

The comments are now visible, but we need to add a form so users can add them to the website. Web forms are a very complicated topic since security is essential: any time you accept data from a user that will be stored in a database, you must be highly cautious. The good news is Django forms handle most of this work for us.

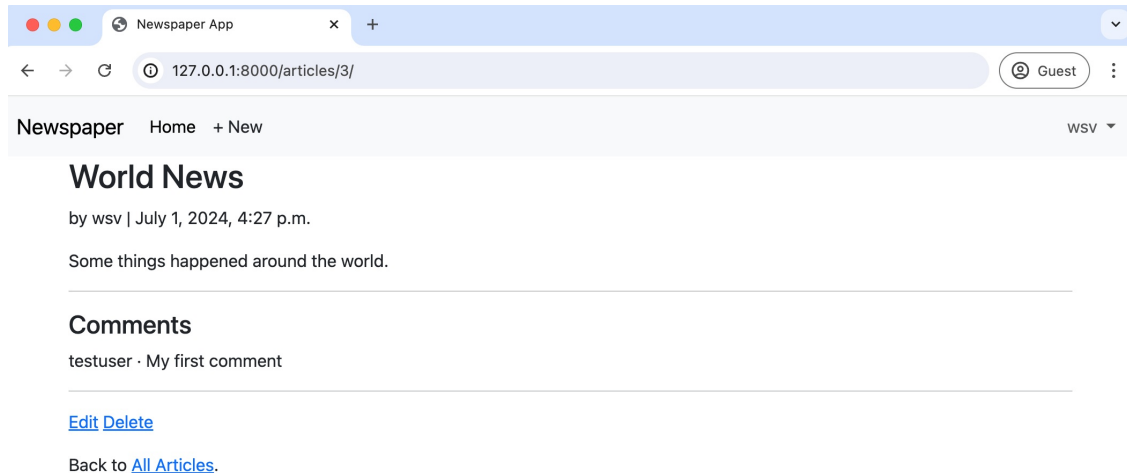
[ModelForm](#) is a helper class that translates database models into forms. We can use it to create a form called, appropriately enough, `CommentForm`. We *could* put this form in our existing `articles/models.py` file, but generally, the best practice is to put all forms in a dedicated `forms.py` file within your app. That's the approach we'll use here.

```

<a href="{% url 'article_delete' article.pk %}">삭제</a>
</p>{%
endif %}
<p>다시 <a href="{% url 'article_list' %}">모든 기사</a>로 돌아가기.</p>
{% endblock content %}

```

댓글이 있는 기사 세부 정보 페이지로 이동하면 모든 댓글이 표시됩니다.



댓글이 있는 기사 세부 정보 페이지

이 레이아웃으로 디자인 상을 받을 수는 없겠지만, 이것은 Django에 관한 책이므로 우리의 목표는 올바른 콘텐츠를 출력하는 것입니다.

댓글 양식

댓글이 이제 표시되지만, 사용자가 웹사이트에 댓글을 추가할 수 있도록 양식을 추가해야 합니다. 웹 양식은 보안이 필수적이기 때문에 매우 복잡한 주제입니다: 사용자가 데이터베이스에 저장될 데이터를 수락할 때마다 매우 주의해야 합니다. 좋은 소식은 Django 양식이 대부분의 작업을 처리해 준다는 것입니다.

`ModelForm`은 데이터베이스 모델을 양식으로 변환하는 도우미 클래스입니다. 우리는 이를 사용하여 적절하게 `CommentForm`이라는 양식을 만들 수 있습니다. 이 양식을 기존의 `articles/models.py` 파일에 넣을 수 있지만, 일반적으로 최선의 방법은 모든 양식을 앱 내의 전용 `forms.py` 파일에 넣는 것입니다. 이것이 우리가 여기서 사용할 접근 방식입니다.

With your text editor, create a new file called `articles/forms.py`. At the top, import `forms`, which has `ModelForm` as a module. Then import our model, `Comment`, since we'll need to add that, too. Finally, create the class `CommentForm`, specifying both the underlying model and the specific field to expose, `comment`. When we create the corresponding view, we will automatically set the author to the currently logged-in user.

Code

```
# articles/forms.py
from django import forms

from .models import Comment

class CommentForm(forms.ModelForm):
    class Meta:
        model = Comment
        fields = ("comment",)
```

Web forms can be incredibly complex, but Django has thankfully abstracted away much of the complexity for us.

Comment View

Currently, we rely on the generic class-based `DetailView` to power our `ArticleDetailView`. It displays individual entries but needs to be configured to add additional information like a form. Class-based views are powerful because their inheritance structure means that if we know where to look, there is often a specific module we can override to attain our desired outcome.

The one we want in this case is called `get_context_data()`. It is used to add information to a template by updating the `context`, a dictionary object containing all the variable names and values available in our template. For performance reasons, Django templates compile only once; if we want something available in the template, it must load into the context at the beginning.

What do we want to add in this case? Well, just our `CommentForm`. And since `context` is a dictionary, we must also assign a variable name. How about `form`? Here is what the new code looks like in `articles/views.py`.

Code

```
# articles/views.py
...
from .models import Article
```

텍스트 편집기를 사용하여 `articles/forms.py`라는 새 파일을 만듭니다. 맨 위에 `forms`를 가져오고, 이 모듈에는 `ModelForm`이 포함되어 있습니다. 그런 다음 모델인 `Comment`를 가져옵니다. 왜냐하면 그것도 추가해야 하니까요. 마지막으로 `CommentForm` 클래스를 생성하고, 기본 모델과 노출할 특정 필드인 `comment`를 지정합니다. 해당 뷰를 생성할 때 현재 로그인한 사용자로 저자를 자동으로 설정합니다.

코드

```
# articles/forms.py
from django import forms

from .models import Comment

class CommentForm
    (forms.ModelForm):
    class Meta:
        model = Comment
        fields = (
            "comment",
        )
```

웹 양식은 매우 복잡할 수 있지만, 다행히도 Django는 많은 복잡성을 추상화해 주었습니다.

댓글 보기

현재 우리는 `ArticleDetailView`를 지원하기 위해 일반 클래스 기반 `DetailView`에 의존하고 있습니다. 이는 개별 항목을 표시하지만 양식과 같은 추가 정보를 추가하도록 구성해야 합니다. 클래스 기반 뷰는 상속 구조 덕분에 강력합니다. 우리가 어디를 찾아야 할지 안다면, 종종 원하는 결과를 얻기 위해 재정의할 수 있는 특정 모듈이 있습니다.

이 경우 우리가 원하는 것은 `get_context_data()`라고 불립니다. 이는 템플릿에 정보를 추가하기 위해 컨텍스트를 업데이트하는 데 사용되며, 컨텍스트는 템플릿에서 사용할 수 있는 모든 변수 이름과 값이 포함된 사전 객체입니다. 성능상의 이유로 Django 템플릿은 한 번만 컴파일됩니다. 템플릿에서 사용할 수 있는 무언가가 필요하다면, 시작할 때 컨텍스트에 로드되어야 합니다.

이 경우 무엇을 추가하고 싶습니까? 음, 우리의 `CommentForm`만 추가하면 됩니다. 그리고 컨텍스트가 사전이기 때문에 변수 이름도 할당해야 합니다. `form`은 어떻습니까? 새 코드가 `articles/views.py`에서 어떻게 보이는지 여기 있습니다.코드

```
# articles/views.py
...
from .models import Article
```

```

from .forms import CommentForm # new

class ArticleDetailView(LoginRequiredMixin, DetailView):
    model = Article
    template_name = "article_detail.html"

    def get_context_data(self, **kwargs): # new
        context = super().get_context_data(**kwargs)
        context["form"] = CommentForm()
        return context

```

Near the top of the file, just above `from .models import Article`, we added an import line for `CommentForm` and then updated the module for `get_context_data()`. First, we pulled all existing information into the context using `super()`, added the variable name `form` with the value of `CommentForm()`, and returned the updated context.

Comment Template

To display the form in our `article_detail.html` template file, we'll rely on the form variable and `crispy forms`. This pattern is the same as what we've done before in our other forms. At the top, load `crispy_forms_tags`, create a standard-looking post form that uses a `csrf_token` for security, and display our form fields via `{{ form|crispy }}`.

Code

```

<!-- templates/article_detail.html -->
{% extends "base.html" %}
{% load crispy_forms_tags %} <!-- new! -->

{% block content %}
<div class="article-entry">
    <h2>{{ object.title }}</h2>
    <p>by {{ object.author }} | {{ object.date }}</p>
    <p>{{ object.body }}</p>
</div>

<hr>
<h4>Comments</h4>
{% for comment in article.comment_set.all %}
    <p>{{ comment.author }} &middot; {{ comment }}</p>
{% endfor %}
<hr>

<!-- Changes start here! -->
<h4>Add a comment</h4>
<form action="" method="post">{% csrf_token %}
    {{ form|crispy }}
    <button class="btn btn-success ms-2" type="submit">Save</button>
</form>
<!-- Changes end here! -->

```

```
from .forms import CommentForm # new
```

```
class ArticleDetailView(LoginRequiredMixin, DetailView): model = Article
```

```
    template_name = "article_detail.html"
```

```
    def get_context_data(self, **kwargs): # new
        context = super().get_context_data(**kwargs)
        context["form"] = CommentForm()
        return context
```

파일의 맨 위, `from .models import Article` 바로 위에 `CommentForm`을 위한 `import` 라인을 추가하고 `forget_context_data()` 모듈을 업데이트했습니다. 먼저, `super()`를 사용하여 모든 기존 정보를 `context`로 가져오고, `CommentForm()`의 값을 가진 변수 이름 `form`을 추가한 후 업데이트된 `context`를 반환했습니다.

댓글 템플릿

`article_detail.html` 템플릿 파일에 양식을 표시하기 위해 `form` 변수와 `crispy forms`에 의존할 것입니다. 이 패턴은 이전의 다른 양식에서 했던 것과 동일합니다. 맨 위에서 `crispy_form_tags`를 로드하고 보안을 위해 `csrf_token`을 사용하는 표준 포스트 양식을 생성한 다음 `{{ form|crispy }}`를 통해 양식 필드를 표시합니다.

코드

```
<!-- templates/article_detail.html -->
{% extends "base.html" %}
{% load crispy_forms_tags %} <!-- 새로 추가! -->{%
block content %}

<div class="article-entry">
    <h2>{{ object.title }}</h2>
    <p>작성자 {{ object.author }} | {{ object.date }}</p>
    <p>{{ object.body }}</p>
</div>

<hr><h4>댓글</h4>

{% for comment in article.comment_set.all %}<p>
    {{ comment.author }} &middot; {{ comment }}</p>
{% endfor %}
<hr>

<!-- 변경 사항 시작! -->
<h4>댓글 추가</h4>
<form action="" method="post">{% csrf_token %}{{ form|
    crispy }}
    <button class="btn btn-success ms-2" type="submit">저장</button></form>

<!-- 변경 사항 끝! -->
```



```

<div>
  {% if article.author.pk == request.user.pk %}
  <p><a href="{% url 'article_edit' article.pk %}">Edit</a>
    <a href="{% url 'article_delete' article.pk %}">Delete</a>
  </p>
  {% endif %}
  <p>Back to <a href="{% url 'article_list' %}">All Articles</a>.</p>
</div>
{% endblock content %}

```

If you refresh the detail page, the form is now displayed with familiar Bootstrap and crispy forms styling.

Newspaper App

127.0.0.1:8000/articles/3/

Guest

Newspaper Home + New WSV

World News

by wsv | July 1, 2024, 4:27 p.m.

Some things happened around the world.

Comments

testuser · My first comment

Add a comment

Comment*

Save

[Edit](#) [Delete](#)

Back to [All Articles](#).

Form Displayed

Success! However, we are only half done. If you attempt to submit the form, you'll receive an error because our view doesn't yet support any POST methods!

Comment Post View

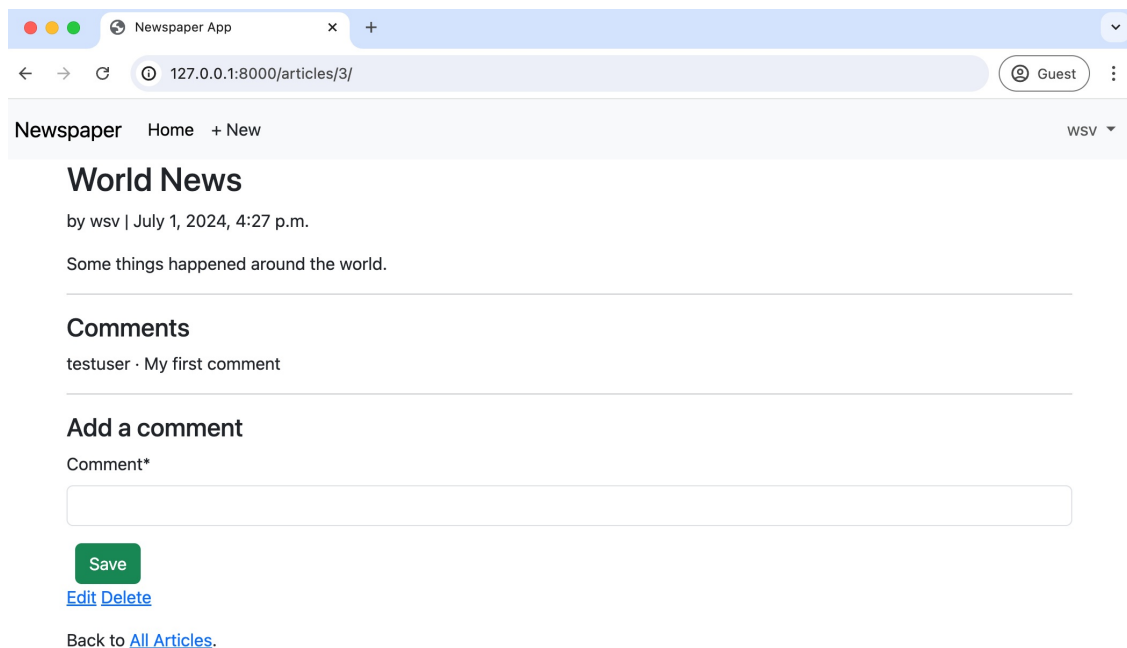
We ultimately need a view that handles both GET and POST requests depending upon whether the form should be merely displayed or capable of being submitted. We could reach for [FormMixin](#) to combine both into our `ArticleDetailView`, but as the [Django docs illustrate quite well](#), there are risks with this approach.

```

<div>
    {% if article.author.pk == request.user.pk %}
    <p><a href="{% url 'article_edit' article.pk %}">수정</a><a href="{% url
      'article_delete' article.pk %}">삭제</a>
    </p>{% endif %}
    <p>다시 <a href="{% url 'article_list' %}">모든 기사</a>로 돌아가기.</p>
</div>
{% endblock content %}

```

상세 페이지를 새로 고치면, 양식이 이제 익숙한 Bootstrap과 crispy forms 스타일로 표시됩니다.



양식 표시됨

성공! 그러나 우리는 아직 절반만 완료했습니다. 양식을 제출하려고 하면, 우리의 뷰가 아직 POST 메서드를 지원하지 않기 때문에 오류가 발생합니다!

댓글 게시물 보기

궁극적으로 우리는 양식이 단순히 표시되어야 하는지 아니면 제출할 수 있어야 하는지에 따라 GET 및 POST 요청을 모두 처리하는 뷰가 필요합니다. 우리는 FormMixin을 사용하여 이를 ArticleDetailView에 결합할 수 있지만, Django 문서에서 잘 설명하듯이 이 접근 방식에는 위험이 있습니다.

To avoid subtle interactions between `DetailView` and `FormMixin`, we will separate the GET and POST variations into their dedicated views. We can then transform `ArticleDetailView` into a *wrapper* view that combines them. This is a very common pattern in more advanced Django development because a single URL must often behave differently based on the user request (GET, POST, etc.) or even the format (returning HTML vs. JSON).

Let's start by renaming `ArticleDetailView` into `CommentGet` since it handles GET requests but not POST requests. We'll then create a new `CommentPost` view that is empty for now. And we can combine both `CommentPost` and `CommentGet` into a new `ArticleDetailView` that subclasses [View](#), the foundational class upon which all other class-based views are built.

Code

```
# articles/views.py
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.views import View # new
from django.views.generic import ListView, DetailView
...

class CommentGet(DetailView): # new
    model = Article
    template_name = "article_detail.html"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["form"] = CommentForm()
        return context

class CommentPost(): # new
    pass

class ArticleDetailView(LoginRequiredMixin, View): # new
    def get(self, request, *args, **kwargs):
        view = CommentGet.as_view()
        return view(request, *args, **kwargs)

    def post(self, request, *args, **kwargs):
        view = CommentPost.as_view()
        return view(request, *args, **kwargs)

...
```

Navigate back to the homepage in your web browser and then reload the article page with a comment. Everything should work as before.

DetailView와 FormMixin 간의 미세한 상호작용을 피하기 위해 GET 및 POST 변형을 전용 뷰로 분리할 것입니다. 그런 다음 ArticleDetailView를 이들을 결합하는 래퍼 뷰로 변환할 수 있습니다. 이는 단일 URL이 사용자 요청(GET, POST 등)이나 형식(HTML 반환 vs. JSON)에 따라 다르게 동작해야 하는 경우가 많기 때문에 더 고급 Django 개발에서 매우 일반적인 패턴입니다.

먼저 ArticleDetailView의 이름을 CommentGet으로 변경하겠습니다. 이는 GET 요청을 처리하지만 POST 요청은 처리하지 않기 때문입니다. 그런 다음 현재는 비어 있는 새로운 CommentPost 뷰를 생성하겠습니다. 그리고 CommentPost와 CommentGet을 결합하여 View를 서브클래스하는 새로운 ArticleDetailView를 만들 수 있습니다. View는 모든 다른 클래스 기반 뷰가 구축되는 기본 클래스입니다.

코드

```
# articles/views.py
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.views import View
from django.views.generic import ListView, DetailView...

class CommentGet(DetailView): # new
    model = Article
    template_name = "article_detail.html"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["form"] = CommentForm()
        return context

class CommentPost(): # new
    pass

class ArticleDetailView(LoginRequiredMixin, View): # new
    def get(self, request, *args, **kwargs):
        view = CommentGet.as_view()
        return view(request, *args, **kwargs)

    def post(self, request, *args, **kwargs):
        view = CommentPost.as_view()
        return view(request, *args, **kwargs)...
```

웹 브라우저에서 홈페이지로 돌아가서 댓글이 있는 기사 페이지를 새로 고치세요. 모든 것이 이전과 같이 작동해야 합니다.

We're ready to write `CommentPost` and complete the task of adding comments to our website. We are almost done!

[FormView](#) is a built-in view that displays a form, any validation errors, and redirects to a new URL. We will use it with [SingleObjectMixin](#) to associate the current article with our form; in other words, if you have a comment at `articles/4/`, as I do in the screenshots, then `SingleObjectMixin` will grab the 4 so that our comment is saved to the article with a pk of 4.

Here is the complete code, which we'll run through below line-by-line.

Code

```
# articles/views.py
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.views import View
from django.views.generic import ListView, DetailView, FormView # new
from django.views.generic.detail import SingleObjectMixin # new
from django.views.generic.edit import UpdateView, DeleteView, CreateView
from django.urls import reverse_lazy, reverse # new

from .forms import CommentForm
from .models import Article

...
class CommentPost(SingleObjectMixin, FormView): # new
    model = Article
    form_class = CommentForm
    template_name = "article_detail.html"

    def post(self, request, *args, **kwargs):
        self.object = self.get_object()
        return super().post(request, *args, **kwargs)

    def form_valid(self, form):
        comment = form.save(commit=False)
        comment.article = self.object
        comment.author = self.request.user
        comment.save()
        return super().form_valid(form)

    def get_success_url(self):
        article = self.object
        return reverse("article_detail", kwargs={"pk": article.pk})

...
```

At the top, import `FormView`, `SingleObjectMixin`, and `reverse`. `FormView` relies on `form_class` to set the form name we're using, `CommentForm`. First up is `post()`: we use `get_object()` from `SingleObjectMixin` to grab the article pk from the URL. Next is `form_valid()`, which is called when form validation has succeeded. Before we save our comment to the database, we must specify the

우리는 CommentPost를 작성할 준비가 되었고, 웹사이트에 댓글을 추가하는 작업을 완료할 것입니다. 거의 다 끝났습니다!

[FormView](#)는 양식을 표시하고, 모든 유효성 검사 오류를 보여주며, 새 URL로 리디렉션하는 내장 뷰입니다. [우리는 이를 SingleObjectMixin과 함께 사용하여 현재 기사를 우리의 양식과 연결할 것입니다.](#) 다시 말해, `articles/4/`에 댓글이 있다면, 스크린샷에서 보듯이 SingleObjectMixin은 4를 가져와서 우리의 댓글이 pk가 4인 기사에 저장되도록 합니다.

여기 아래에서 한 줄씩 실행할 전체 코드가 있습니다.

코드

```
# articles/views.py
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.views import View
from django.views.generic import ListView, DetailView, FormView # new
from django.views.generic.detail import SingleObjectMixin # new
from django.views.generic.edit import UpdateView, DeleteView, CreateView
from django.urls import reverse_lazy, reverse # new

from .forms import CommentForm
from .models import Article

...
class CommentPost(SingleObjectMixin, FormView): # new
    model = Article
    form_class = CommentForm
    template_name = "article_detail.html"

    def post(self, request, *args, **kwargs):
        self.object = self.get_object()
        return super().post(request, *args, **kwargs)

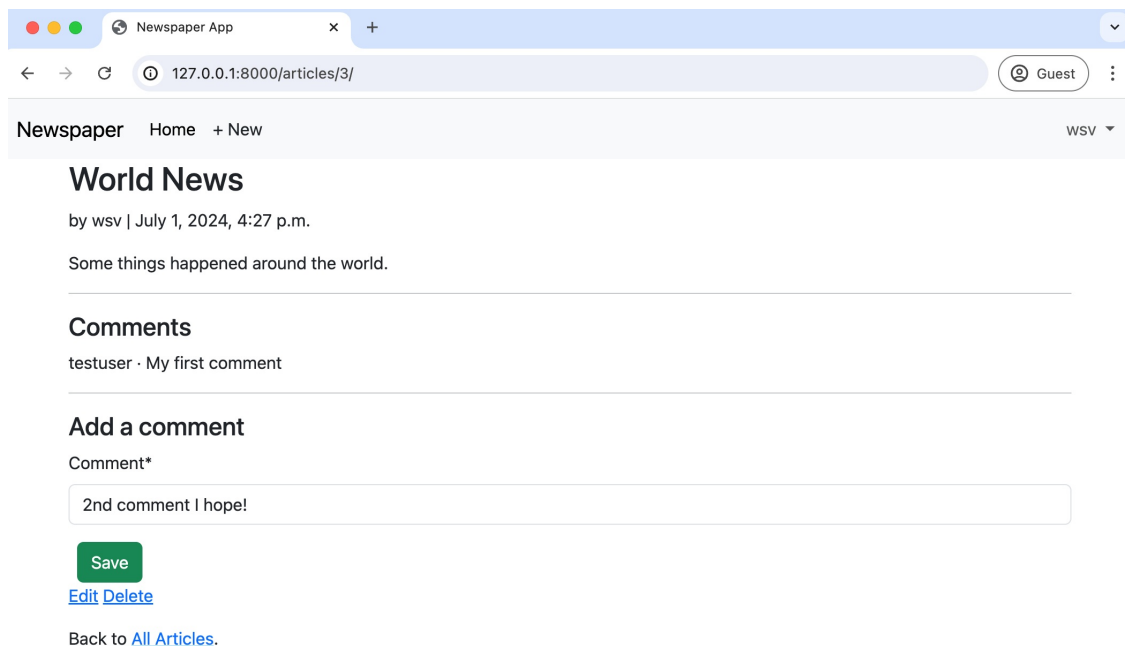
    def form_valid(self, form):
        comment = form.save(commit=False)
        comment.article = self.object
        comment.author = self.request.user
        comment.save()
        return super().form_valid(form)

    def get_success_url(self):
        article = self.object
        return reverse("article_detail", kwargs={"pk": article.pk})...
```

맨 위에서 FormView, SingleObjectMixin 및 reverse를 가져옵니다. FormView는 사용 중인 양식 이름인 CommentForm을 설정하기 위해 form_class에 의존합니다. 첫 번째는 post(): SingleObjectMixin의 get_object()를 사용하여 URL에서 기사 pk를 가져옵니다. 다음은 form_valid()로, 양식 검증이 성공했을 때 호출됩니다. 데이터베이스에 댓글을 저장하기 전에 반드시 명시해야 합니다.

article it belongs to. Initially, we save the form but set `commit` to `False` because we associate the correct article with the form object in the next line. We also set the `author` field in our `Comment` model to the current user. In the following line, we save the form. Finally, we return it as part of `form_valid()`. The final module, `get_success_url()`, is called after the form data is saved; we redirect the user to the current page.

And we're done! Go ahead and load your articles page now, refresh the page, then try to submit a second comment.



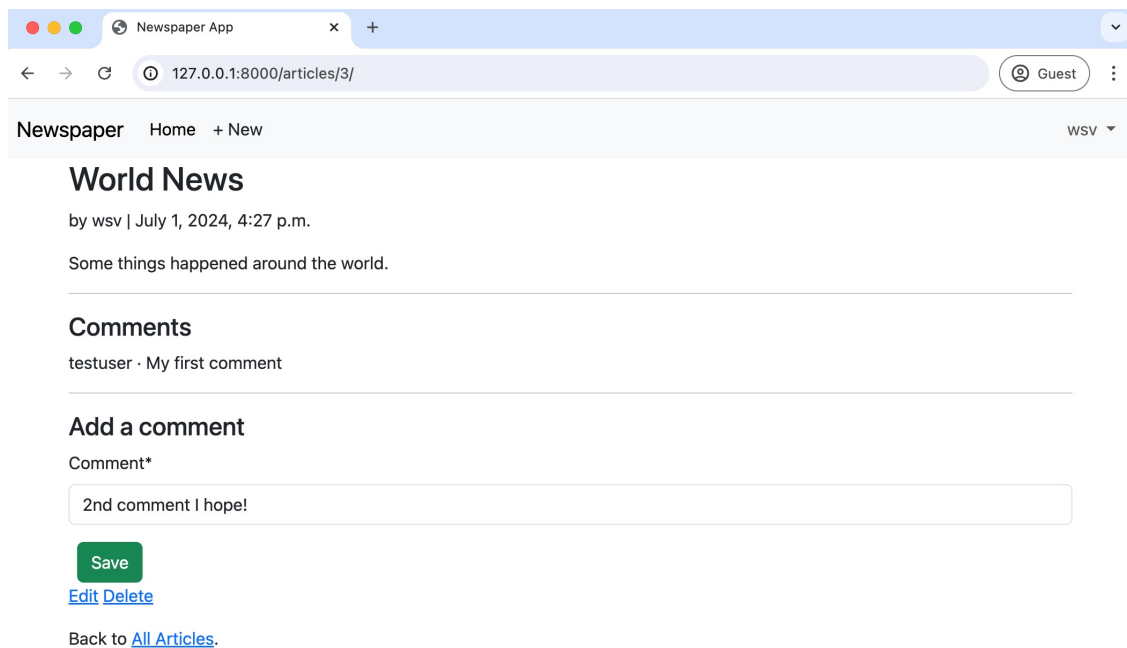
The screenshot shows a web browser window titled "Newspaper App" with the address bar displaying "127.0.0.1:8000/articles/3/". The page layout includes a navigation bar with "Newspaper", "Home", and "+ New" links, and a user profile "Guest". The main content area features an article titled "World News" by "wsv" dated "July 1, 2024, 4:27 p.m.", with the text "Some things happened around the world." Below the article is a "Comments" section showing a single comment by "testuser" with the text "My first comment". Underneath is an "Add a comment" form with a label "Comment*", a text input field containing "2nd comment I hope!", a green "Save" button, and links for "Edit" and "Delete". At the bottom of the form area is a link "Back to [All Articles](#)".

Submit Comment in Form

It should automatically reload the page with the new comment displayed like this:

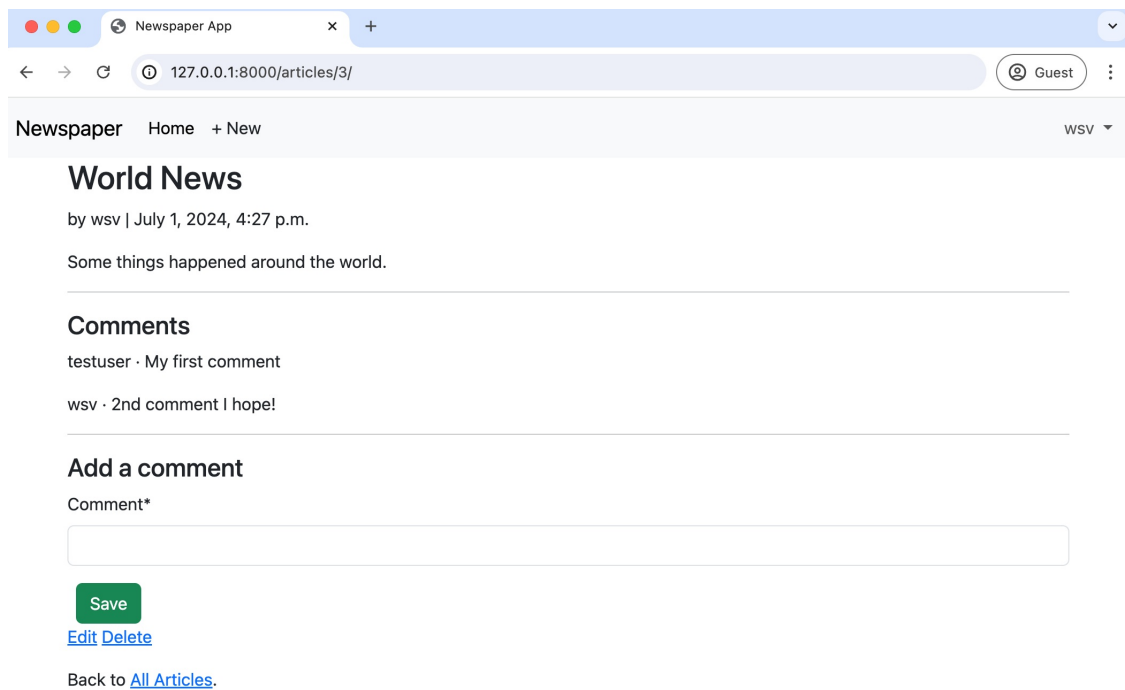
이 기사가 속한 것입니다. 처음에 우리는 양식을 저장하지만 커밋을 False로 설정합니다. 이는 다음 줄에서 양식 객체와 올바른 기사를 연결하기 위함입니다. 또한 우리의 댓글 모델에서 저자 필드를 현재 사용자로 설정합니다. 다음 줄에서 우리는 양식을 저장합니다. 마지막으로, 우리는 `form_valid()`의 일부로 그것을 반환합니다. 최종 모듈인 `get_success_url()`은 양식 데이터가 저장된 후 호출됩니다; 우리는 사용자를 현재 페이지로 리디렉션합니다.

이제 끝났습니다! 이제 기사의 페이지를 로드하고, 페이지를 새로 고친 다음, 두 번째 댓글을 제출해 보세요.



양식에서 댓글 제출

새 댓글이 다음과 같이 표시되면서 페이지가 자동으로 새로 고쳐져야 합니다:



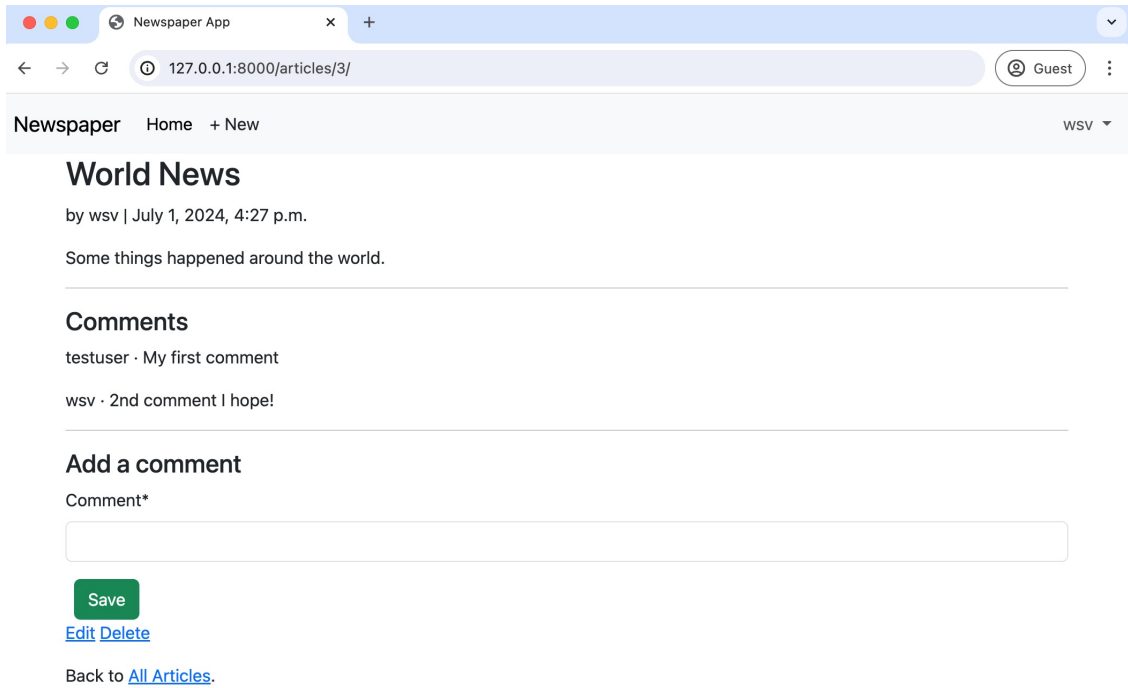
Comment Displayed

New Comment Link

Although you can add a comment to an article from its detail page, it is far more likely that a reader will want to comment on something from the all articles page. They can do that if they know to click on the detail link but that is not very good user design. Let's add a "New Comment" link to each article listed; it will navigate to the detail page but allow for that functionality. Do so by adding one line to the card-body section outside the if/endif loop.

Code

```
<!-- templates/article_list.html -->
...
<div class="card-body">
  <p>{{ article.body }}</p>
  {% if article.author.pk == request.user.pk %}
    <a href="{% url 'article_edit' article.pk %}">Edit</a>
    <a href="{% url 'article_delete' article.pk %}">Delete</a>
  {% endif %}
  <a href="{% article.get_absolute_url %}">New Comment</a> <!-- new -->
</div>
```



댓글 표시됨

새 댓글 링크

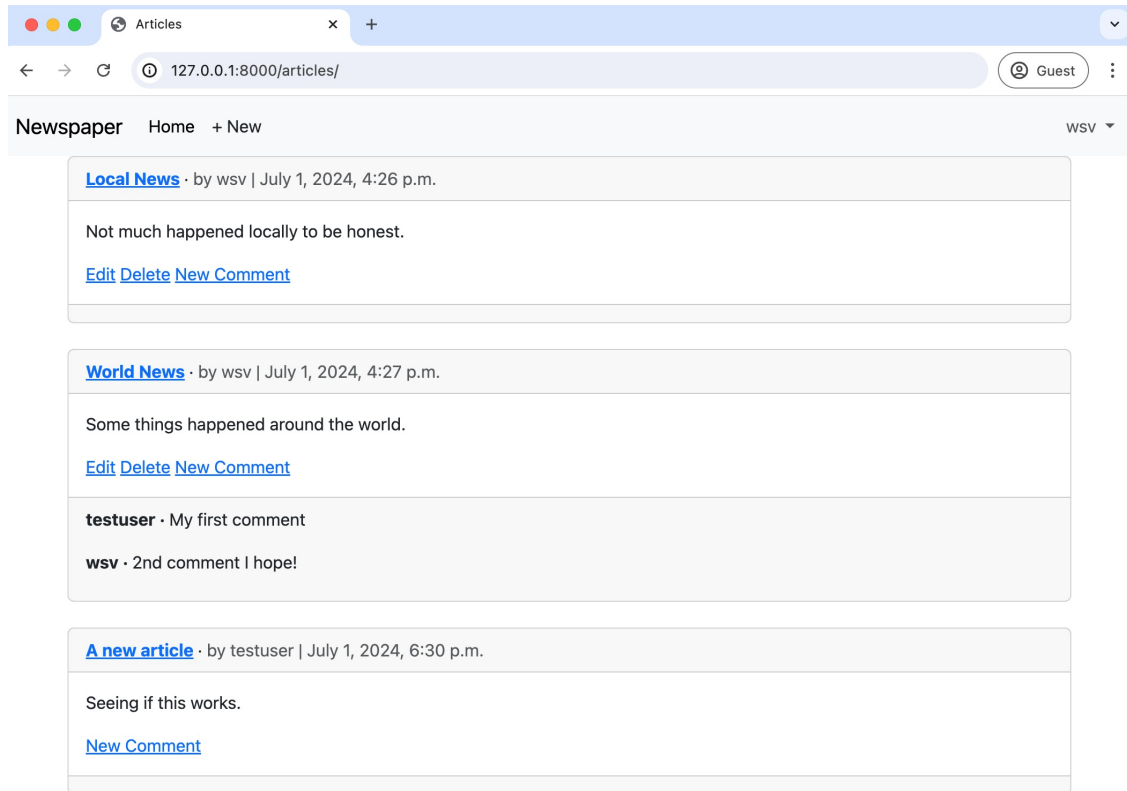
기사의 상세 페이지에서 댓글을 추가할 수 있지만, 독자가 모든 기사 페이지에서 무언가에 댓글을 달고 싶어할 가능성이 훨씬 더 높습니다. 그들은 상세 링크를 클릭해야 한다는 것을 알면 그렇게 할 수 있지만, 이는 사용자 디자인이 그리 좋지 않습니다. 각 기사에 '새 댓글' 링크를 추가합시다; 이는 상세 페이지로 이동하지만 그 기능을 허용할 것입니다. 그렇게 하려면 if/endif 루프 외부의 card-body 섹션에 한 줄을 추가하세요. 코드

```
<!-- templates/article_list.html -->
```

```
...
```

```
<div class="card-body"><p>
  {{ article.body }}</p>
  {% if article.author.pk == request.user.pk %}<a href="{% url
    'article_edit' article.pk %}">편집</a>
    <a href="{% url 'article_delete' article.pk %}">삭제</a>
  {% endif %}
  <a href="{% article.get_absolute_url %}">새 댓글</a> <!-- new --></div>
```

Refresh the all articles webpage to see the change and then click the “New Comment” link to confirm it works as expected.



New Comment on All Articles Page

Git

We added quite a lot of code in this chapter. Let's make sure to save our work before the upcoming final chapter.

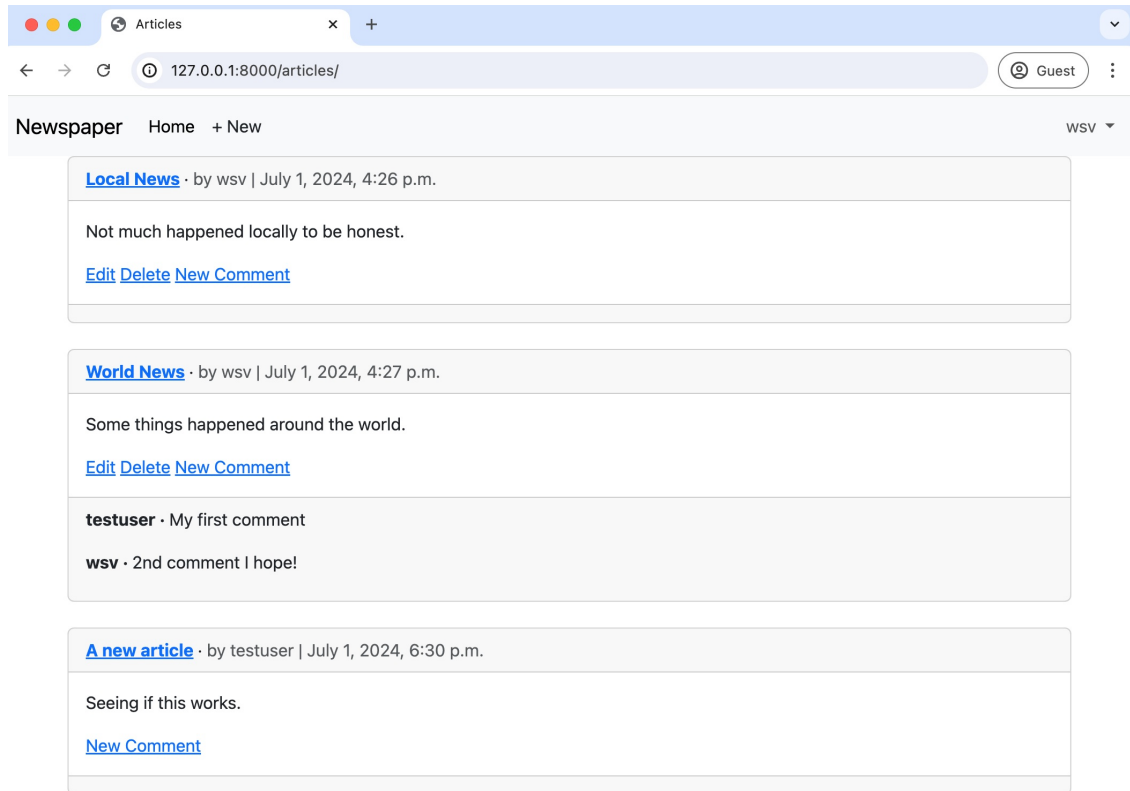
Shell

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "comments app"
(.venv) $ git push origin main
```

Conclusion

Our *Newspaper* app is now complete. It has a robust user authentication flow that uses a custom user model, articles, comments, and improved styling with

모든 기사 웹페이지를 새로 고쳐 변경 사항을 확인한 후 "NewComment" 링크를 클릭하여 예상대로 작동하는지 확인하세요.



모든 기사 페이지에 대한 새 댓글

Git

이번 장에서는 꽤 많은 코드를 추가했습니다. 다가오는 마지막 장 전에 작업을 저장하는 것을 잊지 맙시다.

Shell

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "comments app"
(.venv) $ git push origin main
```

결론

우리의 신문 앱이 이제 완성되었습니다. 사용자 정의 사용자 모델, 기사, 댓글 및 개선된 스타일링을 사용하는 강력한 사용자 인증 흐름이 있습니다.

Bootstrap. We even dipped our toes into permissions and authorizations.

The remaining task is to deploy it online. In the next chapter, we'll see how to properly deploy a Django site using environment variables, PostgreSQL, and additional settings.

부트스트랩. 우리는 심지어 권한 및 인증에 발을 담갔습니다.

남은 작업은 이를 온라인에 배포하는 것입니다. 다음 장에서는 환경 변수, PostgreSQL 및 추가 설정을 사용하여 Django 사이트를 올바르게 배포하는 방법을 살펴보겠습니다.

Chapter 16: Deployment

There is a fundamental tension between the ease of use desired in a local Django development environment and the security and performance necessary in production. Django is designed to make web developers' lives easier, and therefore, it defaults to a local configuration when the `startproject` command is first run. We've seen this in the use of SQLite as the local file-based database, the built-in `runserver` command that launches a local web server in your web browser, and various default configurations in the `settings.py` file, including `DEBUG` set to `True` and an auto-generated `SECRET_KEY`.

In production, things are different. All the configurations optimized for ease of use in the development environment need to focus instead on security, performance, and scalability.

Deploying a Django website to production requires several steps—so many, in fact, that the Django docs even have a [deployment checklist](#). It is a very helpful tool, but unfortunately, it is not sufficient. Additional factors include various hosting options, environment variables, database configurations, handling static files, and more.

In this chapter, we will create a deployment checklist and deploy the Newspaper project using Heroku. The techniques covered will apply to almost any Django website that needs to be readied for production, regardless of the hosting platform.

Hosting Options

If you ask five Django developers for the best hosting option, you'll likely receive five different answers. Everyone has a different preference based on their own experience and project needs.

Ultimately, though, we can divide hosting options into three main categories:

- **1. Dedicated Server:** a physical server sitting in a data center that belongs exclusively to you. Generally, only the largest companies adopt this approach since it requires a lot of technical expertise to configure and maintain.

16장: 배포

로컬 Django 개발 환경에서 원하는 사용의 용이성과 프로덕션에서 필요한 보안 및 성능 사이에는 근본적인 긴장이 존재합니다. Django는 웹 개발자의 삶을 더 쉽게 만들기 위해 설계되었으며, 따라서 `startproject` 명령이 처음 실행될 때 로컬 구성으로 기본 설정됩니다. 우리는 SQLite를 로컬 파일 기반 데이터베이스로 사용하는 것, 웹 브라우저에서 로컬 웹 서버를 시작하는 내장 `runserver` 명령, `DEBUG`가 `True`로 설정되고 자동 생성된 `SECRET_KEY`를 포함한 `settings.py` 파일의 다양한 기본 구성에서 이를 보았습니다.

프로덕션에서는 상황이 다릅니다. 개발 환경에서 사용의 용이성을 위해 최적화된 모든 구성은 대신 보안, 성능 및 확장성에 집중해야 합니다.

Django 웹사이트를 프로덕션에 배포하려면 여러 단계가 필요합니다. 사실, 너무 많은 단계가 있어서 Django 문서에는 [배포 체크리스트](#)가 있습니다. 매우 유용한 도구이지만, 불행히도 충분하지는 않습니다. 추가적인 요소로는 다양한 호스팅 옵션, 환경 변수, 데이터베이스 구성, 정적 파일 처리 등이 있습니다.

이 장에서는 배포 체크리스트를 만들고 Heroku를 사용하여 Newspaper 프로젝트를 배포할 것입니다. 다루는 기술은 호스팅 플랫폼에 관계없이 프로덕션 준비가 필요한 거의 모든 Django 웹사이트에 적용됩니다.

호스팅 옵션

다섯 명의 Django 개발자에게 최고의 호스팅 옵션을 물어보면, 아마도 다섯 가지 다른 답변을 받을 것입니다. 각자는 자신의 경험과 프로젝트 요구에 따라 다른 선호도를 가지고 있습니다.

궁극적으로, 우리는 호스팅 옵션을 세 가지 주요 범주로 나눌 수 있습니다:

- 1. 전용 서버: 데이터 센터에 위치한 물리적 서버로, 전적으로 귀하에게 속합니다. 일반적으로, 이 접근 방식을 채택하는 것은 가장 큰 회사들뿐입니다. 이는 구성 및 유지 관리에 많은 기술 전문 지식이 필요하기 때문입니다.

- **2. Virtual Private Server (VPS):** a server can be divided into multiple virtual machines that use the same hardware, which is much more affordable than a dedicated server. This approach also means you don't have to worry about maintaining the hardware.
- **3. Platform as a Service (PaaS):** a managed VPS solution that is pre-configured and maintained, making it the fastest option to deploy and scale a website. It typically comes with a managed database, as well. The downside is that a developer cannot access the same degree of customization that a VPS or a dedicated server provides. At scale, PaaS can become quite expensive.

The choices here are all about tradeoffs. Many Django developers and small companies are happy using a PaaS to essentially abstract away many of the difficulties inherent in putting code into production. Popular PaaS options include Heroku, Fly, and Render, among many others. For a VPS, Digital Ocean has the cleanest interface for a solo developer or small team, but for enterprise applications, the choice is typically between AWS, Google, or Microsoft.

For our Newspaper website, we will use a PaaS, specifically Heroku, because it is mature, widely used, and has a relatively straightforward deployment process. However, with the exception of creating a Heroku-specific `Procfile` file at the end, the steps outlined in this chapter will work with any PaaS provider.

Web Servers and WSGI/ASGI Servers

The local server provided by Django and run via `runserver` handles multiple jobs that must be handled differently in production. First, it acts as a *web server*, software that sits in front of our Django application to process HTTP requests and responses. It *also* manages static file requests. Before Platforms-as-a-Service became available, it was up to the web developer to install, configure, and maintain a dedicated web server like [Nginx](#) or [Apache](#): no small task and requiring a completely different skill set than web development. Fortunately, a Platform-as-a-Service knows we are deploying a website and automatically bundles a web server, generally Nginx, so we don't have to install or manage it ourselves.

The other role that `runserver` has provided us is acting as an application server to help Django generate dynamic content. When a request comes in, `runserver` powers that request through URLs, views, models, the database, and templates

- 2. 가상 사설 서버 (VPS): 서버는 동일한 하드웨어를 사용하는 여러 가상 머신으로 나눌 수 있으며, 이는 전용 서버보다 훨씬 더 저렴합니다. 이 접근 방식은 하드웨어 유지 관리에 대해 걱정할 필요가 없다는 것을 의미합니다.
- 3. 서비스로서의 플랫폼 (PaaS): 사전 구성되고 유지 관리되는 관리형 VPS 솔루션으로, 웹사이트를 배포하고 확장하는 가장 빠른 옵션입니다. 일반적으로 관리형 데이터베이스도 함께 제공됩니다. 단점은 개발자가 VPS 또는 전용 서버가 제공하는 동일한 수준의 사용자 정의에 접근할 수 없다는 것입니다. 규모가 커지면 PaaS는 상당히 비쌀 수 있습니다.

여기서의 선택은 모두 트레이드오프에 관한 것입니다. 많은 Django 개발자와 소규모 기업들은 코드 배포에 내재된 많은 어려움을 본질적으로 추상화하기 위해 PaaS를 사용하는 것에 만족합니다. 인기 있는 PaaS 옵션으로는 Heroku, Fly, Render 등이 있습니다. VPS의 경우, Digital Ocean은 솔로 개발자나 소규모 팀을 위한 가장 깔끔한 인터페이스를 제공하지만, 기업 애플리케이션의 경우 선택은 일반적으로 AWS, Google 또는 Microsoft 사이에서 이루어집니다.

우리의 신문 웹사이트를 위해 우리는 PaaS, 특히 Heroku를 사용할 것입니다. 이는 성숙하고 널리 사용되며 상대적으로 간단한 배포 프로세스를 가지고 있기 때문입니다. 그러나 마지막으로 Heroku 전용 Procfile 파일을 생성하는 것을 제외하면, 이 장에서 설명한 단계는 모든 PaaS 제공업체와 함께 작동합니다.

웹 서버 및 WSGI/ASGI 서버

Django에서 제공하고 runserver를 통해 실행되는 로컬 서버는 프로덕션에서 다르게 처리해야 하는 여러 작업을 처리합니다. 첫째, 이는 웹 서버로 작용하며, HTTP 요청 및 응답을 처리하기 위해 우리의 Django 애플리케이션 앞에 위치하는 소프트웨어입니다. 또한 정적 파일 요청을 관리합니다. 서비스로서의 플랫폼이 제공되기 전에는 웹 개발자가 Nginx 또는 Apache와 같은 전용 웹 서버를 설치, 구성 및 유지 관리해야 했습니다: 이는 작은 작업이 아니며 웹 개발과는 완전히 다른 기술 세트를 요구합니다. 다행히도, 서비스로서의 플랫폼은 우리가 웹사이트를 배포하고 있다는 것을 알고 있으며, 일반적으로 Nginx인 웹 서버를 자동으로 번들로 제공하므로 우리가 직접 설치하거나 관리할 필요가 없습니다.

runserver가 제공하는 또 다른 역할은 Django가 동적 콘텐츠를 생성하는 데 도움을 주는 애플리케이션 서버로 작용하는 것입니다. 요청이 들어오면 runserver는 해당 요청을 URL, 뷰, 모델, 데이터베이스 및 템플릿을 통해 처리합니다.

and then generates an HTTP response. In other words, runserver also acts as an *application server*, not just a web server.

Application servers are colloquially referred to as “WSGI servers” because they use WSGI to connect Python web apps to a server. In the early days of web development, web frameworks didn’t implicitly work well with various web servers without a lot of customization. For Python web frameworks, this led to the creation of the *Web Server Gateway Interface (WSGI)* in 2003. WSGI is not a server or framework but a set of rules that standardizes how web servers should connect to *any* Python web app. By abstracting away this headache, it opened the door to newer Python web frameworks—like Django, which was first released in 2005—that could work on any web server and did not have to worry about this step in the process. Common examples of production WSGI servers include Gunicorn, uWSGI, and Daphne.

Traditionally, Python was a *synchronous* programming language: code executed sequentially, meaning each piece of code had to be completed before another piece of code could begin. As a result, complex tasks might take a while. Starting in 2012 with Python 3.3, *asynchronous* programming was added to Python via the [asyncio](#) module. While synchronous processing is done sequentially in a specific order, asynchronous processing occurs in parallel. Tasks that are not dependent on others can be offloaded and executed at the same time as the main operation, and the result can then be reported back when complete.

Django has been gradually adding [asynchronous support](#) since version 3.0 in 2019. One layer is the introduction of the *Asynchronous Server Gateway Interface (ASGI)*, which, as the name suggests, standardizes how servers should connect to Python web apps that support both synchronous and asynchronous communication. ASGI is intended to be the eventual successor to WSGI.

ASGI and WSGI are both included in new Django projects now. When you run the `startproject` command, Django generates a `wsgi.py` file and an `asgi.py` file in the project-level directory, `django_project`.

Full async support for the entire Django stack is still in the works but comes ever closer with each new major release. Given that this book is for beginners, it is important to recognize asynchronous developments rather than dwell on them. While they are exciting from a technical perspective, they are also challenging to

그런 다음 HTTP 응답을 생성합니다. 다시 말해, runserver는 단순한 웹 서버가 아니라 애플리케이션 서버로도 작동합니다.

애플리케이션 서버는 Python 웹 앱을 서버에 연결하기 위해 WSGI를 사용하기 때문에 일반적으로 "WSGI 서버"라고 불립니다. 웹 개발 초기에는 웹 프레임워크가 많은 사용자 정의 없이 다양한 웹 서버와 잘 작동하지 않았습니다. Python 웹 프레임워크의 경우, 이는 2003년에 웹 서버 게이트웨이 인터페이스(WSGI)의 생성으로 이어졌습니다. WSGI는 서버나 프레임워크가 아니라 웹 서버가 모든 Python 웹 앱에 연결하는 방법을 표준화하는 규칙 집합입니다. 이 문제를 추상화함으로써, 2005년에 처음 출시된 Django와 같은 새로운 Python 웹 프레임워크가 모든 웹 서버에서 작동할 수 있도록 하여 이 과정의 이 단계를 걱정할 필요가 없었습니다. 생산 WSGI 서버의 일반적인 예로는 Gunicorn, uWSGI 및 Daphne가 있습니다.

전통적으로 Python은 동기 프로그래밍 언어였습니다: 코드는 순차적으로 실행되며, 이는 각 코드 조각이 다른 코드 조각이 시작되기 전에 완료되어야 함을 의미합니다. 결과적으로 복잡한 작업은 시간이 걸릴 수 있습니다. 2012년 Python 3.3부터 비동기 프로그래밍이 asyncio 모듈을 통해 Python에 추가되었습니다. 동기 처리는 특정 순서로 순차적으로 수행되는 반면, 비동기 처리는 병렬로 발생합니다. 다른 작업에 의존하지 않는 작업은 오프로드되어 주요 작업과 동시에 실행될 수 있으며, 완료되면 결과를 보고할 수 있습니다.

Django는 2019년 버전 3.0부터 비동기 지원을 점진적으로 추가하고 있습니다. 한 가지 계층은 비동기 서버 게이트웨이 인터페이스(ASGI)의 도입으로, 이름에서 알 수 있듯이 서버가 동기 및 비동기 통신을 지원하는 Python 웹 앱에 연결하는 방법을 표준화합니다. ASGI는 궁극적으로 WSGI의 후계자가 될 예정입니다.

ASGI와 WSGI는 이제 새로운 Django 프로젝트에 모두 포함되어 있습니다. startproject 명령을 실행하면 Django는 프로젝트 수준 디렉토리인 django_project에 wsgi.py 파일과 asgi.py 파일을 생성합니다.

Django 스택 전체에 대한 완전한 비동기 지원은 아직 진행 중이지만, 각 새로운 주요 릴리스와 함께 점점 더 가까워지고 있습니다. 이 책이 초보자를 위한 것인 만큼, 비동기 개발을 인식하는 것이 중요하며, 이에 대해 깊이 고민하지 않는 것이 중요합니다. 기술적인 관점에서 흥미롭지만, 또한 도전적입니다.

reason about and are most relevant for websites that need “real-time” functionality, such as live notifications, chat applications, real-time data updates, and interactive dashboards.

Deployment Checklist

It can be overwhelming to see a complete deployment checklist right at the beginning, but it is a helpful guide for this chapter. Here, then, is the deployment checklist we will cover:

- configure static files and install `WhiteNoise`
- add environment variables with `envvars`
- create a `.env` file and update the `.gitignore` file
- update `DEBUG`, `ALLOWED_HOSTS`, `SECRET_KEY`, and `CSRF_TRUSTED_ORIGINS`
- update `DATABASES` to run PostgreSQL in production and install `psycopg2`
- install Gunicorn as a production WSGI server
- create a `Procfile`
- update the `requirements.txt` file
- create a new Heroku project, push the code to it, and start a dyno web process

We could toggle many more production settings, but this list covers the most critical security and performance concerns.

Static Files

Static files are the images, JavaScript, and CSS used by a website. We worked with them in Chapter 6 on the *Blog* project, where we added custom CSS. For local usage, as long as `DEBUG` is set to `True` in `settings.py`, the files are served automatically by the `runserver` command.

Django automatically looks for static files within each app in a folder called “static,” but a common technique is to place all static files in a project-level folder called “static” instead. We’ll do that here. Quit the local server with `Control+c` and create a new static directory in the same folder as the `manage.py` file. Add new folders within it for `css`, `js`, and `img` on the command line.

"실시간" 기능이 필요한 웹사이트, 예를 들어 실시간 알림, 채팅 애플리케이션, 실시간 데이터 업데이트 및 대화형 대시보드와 가장 관련이 있습니다.

배포 체크리스트

처음부터 완전한 배포 체크리스트를 보는 것은 압도적일 수 있지만, 이 장에 유용한 가이드입니다. 여기서 다룰 배포 체크리스트는 다음과 같습니다:

- 정적 파일을 구성하고 WhiteNoise를 설치하고 환경 변수를 추가합니다.
- .env 파일을 만들고 .gitignore 파일을 업데이트합니다.
- DEBUG, ALLOWED_HOSTS, SECRET_KEY 및 CSRF_TRUSTED_ORIGINS를 업데이트하고, 프로덕션에서 PostgreSQL을 실행하기 위해 DATABASES를 업데이트하고 psycopg2를 설치하고, 프로덕션 WSGI 서버로 Gunicorn을 설치합니다.
- Procfile을 만듭니다.
- requirements.txt 파일을 업데이트합니다.
- 새로운 Heroku 프로젝트를 만들고, 코드를 푸시하고, dyno 웹 프로세스를 시작합니다.

더 많은 프로덕션 설정을 전환할 수 있지만, 이 목록은 가장 중요한 보안 및 성능 문제를 다룹니다.

정적 파일

정적 파일은 웹사이트에서 사용하는 이미지, JavaScript 및 CSS입니다. 우리는 블로그 프로젝트의 6장에서 사용자 정의 CSS를 추가하면서 이들과 작업했습니다. 로컬 사용의 경우, settings.py에서 DEBUG가 True로 설정되어 있는 한, 파일은 runserver 명령에 의해 자동으로 제공됩니다.

장고는 각 앱 내의 "static"이라는 폴더에서 정적 파일을 자동으로 찾지만, 일반적인 기법은 모든 정적 파일을 "static"이라는 프로젝트 수준 폴더에 배치하는 것입니다. 여기서 그렇게 하겠습니다. Control+c로 로컬 서버를 종료하고 manage.py 파일과 동일한 폴더에 새 정적 디렉토리를 만듭니다. 명령줄에서 css, js 및 img를 위한 새 폴더를 추가합니다.

```
(.venv) $ mkdir static
(.venv) $ mkdir static/css
(.venv) $ mkdir static/js
(.venv) $ mkdir static/img
```

A quirk of Git is that it will not track empty folders. If no files are within a folder, Git ignores it by default. One solution—which we will adopt here—is to add a `.keep` file to the three subfolders with your text editor:

- `static/css/.keep`
- `static/js/.keep`
- `static/img/.keep`

For local usage, only two settings are required for static files: `STATIC_URL`, which is the base URL for serving static files, and `STATICFILES_DIRS`, which defines the additional locations the built-in `staticfiles` app will traverse looking for static files beyond an `app/static` folder.

Code

```
# django_project/settings.py
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"] # new
```

Our local Django server is not designed to host static files in production. A best practice is to bundle all the static files into a single directory and then have the production web server, not the Django server, serve them. Django has a management command, `collectstatic`, for just this purpose: it copies all static files into a single location for deployment. The one configuration required of us is setting `STATIC_ROOT` to define the location of compiled static files. By convention, we will create a new project-level directory called `staticfiles`.

Code

```
# django_project/settings.py
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
STATIC_ROOT = BASE_DIR / "staticfiles" # new
```

Now run the command `python manage.py collectstatic` to compile all static files into the `staticfiles` folder.

Shell

```
(.venv) $ python manage.py collectstatic
```

```
(.venv) $ mkdir static(.venv) $  
mkdir static/css(.venv) $ mkdir  
static/js(.venv) $ mkdir  
static/img
```

Git의 특이점은 빈 폴더를 추적하지 않는다는 것입니다. 폴더 안에 파일이 없으면 Git은 기본적으로 이를 무시합니다. 여기서 채택할 해결책은 텍스트 편집기를 사용하여 세 개의 하위 폴더에 .keep 파일을 추가하는 것입니다:

- static/css/ .keepstat
- static/js/ .keepstatic/im
- g/ .keep

로컬 사용을 위해 정적 파일에 필요한 설정은 두 가지뿐입니다: STATIC_URL, 이는 정적 파일을 제공하기 위한 기본 URL이며, STATICFILES_DIRS는 내장된 staticfiles 앱이 탐색할 추가 위치를 정의합니다.

정적 파일은 앱/static 폴더를 넘어선다.코드

```
# django_project/settings.py  
STATIC_URL = "static/"  
STATICFILES_DIRS = [BASE_DIR / "static"] # new
```

우리의 로컬 Django 서버는 프로덕션에서 정적 파일을 호스팅하도록 설계되지 않았습니다. 모범 사례는 모든 정적 파일을 단일 디렉토리에 묶은 다음 프로덕션 웹 서버가 Django 서버가 아닌 이를 제공하도록 하는 것입니다. Django에는 collectstatic이라는 관리 명령이 있습니다. 이는 배포를 위해 모든 정적 파일을 단일 위치로 복사합니다. 우리가 설정해야 하는 유일한 구성은 컴파일된 정적 파일의 위치를 정의하기 위해 STATIC_ROOT를 설정하는 것입니다. 관례상, 우리는 staticfiles라는 새로운 프로젝트 수준 디렉토리를 생성할 것입니다.

코드

```
# django_project/settings.py  
STATIC_URL = "static/"  
STATICFILES_DIRS = [BASE_DIR / "static"]  
STATIC_ROOT = BASE_DIR / "staticfiles" # 새로 만들기
```

이제 python manage.py collectstatic 명령어를 실행하여 모든 정적 파일을 staticfiles 폴더에 컴파일합니다.

셸

```
(.venv) $ python manage.py collectstatic
```

The only static files right now are contained within the built-in admin app, so a new `staticfiles` directory should appear with sections for the admin. When additional static files are added in the future, they will also be compiled in this directory.

Code

```
# staticfiles/
└─ admin
    ├── css
    ├── img
    └── js
```

We need to use the `{% load static %}` template tag to display static files in the templates. Add it now to the top of the `base.html` file.

Code

```
<!-- templates/base.html -->
{% load static %}
<!DOCTYPE html>
...
```

If we were deploying with a dedicated server or VPS, it would be up to us to write code for the web server, likely Nginx or Apache, to serve static files. But since we are using the PaaS Heroku, we can leverage the popular [WhiteNoise](#) third-party package optimized to serve static files from Django. It allows additional compression and immutable file storage and sets appropriate HTTP caching headers. In short, it makes our deployment process much more straightforward.

Install the latest version of `WhiteNoise` using `pip`.

Shell

```
(.venv) $ python -m pip install whitenoise==6.7.0
```

Then, in the `django_project/settings.py` file, make three changes:

- add `whitenoise` to the `INSTALLED_APPS` **above** the built-in `staticfiles` app
- under `MIDDLEWARE`, add a new `WhiteNoiseMiddleware` on the third line
- change `STORAGES` to use `WhiteNoise`.

Code

```
# django_project/settings.py
INSTALLED_APPS = [
```

현재 정적 파일은 내장된 관리자 앱에만 포함되어 있으므로, 관리자 섹션이 있는 새로운 staticfiles 디렉토리가 나타나야 합니다. 앞으로 추가되는 정적 파일도 이 디렉토리에 컴파일될 것입니다.

코드

```
# staticfiles/
└─ admin
    └─
        ├── css
        └── img
            └── js
```

템플릿에서 정적 파일을 표시하려면 {% load static %} 템플릿 태그를 사용해야 합니다. 지금 base.html 파일의 맨 위에 추가하세요.

코드

```
<!-- templates/base.html -->
{% load static %}
<!DOCTYPE html>
...
```

전용 서버나 VPS로 배포하고 있었다면, 정적 파일을 제공하기 위해 웹 서버, 아마도 Nginx나 Apache를 위한 코드를 작성하는 것이 우리의 몫이었을 것입니다. 하지만 PaaS Heroku를 사용하고 있기 때문에, Django에서 정적 파일을 제공하기 위해 최적화된 인기 있는 WhiteNoise 서드파티 패키지를 활용할 수 있습니다. 이는 추가 압축과 불변 파일 저장을 허용하고 적절한 HTTP 캐싱 헤더를 설정합니다. 요약하자면, 우리의 배포 프로세스를 훨씬 더 간단하게 만들어 줍니다.

pip를 사용하여 WhiteNoise의 최신 버전을 설치합니다. 셸

```
(.venv) $ python -m pip install whitenoise==6.7.0
```

그런 다음, django_project/settings.py 파일에서 세 가지 변경을 합니다:

- 내장된 staticfiles 앱 위에 INSTALLED_APPS에 whitenoise를 추가합니다.
- MIDDLEWARE 아래에 세 번째 줄에 새로운 WhiteNoiseMiddleware를 추가합니다.
- STORAGES를 WhiteNoise를

```
# django_project/settings.py
INSTALLED_APPS = [
```

사용
하도
록
변경
합니
다.
코드

```

    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "whitenoise.runserver_nostatic", # new
    "django.contrib.staticfiles",
    # 3rd Party
    "crispy_forms",
    "crispy_bootstrap5",
    # Local
    "accounts",
    "pages",
    "articles",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware", # new
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]

...
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
STATIC_ROOT = BASE_DIR / "staticfiles"
STORAGES = {
    "default": {
        "BACKEND": "django.core.files.storage.FileSystemStorage",
    },
    "staticfiles": {
        "BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage", # new
    },
}

```

[STORAGES](#) is a new setting in Django 4.2+ that defines how files are stored. It is implicitly set in `settings.py` but we are changing the `staticfiles` section to use `WhiteNoise` compression.

Run the `collectstatic` command again. The prompt will warn about overwriting existing files, but that is intentional: we want to compile them using `WhiteNoise` now. Type `yes` and press `Return` to continue:

```

Shell
(.venv) $ python manage.py collectstatic

```

That's it! We have configured our static files to be compiled in one place for production, added the `static` template tag to our `base.html` template, and

```

    "django.contrib.admin", "django.contrib.auth", "
    django.contrib.contenttypes", "
    django.contrib.sessions", "
    django.contrib.messages", "
    whitenoise.runserver_nostatic", # new

    "django.contrib.staticfiles",
    # 3rd Party
    "crispy_forms", "
    crispy_bootstrap5",
    # Local
    "accounts", "
    pages", "
    articles",
]

MIDDLEWARE = ["django.middleware.security.SecurityMiddleware", "
    django.contrib.sessions.middleware.SessionMiddleware", "
    whitenoise.middleware.WhiteNoiseMiddleware", # new"
    django.middleware.common.CommonMiddleware", "
    django.middleware.csrf.CsrfViewMiddleware", "
    django.contrib.auth.middleware.AuthenticationMiddleware", "
    django.contrib.messages.middleware.MessageMiddleware", "
    django.middleware.clickjacking.XFrameOptionsMiddleware",

]

...
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
STATIC_ROOT = BASE_DIR / "staticfiles"
STORAGES =
{
    "default": {
        "BACKEND": "django.core.files.storage.FileSystemStorage", },
    {
        "staticfiles": {
            "BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage", # new},
}

```

STORAGES는 Django 4.2+에서 파일이 저장되는 방식을 정의하는 새로운 설정입니다. settings.py 에서 암묵적으로 설정되지만, 우리는 staticfiles 섹션을 WhiteNoise 압축을 사용하도록 변경하고 있습니다.

collectstatic 명령을 다시 실행하십시오. 프롬프트는 기존 파일을 덮어쓰는 것에 대해 경고하지만, 이는 의도된 것입니다: 우리는 그것들을 컴파일하고 싶습니다. 이제 WhiteNoise를 사용합니다. yes를 입력하고 Return을 눌러 계속

하십시오:Shell

```
(.venv) $ python manage.py collectstatic
```

그게 전부입니다! 우리는 정적 파일을 프로덕션을 위해 한 곳에 컴파일하도록 구성하고, base.html 템플릿에 정적 템플릿 태그를 추가했습니다.

installed `WhiteNoise` to efficiently serve them.

Middleware

Adding `WhiteNoise` is the first time we've updated the Django [middleware](#), a framework of hooks into Django's request/response processing. It is a way to add functionality such as authentication, security, sessions, and more. During the HTTP request phase, middleware is applied in the order it is defined in `MIDDLEWARE` **top-down**. That means `SecurityMiddleware` comes first, then `SessionMiddleware`, and so on.

Code

```
# django_project/settings.py
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware", # new
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

During the HTTP response phase, after the view is called, middleware are applied in reverse order, from the bottom up, starting with `XFrameOptionsMiddleware`, then `MessageMiddleware`, and so on. The traditional way of describing middleware is like an onion, where each middleware class is a “layer” that wraps the view.

Truly diving into middleware is an advanced topic beyond the scope of this book. It is important, however, to be conceptually aware of how all the pieces in the Django architecture fit together.

Environment Variables

Real-world Django projects require at least two environments (local and production) but typically have more if multiple testing servers are involved. There are two ways to toggle between different environments in the same project: environment variables and multiple settings files. These days, the most popular approach is to use environment variables, which we will do here. An environment variable is a variable whose value is set outside the current program

WhiteNoise를 설치하여 효율적으로 제공합니다.

미들웨어

WhiteNoise를 추가하는 것은 Django 미들웨어를 업데이트한 첫 번째 사례입니다. 미들웨어는 Django의 요청/응답 처리에 대한 후크의 프레임워크입니다. 인증, 보안, 세션 등과 같은 기능을 추가하는 방법입니다. HTTP 요청 단계에서 미들웨어는 MIDDLEWARE에 정의된 순서대로 위에서 아래로 적용됩니다. 즉, SecurityMiddleware가 먼저 오고, 그 다음에 SessionMiddleware가 오며, 계속해서 진행됩니다.

코드

```
# django_project/settings.py
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    django.contrib.sessions.middleware.SessionMiddleware",
    whitenoise.middleware.WhiteNoiseMiddleware", # 새
    django.middleware.common.CommonMiddleware",
    django.middleware.csrf.CsrfViewMiddleware",
    django.contrib.auth.middleware.AuthenticationMiddleware",
    django.contrib.messages.middleware.MessageMiddleware",
    django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

HTTP 응답 단계에서, 뷰가 호출된 후, 미들웨어는 아래에서 위로 역순으로 적용되며, XFrameOptionsMiddleware로 시작하여 MessageMiddleware 등으로 이어집니다. 미들웨어를 설명하는 전통적인 방법은 각 미들웨어 클래스가 뷰를 감싸는 "층"인 양파와 같습니다.

미들웨어에 진정으로 다가가는 것은 이 책의 범위를 넘어선 고급 주제입니다. 그러나 Django 아키텍처의 모든 요소가 어떻게 맞물리는지 개념적으로 인식하는 것이 중요합니다.

환경 변수

실제 Django 프로젝트는 최소한 두 개의 환경(로컬 및 프로덕션)이 필요하지만, 여러 테스트 서버가 포함된 경우 일반적으로 더 많은 환경을 가집니다. 동일한 프로젝트에서 서로 다른 환경 간에 전환하는 두 가지 방법이 있습니다: 환경 변수와 여러 설정 파일. 요즘 가장 인기 있는 접근 방식은 환경 변수를 사용하는 것이며, 여기서도 그렇게 할 것입니다. 환경 변수는 현재 프로그램 외부에서 값이 설정된 변수입니다.

and can be loaded in at runtime. We can store these variables securely and load them into our Django project as needed.

There are multiple ways to work with environment variables, but for this project, we will use [environs](#), a popular third-party package that comes with additional Django-specific features. Use pip to add environs and include the double quotes, "", to install the helpful Django extension.

Shell

```
(.venv) $ python -m pip install "environs[django]"==11.0.0
```

Then, add three new lines to the top of the `django_project/settings.py` file.

Code

```
# django_project/settings.py
from pathlib import Path
from environs import Env # new

env = Env() # new
env.read_env() # new
```

Next, create a new file, `.env`, in the root project directory, containing our environment variables. We already know that any file or directory starting with a period will be treated as a hidden file and not displayed by default during a directory listing. The file still exists, though, and needs to be added to the `.gitignore` file to avoid being added to our Git source control.

`.gitignore`

```
.venv/
__pycache__/
db.sqlite3
.env # new!
```

DEBUG and ALLOWED_HOSTS

The first setting we will configure with environment variables is `DEBUG`. By default, `DEBUG` is set to `True`, which is helpful for local development but a major security issue if deployed into production. For example, if you start up the local server with `python manage.py runserver` and navigate to a page that does not exist, like `http://127.0.0.1:8000/debug`, you will see the following:

런타임에 로드할 수 있습니다. 이러한 변수를 안전하게 저장하고 필요에 따라 Django 프로젝트에 로드할 수 있습니다.

환경 변수를 다루는 방법은 여러 가지가 있지만, 이 프로젝트에서는 추가적인 Django 전용 기능이 포함된 인기 있는 서드파티 패키지인 environs를 사용할 것입니다. pip를 사용하여 environs를 추가하고 유용한 Django 확장을 설치하기 위해 쌍따옴표, "",를 포함하세요.

셸

```
(.venv) $ python -m pip install "environs[django]"==11.0.0
```

그런 다음 django_project/settings.py 파일의 맨 위에 세 줄을 추가하세요. 코드

```
# django_project/settings.py
from pathlib import Path
from environs import Env # new

env = Env() # new
env.read_env() # new
```

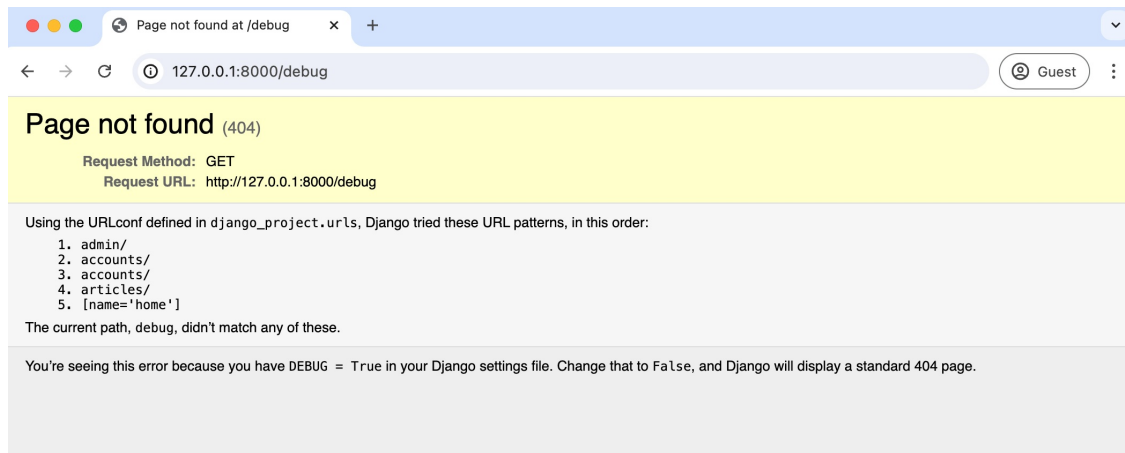
다음으로, 루트 프로젝트 디렉토리에 .env라는 새 파일을 생성하여 환경 변수를 포함합니다. 점으로 시작하는 파일이나 디렉토리는 기본적으로 숨김 파일로 처리되어 디렉토리 목록에 표시되지 않는다는 것을 이미 알고 있습니다. 그러나 파일은 여전히 존재하며, Git 소스 제어에 추가되지 않도록 .gitignore 파일에 추가해야 합니다.

```
.gitignore
```

```
.venv/__pycache__
__db.sqlite3
3.env # 새!
```

DEBUG 및 ALLOWED_HOSTS

환경 변수로 구성할 첫 번째 설정은 DEBUG입니다. 기본적으로 DEBUG는 True로 설정되어 있으며, 이는 로컬 개발에 유용하지만 프로덕션에 배포할 경우 주요 보안 문제입니다. 예를 들어, python manage.py runserver로 로컬 서버를 시작하고 http://127.0.0.1:8000/debug와 같이 존재하지 않는 페이지로 이동하면 다음과 같은 내용을 볼 수 있습니다:



Debug Page

This page lists all the URLs tried and apps loaded, a treasure map for any hacker attempting to break into your site. You'll even see that at the bottom of the error page, it says that Django will display a standard 404 page if `DEBUG=False`. That's what we want! The first step is to change `DEBUG` to `False` in the `settings.py` file.

Code

```
# django_project/settings.py
DEBUG = False
```

Refresh the web page `http://127.0.0.1:8000/debug`, and you'll see an error: the site doesn't load at all. On the command line, Django has provided us with the explanation via [CommandError](#), which is raised for serious problems.

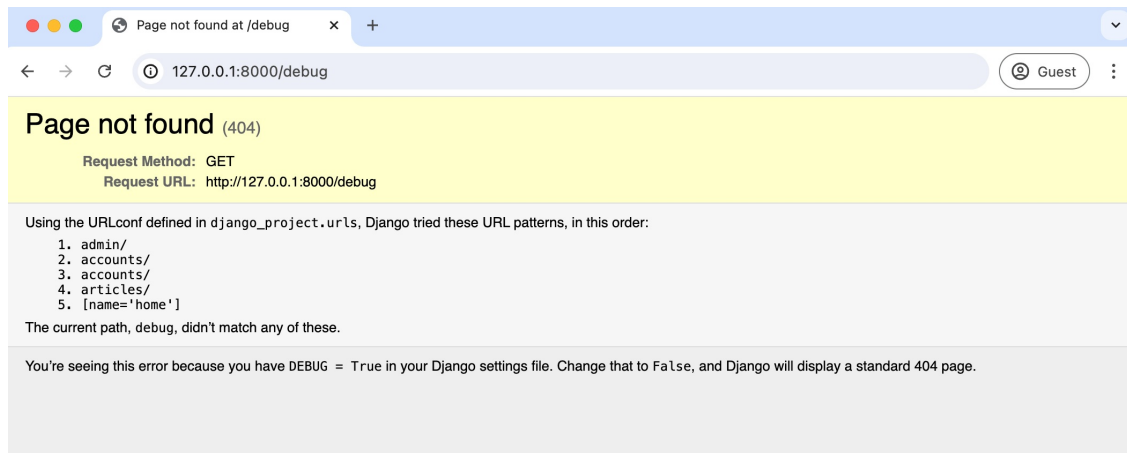
Shell

```
(.venv) $ python manage.py runserver
...
CommandError: You must set settings.ALLOWED_HOSTS if DEBUG is False.
```

In this case, Django is telling us that we can't set `DEBUG` to `False` if we have not set `ALLOWED_HOSTS`. So what is [ALLOWED_HOSTS](#)? It is a list of strings representing host/domain names that our Django site can serve. By default, `ALLOWED_HOSTS` is set to accept all hosts, which is not secure! We must update it to accept local ports (`localhost` and `127.0.0.1`) and `.herokuapp.com` for our Heroku deployment. We can add all three routes to our config.

Code

```
# django_project/settings.py
ALLOWED_HOSTS = [".herokuapp.com", "localhost", "127.0.0.1"] # new
```



디버그 페이지

이 페이지는 시도된 모든 URL과 로드된 앱을 나열하며, 해커가 귀하의 사이트를 해킹하려고 시도할 때의 보물 지도입니다. 오류 페이지 하단에 `DEBUG=False`일 경우 Django가 표준 404 페이지를 표시한다고 적혀 있는 것을 볼 수 있습니다. 우리가 원하는 것입니다! 첫 번째 단계는 `settings.py` 파일에서 `DEBUG`를 `False`로 변경하는 것입니다.

코드

```
# django_project/settings.py
DEBUG = False
```

웹 페이지 `http://127.0.0.1:8000/debug`를 새로 고치면 오류가 발생합니다: 사이트가 전혀 로드되지 않습니다. 명령줄에서 Django는 심각한 문제에 대해 발생하는 `CommandError`를 통해 설명을 제공합니다.

셸

```
(.venv) $ python manage.py runserver
...
CommandError: DEBUG가 False인 경우 settings.ALLOWED_HOSTS를 설정해야 합니다.
```

이 경우, Django는 `ALLOWED_HOSTS`를 설정하지 않았다면 `DEBUG`를 `False`로 설정할 수 없다고 알려줍니다. 그렇다면 `ALLOWED_HOSTS`란 무엇일까요? 그것은 우리의 Django 사이트가 제공할 수 있는 호스트/도메인 이름을 나타내는 문자열 목록입니다. 기본적으로 `ALLOWED_HOSTS`는 모든 호스트를 수용하도록 설정되어 있는데, 이는 안전하지 않습니다! 우리는 이를 업데이트하여 로컬 포트(로컬 호스트 및 127.0.0.1)와 `.herokuapp.com`을 우리의 Heroku 배포를 위해 수용해야 합니다. 우리는 이 세 가지 경로를 우리의 설정에 추가할 수 있습니다.

코드

```
# django_project/settings.py
ALLOWED_HOSTS = [".herokuapp.com", "localhost", "127.0.0.1"]
```

새로 추가

Now that we've set `ALLOWED_HOSTS` try the `runserver` command again.



Not Found Page

This is the generic Django 404 page that we want displayed in production. It does not give away any information to a potential hacker.

Manually setting configurations for development and production environments is not ideal. For one thing, it is a major pain and easy to make a mistake. For another, it is insecure if we put production information that should be secret into `settings.py` and perform a Git commit by mistake.

This is where environment variables come to the rescue. To add any environment variable to our project, we first add it to the `.env` file and then update `django_project/settings.py`.

Within the `.env` file, create a new environment variable called `DEBUG` and set its value to `True`.

Code

```
# .env  
DEBUG=True
```

Then in `django_project/settings.py`, change the `DEBUG` setting to read the variable "DEBUG" from the `.env` file.

Code

```
# django_project/settings.py  
DEBUG = env.bool("DEBUG", default=False)
```

The syntax of `env.bool` says to load an environment variable from the `.env` file that is a Boolean, meaning true or false, with the name "DEBUG." If an environment variable can't be found, then the `default` value, set here to `False`, will be used. It is a best practice to default to production settings since they are

이제 ALLOWED_HOSTS를 설정했으니 runserver 명령어를 다시 시도해 보세요.



찾을 수 없는 페이지

이것은 우리가 프로덕션에서 표시하고자 하는 일반적인 Django 404 페이지입니다. 이는 잠재적인 해커에게 어떤 정보도 제공하지 않습니다.

개발 및 프로덕션 환경을 위해 수동으로 설정하는 것은 이상적이지 않습니다. 한 가지는, 이는 큰 고통이며 실수를 쉽게 할 수 있습니다. 또 다른 이유는, settings.py에 비밀로 해야 할 프로덕션 정보를 넣고 실수로 Git 커밋을 하는 경우 불안전합니다.

여기서 환경 변수가 구세주 역할을 합니다. 프로젝트에 환경 변수를 추가하려면 먼저 .env 파일에 추가한 다음 django_project/settings.py를 업데이트합니다.

.env 파일 내에서 DEBUG라는 새로운 환경 변수를 만들고 그 값을 True로 설정합니다.

코드

```
# .env  
DEBUG=True
```

그런 다음 django_project/settings.py에서 DEBUG 설정을 .env 파일의 "DEBUG" 변수를 읽도록 변경합니다.

코드

```
# django_project/settings.py  
DEBUG = env.bool("DEBUG", default=False)
```

env.bool의 구문은 .env 파일에서 불리언인 환경 변수를 로드하라고 말합니다. 즉, 이름이 "DEBUG"인 true 또는 false입니다. 환경 변수를 찾을 수 없는 경우 여기서 기본값으로 설정된 False가 사용됩니다. 프로덕션 설정을 기본값으로 설정하는 것이 모범 사례입니다.

more secure, and if something goes wrong in our code, we won't default to exposing all our secrets.

SECRET_KEY, and CSRF_TRUSTED_ORIGINS

SECRET_KEY, a random 50-character string generated each time startproject is run. This string provides cryptographic protection throughout our Django project. In the settings file, you'll see the current value that begins with django-insecure. Here is the django_project/settings.py value of the SECRET_KEY in my project. Yours will be different.

Code

```
# django_project/settings.py
SECRET_KEY = "django-insecure-3$k(g9eheqqbZR@#&tt)r6%ab-g1=!j@2c^y7*s16+ltzys05!"
```

And here it is, without the double quotes, in the .env file

.env

```
DEBUG=True
SECRET_KEY=django-insecure-3$k(g9eheqqbZR@#&tt)r6%ab-g1=!j@2c^y7*s16+ltzys05! # new
```

Update django_project/settings.py so that SECRET_KEY points to this new environment variable. It is a string so the syntax is env.str.

Code

```
# django_project/settings.py
SECRET_KEY = env.str("SECRET_KEY")
```

The SECRET_KEY is out of the settings file and safe now, right? Actually no! Because we made previous Git commits that included the value, it is stored in our Git history no matter what we do. The solution is to create a new SECRET_KEY and add it to the .env file. One way to generate a new one is by invoking Python's built-in [secrets](#) module by running `python -c "import secrets; print(secrets.token_urlsafe())"` on the command line.

Shell

```
(.venv) $ python -c "import secrets; print(secrets.token_urlsafe())"
imDnFLXy-8Y-YozfJmP2Rw_81YA_qx1XK15FeY0mXyY
```

Copy and paste this new value into the .env file.

.env

더 안전하며, 코드에서 문제가 발생하더라도 모든 비밀을 노출하는 기본값으로 설정되지 않을 것입니다.

SECRET_KEY, 그리고 CSRF_TRUSTED_ORIGINS

SECRET_KEY는 startproject가 실행될 때마다 생성되는 무작위 50자 문자열입니다. 이 문자열은 우리의 Django 프로젝트 전반에 걸쳐 암호화 보호를 제공합니다. 설정 파일에서 django_insecure로 시작하는 현재 값을 볼 수 있습니다. 여기 내 프로젝트의 SECRET_KEY에 대한 django_project/settings.py 값이 있습니다. 당신의 것은 다를 것입니다.

코드

```
# django_project/settings.py
SECRET_KEY = "django-insecure-3$k(g9eheqqbzr@#&tt)r6%ab-g1=!j@2c^y7*sl6+ltzys05!"
```

그리고 여기 .env 파일에서 큰따옴표 없이 있습니다.

```
DEBUG=True
SECRET_KEY=django-insecure-3$k(g9eheqqbzr@#&tt)r6%ab-g1=!j@2c^y7*sl6+ltzys05! # 새
```

django_project/settings.py를 업데이트하여 SECRET_KEY가 이 새로운 환경 변수를 가리키도록 하세요. 문자열이므로 구문은 env.str입니다.

코드

```
# django_project/settings.py
SECRET_KEY = env.str("SECRET_KEY")
```

SECRET_KEY는 설정 파일에서 벗어나 이제 안전한가요? 사실 아닙니다! 이전에 Git 커밋을 만들었기 때문에 값이 포함되어 있으며, 우리가 무엇을 하든 Git 기록에 저장되어 있습니다. 해결책은 새로운 SECRET_KEY를 생성하고 이를 .env 파일에 추가하는 것입니다. 새로운 SECRET_KEY를 생성하는 한 가지 방법은 Python의 내장 secrets 모듈을 호출하는 것입니다. 명령줄에서 `python -c "import secrets; print(secrets.token_urlsafe())"`를 실행하세요.

셸

```
(.venv) $ python -c "import secrets; print(secrets.token_urlsafe())"imDnFLXy-8Y-
YozfJmP2Rw_81YA_qx1XKI5FeY0mXyY
```

이 새로운 값을 .env 파일에 복사하여 붙여넣으세요..env

```
DEBUG=True
SECRET_KEY=imDnFLXy-8Y-YozfJmP2Rw_81YA_qx1XKl5FeY0mXyY
```

Now restart the local server with `python manage.py runserver` and refresh your website. It will work with the new `SECRET_KEY` loaded from the `.env` file but not tracked by Git since `.env` is in the `.gitignore` file.

Our Newspaper project requires that we log into the admin on the production website to create, read, update, or delete posts. That means [CSRF TRUSTED ORIGINS](#) must be correctly configured since it is a list of trusted origins for unsafe HTTP requests like POST. Add it to the bottom of the `settings.py` file and set it to match a production URL on Heroku, `https://*.herokuapp.com`. We will update both `ALLOWED_HOSTS` and `CSRF_TRUSTED_ORIGINS` to match our production URL at the end of the chapter.

Code

```
# django_project/settings.py
CSRF_TRUSTED_ORIGINS = ["https://*.herokuapp.com"] # new
```

DATABASES

We want to use SQLite locally but PostgreSQL in production. Currently, our settings file for DATABASES lists only SQLite. The `ENGINE` specifies what type of database to use, and the `NAME` points to its location.

Code

```
# django_project/settings.py
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

Most PaaS will automatically set a `DATABASE_URL` environment variable inspired by the [Twelve-Factor App](#) approach that contains all the parameters needed to connect to a database in the format. In raw form, for PostgreSQL, it looks something like this:

Code

```
postgres://USER:PASSWORD@HOST:PORT/NAME
```

```
DEBUG=True
SECRET_KEY=imDnfLXy-8Y-YozfJmP2Rw_81YA_qx1XKI5FeY0mXyY
```

이제 `python manage.py runserver`로 로컬 서버를 재시작하고 웹사이트를 새로 고치세요. `.env` 파일에서 로드된 새로운 `SECRET_KEY`로 작동할 것입니다. 그러나 `.env`는 `.gitignore` 파일에 있기 때문에 Git에 의해 추적되지 않습니다.

우리 신문 프로젝트는 게시물을 생성, 읽기, 업데이트 또는 삭제하기 위해 프로덕션 웹사이트의 관리자에 로그인해야 합니다. 이는 `CSRF_TRUSTED_ORIGINS`가 POST와 같은 안전하지 않은 HTTP 요청을 위한 신뢰할 수 있는 출처 목록이므로 올바르게 구성되어야 함을 의미합니다. `settings.py` 파일의 맨 아래에 추가하고 Heroku의 프로덕션 URL인 `https://*.herokuapp.com`과 일치하도록 설정하세요. 우리는 `ALLOWED_HOSTS`와 `CSRF_TRUSTED_ORIGINS`를 장의 끝에서 우리의 프로덕션 URL과 일치하도록 업데이트할 것입니다. 코드

```
# django_project/settings.py
CSRF_TRUSTED_ORIGINS = ["https://*.herokuapp.com"] # new
```

DATABASES

우리는 로컬에서는 SQLite를 사용하고 프로덕션에서는 PostgreSQL을 사용하고 싶습니다. 현재 `DATABASES`에 대한 설정 파일에는 SQLite만 나열되어 있습니다. `ENGINE`은 어떤 유형의 사용할 데이터베이스와 `NAME`은 그 위치를 가리킵니다.

다. Code

```
# django_project/settings.py
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

대부분의 PaaS는 Twelve-Factor App 접근 방식에서 영감을 받아 데이터베이스에 연결하는 데 필요한 모든 매개변수를 포함하는 `DATABASE_URL` 환경 변수를 자동으로 설정합니다. 원시 형태로 PostgreSQL의 경우, 다음과 같은 형식입니다:

Code

```
postgres://USER:PASSWORD@HOST:PORT/NAME
```

In other words, use postgres and here are custom values for USER, PASSWORD, HOST, PORT/NAME. While we *could* manage this manually ourselves, this pattern is so well established in the Django community that a dedicated third-party package, [dj-database-url](#), exists to manage this for us. Conveniently, dj-database-url is already installed since it is one of the helper packages added by `enviro`[django].

This means we can solve all these problems with a single line of code. Here is the brief update to make to `django_project/settings.py` so that our project will try to access a `DATABASE_URL` environment variable.

Code

```
# django_project/settings.py
DATABASES = {"default": env.dj_db_url("DATABASE_URL")}
```

We will also set a `DATABASE_URL` environment variable in the `.env` file for local development.

.env

```
DEBUG=True
SECRET_KEY=imDnflXy-8Y-YozfJmP2Rw_81YA_qx1XKl5FeY0mXyY
DATABASE_URL=sqlite:///db.sqlite3
```

Let's review what's happening here because it is can be confusing initially. For local development, our project will try to find a `.env` file with environment variables. It will use them if it finds them, which is why they are the local defaults.

In production, we have included the `.env` file in our `.gitignore` file so Heroku won't know about them unless we set the environment variables manually. Heroku will *automatically* create a `DATABASE_URL` environment variable with the configuration for our production database, so that will be used.

We also need to install [Psycopg](#), a database adapter that lets Python apps like ours talk to PostgreSQL databases. You can install it with `pip` on Windows, but if you are on macOS, you must install PostgreSQL first via [Homebrew](#).

Shell

```
# Windows
(.venv) $ python -m pip install "psycopg[binary]"==3.2.1

# macOS
(.venv) $ brew install postgresql
(.venv) $ python3 -m pip install "psycopg[binary]"==3.2.1
```

다시 말해, postgres를 사용하고 USER, PASSWORD, HOST, PORT/NAME에 대한 사용자 정의 값을 여기에 입력하세요. 우리가 이것을 수동으로 관리할 수 있지만, 이 패턴은 Django 커뮤니티에서 잘 확립되어 있어, 이를 관리하기 위한 전용 서드파티 패키지인 dj-database-url이 존재합니다. 편리하게도, dj-database-url은 이미 설치되어 있으며, 이는 environs[django]에 추가된 도우미 패키지 중 하나입니다.

이는 우리가 단 한 줄의 코드로 모든 문제를 해결할 수 있음을 의미합니다. 여기 django_project/settings.py에 대한 간단한 업데이트가 있어 우리의 프로젝트가 DATABASE_URL 환경 변수를 접근하려고 시도할 것입니다. 코드

```
# django_project/settings.py
DATABASES = {"default": env.dj_db_url("DATABASE_URL") }
```

우리는 또한 로컬 개발을 위해 .env 파일에 DATABASE_URL 환경 변수를 설정할 것입니다.

```
.env

DEBUG=True
SECRET_KEY=imDnflXy-8Y-YozfJmP2Rw_81YA_qx1XKI5FeY0mXyY
DATABASE_URL=sqlite:///db.sqlite3
```

여기서 무슨 일이 일어나고 있는지 검토해 보겠습니다. 처음에는 혼란스러울 수 있습니다. 로컬 개발을 위해, 우리 프로젝트는 환경 변수가 포함된 .env 파일을 찾으려고 합니다. 찾으면 사용하게 되며, 그래서 그것들이 로컬 기본값이 됩니다.

프로덕션에서는 .env 파일을 .gitignore 파일에 포함시켜 Heroku가 환경 변수를 수동으로 설정하지 않는 한 알지 못하게 됩니다. Heroku는 프로덕션 데이터베이스에 대한 구성으로 DATABASE_URL 환경 변수를 자동으로 생성하므로, 그것이 사용됩니다.

우리는 또한 Python 앱이 PostgreSQL 데이터베이스와 통신할 수 있도록 해주는 데이터베이스 어댑터인 Psycopg를 설치해야 합니다. Windows에서는 pip로 설치할 수 있지만, macOS에서는 먼저 Homebrew를 통해 PostgreSQL을 설치해야 합니다.

```
Shell

# Windows
(.venv) $ python -m pip install "psycopg[binary]"==3.2.1# macOS

(.venv) $ brew install postgresql
(.venv) $ python3 -m pip install "psycopg[binary]"==3.2.1
```

We are using the [binary version](#) because it is the quickest way to start working with Psycopg.

Gunicorn and Procfile

Since Django's default development server, `runserver`, is explicitly not designed for production, we must select a production-ready WSGI server to use. [Gunicorn](#) is one of the most popular and easiest-to-configure options. It can handle multiple requests simultaneously while being scalable, stable, reliable, and compatible with production web servers.

Install Gunicorn with `pip`. Since we are using a PaaS, no additional configuration steps are required.

Shell

```
(.venv) $ python -m pip install gunicorn==22.0.0
```

Heroku relies on a proprietary `Procfile` file that provides instructions on running applications in their stack. In your text editor, create a new file called `Procfile` in the base directory. We only need a single line of configuration for our project, telling Heroku to use Gunicorn as the WSGI server, the location of the WSGI config file at `django_project.wsgi`, and finally, the flag `--log-file` - makes any logging messages visible to us.

Procfile

```
web: gunicorn django_project.wsgi --log-file -
```

requirements.txt

We're almost at the end of implementing the deployment checklist. The last step before deploying to Heroku is to update the `requirements.txt` file. After all, for deployment we have installed the following new packages: `whitenoise`, `environs`, `psycopg`, and `gunicorn`.

Use the command `pip freeze` and the `>` operator to output our virtual environment information into a `requirements.txt` file.

Shell

```
(.venv) $ pip freeze > requirements.txt
```

우리는 Psycopg로 작업을 시작하는 가장 빠른 방법이기 때문에 이전 버전을 사용하고 있습니다.

Gunicorn 및 Procfile

Django의 기본 개발 서버인 runserver는 명시적으로 프로덕션을 위해 설계되지 않았기 때문에, 우리는 사용할 프로덕션 준비가 된 WSGI 서버를 선택해야 합니다. Gunicorn은 가장 인기 있고 구성하기 쉬운 옵션 중 하나입니다. 여러 요청을 동시에 처리할 수 있으며 확장 가능하고 안정적이며 신뢰할 수 있고 프로덕션 웹 서버와 호환됩니다.

pip로 Gunicorn을 설치합니다. PaaS를 사용하고 있기 때문에 추가 구성 단계는 필요하지 않습니다.

셸

```
(.venv) $ python -m pip install gunicorn==22.0.0
```

Heroku는 그들의 스택에서 애플리케이션을 실행하는 방법에 대한 지침을 제공하는 독점 Procfile 파일에 의존합니다. 텍스트 편집기에서 기본 디렉토리에 Procfile이라는 새 파일을 만듭니다. 우리는 프로젝트에 대해 단일 구성 줄만 필요하며, Heroku에 Gunicorn을 WSGI 서버로 사용하고 WSGI 구성 파일의 위치를 `django_project.wsgi`로 설정하며, 마지막으로 `--log-file` 플래그를 사용하여 모든 로깅 메시지를 우리에게 표시하도록 합니다.

Procfile

```
web: gunicorn django_project.wsgi --log-file -
```

requirements.txt

배포 체크리스트 구현이 거의 끝났습니다. Heroku에 배포하기 전 마지막 단계는 requirements.txt 파일을 업데이트하는 것입니다. 결국, 배포를 위해 다음과 같은 새로운 패키지를 설치했습니다: whitenoise, environs, psycopg, gunicorn.

명령어 `pip freeze`와 `>` 연산자를 사용하여 우리의 가상 환경 정보를 requirements.txt 파일로 출력합니다. 셸

```
(.venv) $ pip freeze > requirements.txt
```

The `requirements.txt` file will appear in the root directory containing all our installed packages and their dependencies. My list as of the writing of this book looks as follows:

Code

```
# requirements.txt
asgiref==3.8.1
black==24.4.2
click==8.1.7
crispy-bootstrap5==2024.2
dj-database-url==2.2.0
dj-email-url==1.0.6
Django==5.0.6
django-cache-url==3.4.5
django-crispy-forms==2.2
environs==11.0.0
gunicorn==22.0.0
marshmallow==3.21.3
mypy-extensions==1.0.0
packaging==24.1
pathspec==0.12.1
platformdirs==4.2.2
psycpg==3.2.1
psycpg-binary==3.2.1
python-dotenv==1.0.1
sqlparse==0.5.0
typing_extensions==4.12.2
whitenoise==6.7.0
```

We can use `git status` to check our changes, add the new files, and commit them. We can also push to GitHub for an online backup of our code changes.

Shell

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "New updates for Heroku deployment"
(.venv) $ git push -u origin main
```

Heroku Setup

Before 2022, Heroku had a generous free tier, but unfortunately, that is not the case anymore. It costs a company real money to spin up virtual servers on your behalf, and as a result, few hosting companies offer a free tier anymore.

[Heroku pricing](#) involves multiple tiers of features and bills per hour with a maximum monthly limit. The deployment setup we will implement here costs \$12/month if left on all the time, but if you are cost-conscious and deploying for purely educational purposes, there is no reason to leave your website “live” all

requirements.txt 파일은 모든 설치된 패키지와 그 의존성을 포함하는 루트 디렉토리에 나타납니다. 이 책을 쓰는 시점에서 제 목록은 다음과 같습니다:

코드

```
#
requirements.txtasgiref
==3.8.1black==
24.4.2click==8.1.7
crispy-bootstrap5==2024.2dj-
database-url==2.2.0dj-email-
url==1.0.6Django==5.0.6

django-cache-url==
3.4.5django-crispy-forms==
2.2environs==
11.0.0unicorn==
22.0.0marshmallow==
3.21.3mypy-extensions==
1.0.0packaging==
24.1pathspec==
0.12.1platformdirs==
4.2.2psycpg==3.2.1
psycpg-binary==3.2.1python-
dotenv==1.0.1sqlparse==
0.5.0typing_extensions==
4.12.2whitenoise==6.7.0
```

git status를 사용하여 변경 사항을 확인하고, 새 파일을 추가하고, 커밋할 수 있습니다. 또한 GitHub에 푸시하여 코드 변경 사항의 온라인 백업을 만들 수 있습니다.

셸

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "Heroku 배포를 위한 새로운 업데이트"(.venv) $ git
push -u origin main
```

Heroku 설정

2022년 이전에 Heroku는 관대한 무료 요금제를 제공했지만, 불행히도 이제는 그렇지 않습니다. 귀하를 대신하여 가상 서버를 실행하는 데 회사에 실제 비용이 발생하며, 그 결과로 이제는 무료 요금제를 제공하는 호스팅 회사가 거의 없습니다.

Heroku 가격은 여러 기능 계층과 시간당 청구를 포함하며 최대 월 한도가 있습니다. 여기서 구현할 배포 설정은 항상 켜두면 월 \$12의 비용이 발생하지만, 비용을 신경 쓰고 순수하게 교육 목적으로 배포하는 경우 웹사이트를 '실시간'으로 유지할 이유가 없습니다.

the time. You can deploy the site, share it, and then take it down after a few days and the total cost should only be \$1-\$2.

Sign up for a Heroku account on their website. Fill in the registration form and await an email with a link to confirm your account. This will take you to the password setup page. Once configured, you will be directed to the dashboard section of the site. Heroku now requires enrolling in multi-factor authentication (MFA), which can be done with Salesforce or a tool like Google Authenticator. Heroku now requires adding a credit card for account verification and payment.

Once you are signed up, it is time to install Heroku's *Command Line Interface (CLI)* so we can deploy from the command line. Currently, we are operating within a local virtual environment for the *Newspaper* project. We want to install Heroku globally so it is available for all projects. An easy way to do this is by opening a new command line tab—`Control+t` on Windows or `Command+t` on a Mac—that is not currently operating in a virtual environment. Anything installed here will be global.

On Windows, see the [Heroku CLI page](#) to install the 32-bit or 64-bit version correctly. On a Mac, the package manager [Homebrew](#) is used for installation. If not already on your machine, install Homebrew by copying and pasting the long command on the Homebrew website into your command line and hitting Return. It will look something like this:

Shell

```
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/\ninstall/HEAD/install.sh)"
```

Next, install the Heroku CLI by copying and pasting the following into your command line and hitting Return.

Shell

```
$ brew tap heroku/brew && brew install heroku
```

Once installation is complete, you can close the new command line tab and return to the initial tab with the newspaper virtual environment active.

Type the command `heroku login` and press enter. Then press any key to open up a browser window, where you can log in with your email, password, and two-factor authentication you just set.

시간입니다. 사이트를 배포하고, 공유한 다음 며칠 후에 내릴 수 있으며 총 비용은 \$1-\$2 정도여야 합니다.

그들의 웹사이트에서 Heroku 계정에 가입하세요. 등록 양식을 작성하고 계정을 확인하는 링크가 포함된 이메일을 기다리세요. 이 이메일은 비밀번호 설정 페이지로 안내할 것입니다. 설정이 완료되면 사이트의 대시보드 섹션으로 이동하게 됩니다. Heroku는 이제 다단계 인증(MFA)에 등록할 것을 요구하며, 이는 Salesforce 또는 Google Authenticator와 같은 도구로 수행할 수 있습니다. Heroku는 이제 계정 확인 및 결제를 위해 신용카드를 추가할 것을 요구합니다.

가입이 완료되면 Heroku의 명령줄 인터페이스(CLI)를 설치할 시간입니다. 현재 우리는 신문 프로젝트를 위한 로컬 가상 환경 내에서 작업하고 있습니다. 모든 프로젝트에서 사용할 수 있도록 Heroku를 전역적으로 설치하고자 합니다. 이를 쉽게 하는 방법은 현재 가상 환경에서 작동하지 않는 새 명령줄 탭을 여는 것입니다—Windows에서는 ``Control+t, Mac에서는 Command+t입니다. 여기에서 설치된 모든 것은 전역적입니다.

Windows에서는 [Heroku CLI 페이지](#)를 참조하여 32비트 또는 64비트 버전을 올바르게 설치하세요. Mac에서는 [패키지 관리자 Homebrew](#)를 사용하여 설치합니다. 이미 머신에 없다면 Homebrew 웹사이트에서 긴 명령을 복사하여 명령줄에 붙여넣고 Return을 누르세요. 다음과 비슷하게 보일 것입니다:

셸

```
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

다음으로, 다음을 명령줄에 복사하여 붙여넣고 Return을 눌러 Heroku CLI를 설치하세요.

셸

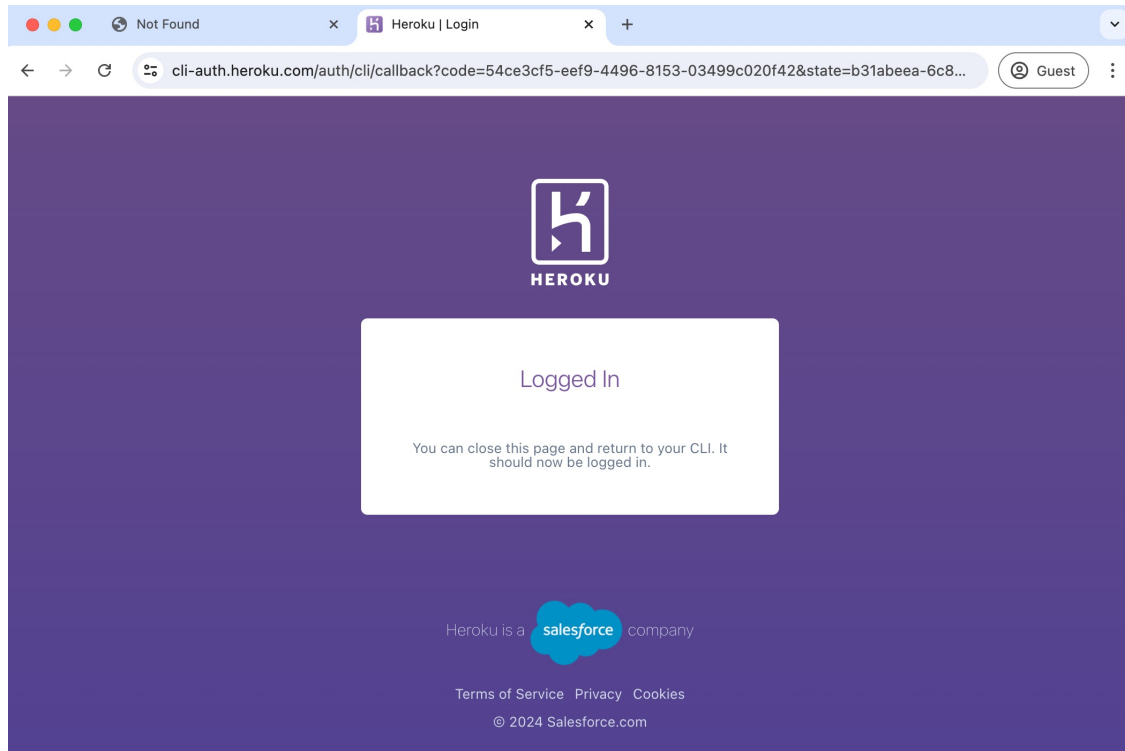
```
$ brew tap heroku/brew && brew install heroku
```

설치가 완료되면, 새 명령줄 탭을 닫고 신문 가상 환경이 활성화된 초기 탭으로 돌아갈 수 있습니다.

heroku login 명령을 입력하고 Enter 키를 누르세요. 그런 다음 아무 키나 눌러 브라우저 창을 열고, 방금 설정한 이메일, 비밀번호 및 이중 인증으로 로그인할 수 있습니다.

Shell

```
(.venv) > heroku login
heroku: Press any key to open up the browser to login or q to exit:
Opening browser to https://cli-auth.heroku.com/auth/cli/browser/....
```



Heroku Login

Once you are logged in we are ready to go!

Deploy with Heroku

There are two ways to interact with Heroku: via its CLI (Command Line Interface) or the website. The CLI is much faster, which we will use here, but the website's more visual nature is helpful if you are new to Heroku.

The first step is to create a new Heroku app. from the command line with `heroku create`. Heroku will create a random name for our app, in my case `fathomless-hamlet-26076`. Your name will be different.

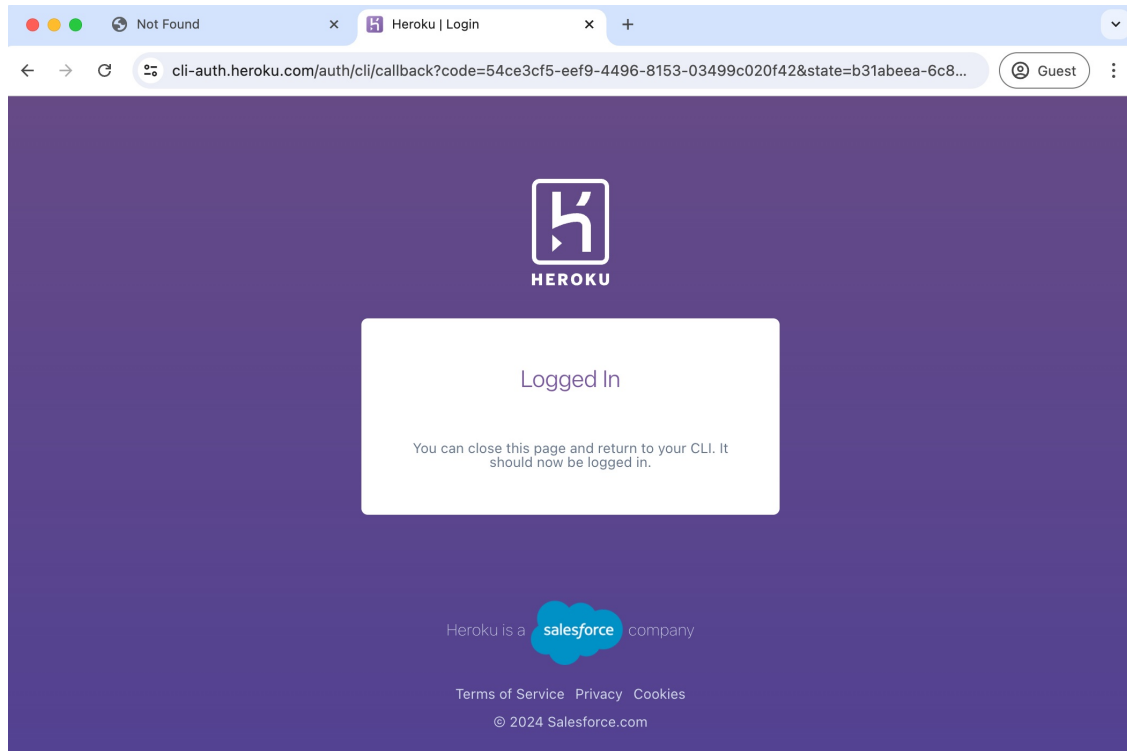
Shell

```
(.venv) $ heroku create
Creating app... done, 🍷 afternoon-wave-82807
```

셸

(.venv) > heroku 로그인

heroku: 로그인하기 위해 브라우저를 열려면 아무 키나 누르거나 종료하려면 q를 누르세요:
브라우저를 <https://cli-auth.heroku.com/auth/cli/browser/>로 여는 중입니다.



Heroku 로그인

로그인하면 준비가 완료됩니다!

Heroku로 배포하기

Heroku와 상호작용하는 방법은 두 가지가 있습니다: CLI(명령줄 인터페이스)를 통해 또는 웹사이트를 통해. CLI는 훨씬 빠르며, 여기서 사용할 것이지만, 웹사이트의 더 시각적인 특성은 Heroku에 익숙하지 않은 경우 유용합니다.

첫 번째 단계는 `heroku create`를 사용하여 명령줄에서 새로운 Heroku 앱을 만드는 것입니다. Heroku는 우리 앱을 위한 무작위 이름을 생성할 것이며, 제 경우에는 `fathomless`입니다.

`hamlet-26076`. 당신의 이름은 다를 것입니다.셸

(.venv) \$ heroku create

앱 생성 중... 완료, ● afternoon-wave-82807

```
https://afternoon-wave-82807-b672795cd97e.herokuapp.com/ |  
https://git.heroku.com/afternoon-wave-82807.git
```

The `heroku create` command also creates a dedicated Git remote named `heroku` for our app. To see this, type `git remote -v`.

Shell

```
(.venv) $ git remote -v  
heroku https://git.heroku.com/afternoon-wave-82807.git (fetch)  
heroku https://git.heroku.com/afternoon-wave-82807.git (push)
```

The next step is creating a PostgreSQL database on Heroku. There are various [Postgres tiers](#) available for different use cases. The five plan tiers are Essential, Standard, Premium, Private, and Shield. The more you pay the less downtime is tolerated. For our use case, the lowest tier, Essential, is more than adequate. Run the following command to create a new Essential Postgres database for our project.

Shell

```
(.venv) $ heroku addons:create heroku-postgresql:essential-0  
Creating heroku-postgresql:essential-0 on 🍬 afternoon-wave-82807...  
~$0.007/hour (max $5/month)  
Database should be available soon  
postgresql-sinuous-77120 is being created in the background. The app will restart  
when complete...  
Use heroku addons:info postgresql-sinuous-77120 to check creation progress  
Use heroku addons:docs heroku-postgresql to view documentation
```

The database might require a moment to provision, in which case you can wait a few minutes and then run the command to “check creation progress.” Make sure the database name matches your project.

Shell

```
(.venv) $ heroku addons:info postgresql-sinuous-77120  
=== postgresql-sinuous-77120  
Attachments: afternoon-wave-82807::DATABASE  
Installed at: Tue Jul 02 2024 10:57:17 GMT-0400 (Eastern Daylight Time)  
Max Price: $5/month  
Owning app: afternoon-wave-82807  
Plan: heroku-postgresql:essential-0  
Price: ~$0.007/hour  
State: created
```

If you run `heroku config`, it will show all configuration variables set on Heroku. At the moment, that is just a `DATABASE_URL` with the information to connect to the production Postgres database.

<https://afternoon-wave-82807-b672795cd97e.herokuapp.com/> |
<https://git.heroku.com/afternoon-wave-82807.git>

heroku create 명령은 또한 우리 앱을 위한 전용 Git 원격을 heroku라는 이름으로 생성합니다. 이를 보려면 git remote -v를 입력하세요.

셸

```
(.env) $ git remote -v
heroku    https://git.heroku.com/afternoon-wave-82807.git (fetch)
heroku    https://git.heroku.com/afternoon-wave-82807.git (push)
```

다음 단계는 Heroku에서 PostgreSQL 데이터베이스를 만드는 것입니다. 다양한 사용 사례에 맞는 여러 Postgres 계층이 있습니다. 다섯 가지 요금제 계층은 Essential, Standard, Premium, Private 및 Shield입니다. 더 많이 지불할수록 다운타임이 덜 허용됩니다. 우리의 사용 사례에 대해 가장 낮은 계층인 Essential이 충분합니다. 다음 명령을 실행하여 우리 프로젝트를 위한 새로운 Essential Postgres 데이터베이스를 만드세요.

셸

```
(.env) $ heroku addons:create heroku-postgresql:essential-0
● afternoon-wave-82807에서 heroku-postgresql:essential-0 생성 중...~$0.007/시간
(최대 $5/월)
데이터베이스가 곧 사용 가능해질 것입니다
postgresql-sinuous-77120이(가) 백그라운드에서 생성되고 있습니다. 완료되면 앱이 재시작됩니다...
```

생성 진행 상황을 확인하려면 heroku addons:info postgresql-sinuous-77120을(를) 사용하세요
문서를 보려면 heroku addons:docs heroku-postgresql을(를) 사용하세요

데이터베이스가 프로비저닝되는 데 잠시 걸릴 수 있으며, 이 경우 몇 분 기다린 후 "생성 진행 상황 확인" 명령을 실행할 수 있습니다. 데이터베이스 이름이 프로젝트와 일치하는지 확인하세요.

Shell

```
(.env) $ heroku addons:info postgresql-sinuous-77120==
postgresql-sinuous-77120
첨부파일: afternoon-wave-82807::DATABASE
설치 날짜: 2024년 7월 2일 화요일 10:57:17 GMT-0400 (동부 일광 절약 시간)
최대 가격: 앱      $5/월
소유: 계획: 가    afternoon-wave-82807
격: 상태:        heroku-postgresql:essential-0~
                  $0.007/시간
                  생성됨
```

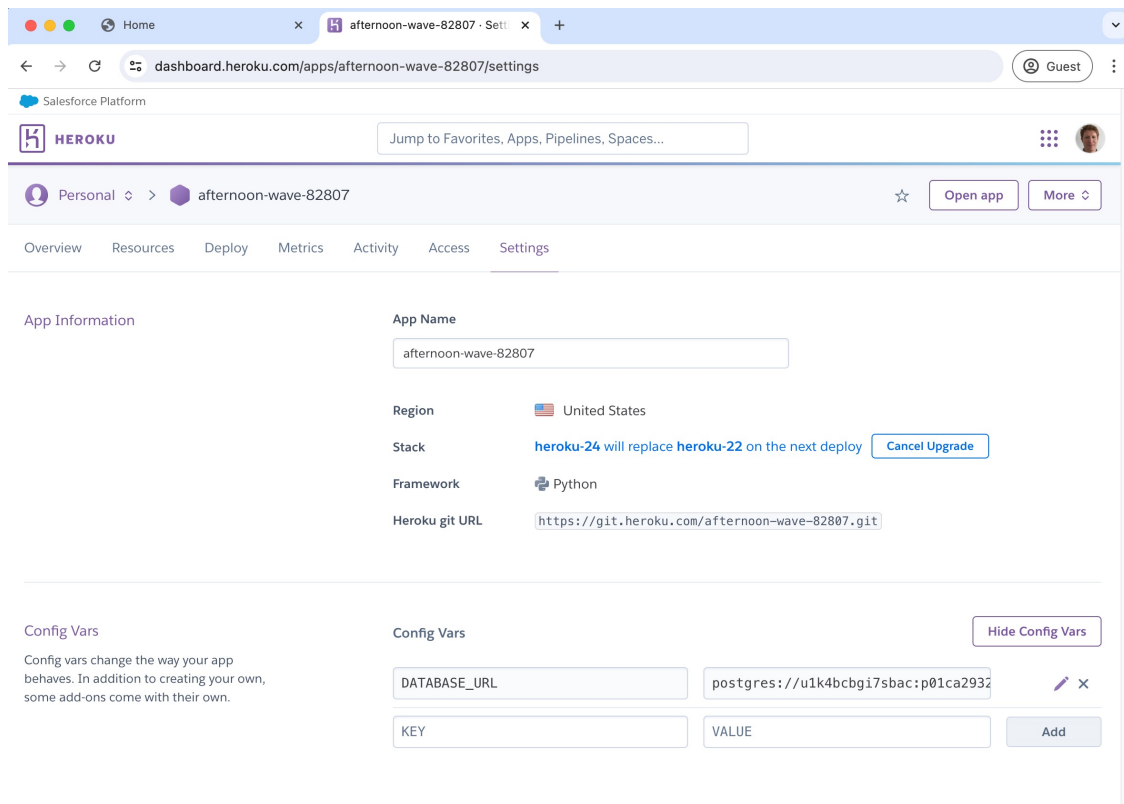
heroku config를 실행하면 Heroku에 설정된 모든 구성 변수가 표시됩니다. 현재는 프로덕션 Postgres 데이터베이스에 연결하기 위한 정보가 포함된 DATABASE_URL만 있습니다.

Shell

```
(.venv) $ heroku config  
=== afternoon-wave-82807 Config Vars
```

```
DATABASE_URL: postgres://u1k...us-east-1.rds.amazonaws.com:5432/d11ac0v0inabta
```

Select your project on the Heroku website dashboard and click “Settings” in the navigation bar. Under “Config Vars” you can see that the `DATABASE_URL` has been set.



Heroku Dashboard Configs

There are two other items in our local `.env` file, `DEBUG` and `SECRET_KEY`. We need to manually set both on Heroku, either in the web interface or the command line. First up is `DEBUG` which should be `False`.

Shell

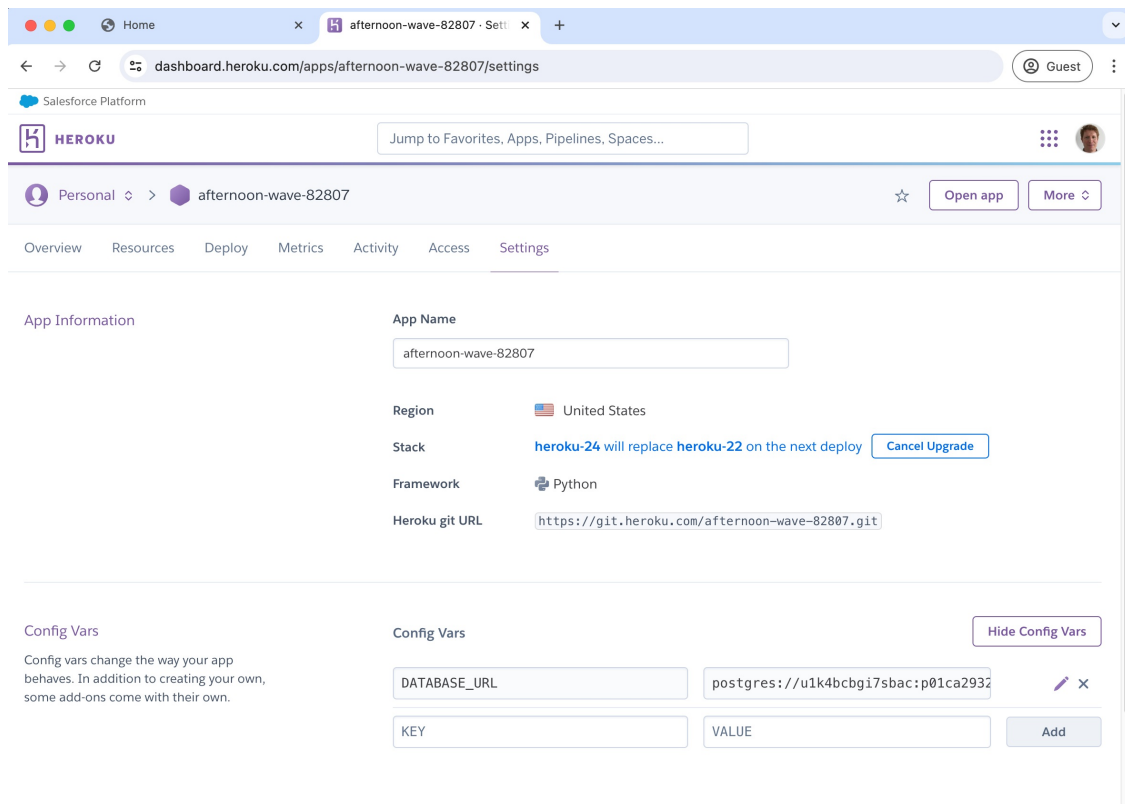
```
(.venv) $ heroku config:set DEBUG=False  
Setting DEBUG and restarting ● afternoon-wave-82807... done, v6  
DEBUG: False
```

셸

```
(.venv) $ heroku config=== afternoon-wave-82807 구성 변수
```

```
DATABASE_URL: postgres://u1k...us-east-1.rds.amazonaws.com:5432/d11ac0v0inabta
```

Heroku 웹사이트 대시보드에서 프로젝트를 선택하고 탐색 막대에서 “설정”을 클릭하세요. “구성 변수” 아래에서 DATABASE_URL이 설정되었음을 확인할 수 있습니다.



Heroku 대시보드 구성

로컬 .env 파일에는 DEBUG와 SECRET_KEY라는 두 가지 항목이 있습니다. 우리는 웹 인터페이스나 명령줄에서 둘 다 수동으로 Heroku에 설정해야 합니다. 먼저 DEBUG가 False여야 합니다.셸

```
(.venv) $ heroku config:set DEBUG=False
```

```
DEBUG 설정 및 재시작 ● afternoon-wave-82807... 완료, v6DEBUG: False
```

Next is the SECRET_KEY. Make sure to wrap it in quotations, "", if you do so via the command line.

Shell

```
(.venv) $ heroku config:set SECRET_KEY="SECRET_KEY=imDnflXy-8Y-YozfJmP2Rw_81YA_qx1XK15FeY0mXyY"
Setting SECRET_KEY and restarting ● afternoon-wave-82807... done, v7
SECRET_KEY: SECRET_KEY=imDnflXy-8Y-YozfJmP2Rw_81YA_qx1XK15FeY0mXyY
```

It's a good idea to double-check that the production environment variables are properly set. From the command line that means using the heroku config command.

Shell

```
(.venv) $ heroku config
=== afternoon-wave-82807 Config Vars

DATABASE_URL: postgres://u1k...us-east-1.rds.amazonaws.com:5432/d11ac0v0inabta
DEBUG:        False
SECRET_KEY:   SECRET_KEY=imDnflXy-8Y-YozfJmP2Rw_81YA_qx1XK15FeY0mXyY
```

You can also look at the web dashboard.

다음은 SECRET_KEY입니다. 명령줄을 통해 그렇게 할 경우, 반드시 따옴표로 감싸야 합니다, "".

셀

```
(.venv) $ heroku config:set SECRET_KEY="SECRET_KEY=imDnflXy-8Y-YozfJmP2Rw_81YA_qx1XKI5FeY0mXyY"
SECRET_KEY를 설정하고 ● afternoon-wave-82807을 (를) 재시작하는 중... 완료, v7
SECRET_KEY: SECRET_KEY=imDnflXy-8Y-YozfJmP2Rw_81YA_qx1XKI5FeY0mXyY
```

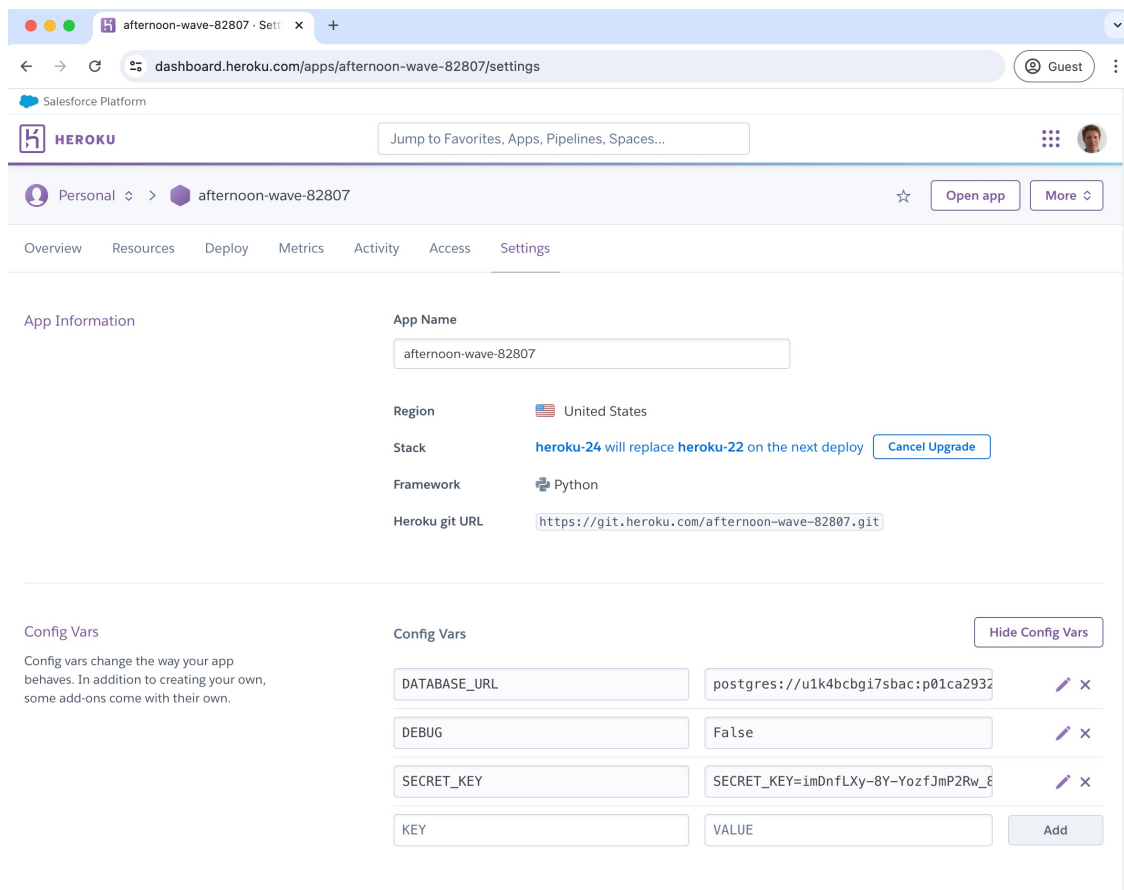
생산 환경 변수가 제대로 설정되었는지 다시 확인하는 것이 좋습니다. 명령줄에서는 heroku config 명령을 사용하는 것을 의미합니다.

셀

```
(.venv) $ heroku config=== afternoon-wave-82807 구성 변수

DATABASE_URL: postgres://u1k...us-east-1.rds.amazonaws.com:5432/d11ac0v0inabta
DEBUG:       False
SECRET_KEY:   SECRET_KEY=imDnflXy-8Y-YozfJmP2Rw_81YA_qx1XKI5FeY0mXyY
```

웹 대시보드를 확인할 수도 있습니다.

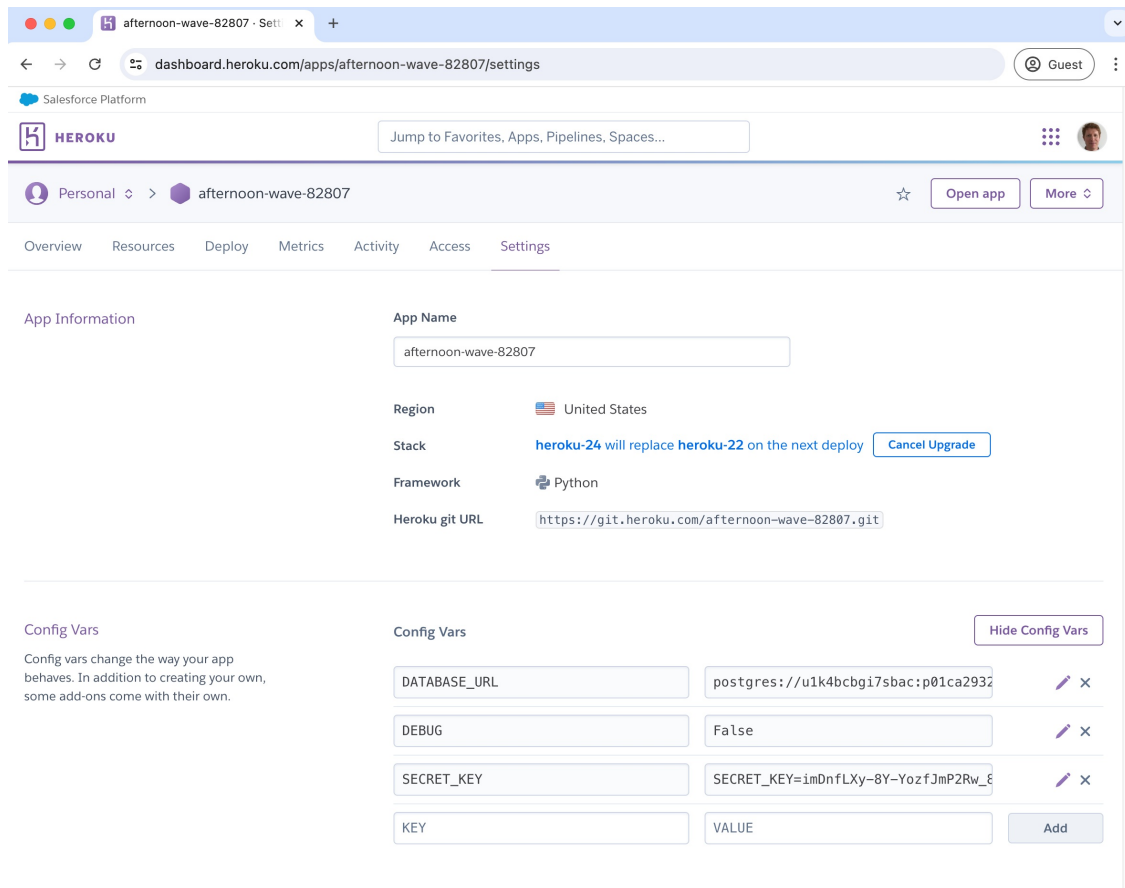


Heroku Dashboard Updated Configs

Now it is time to push our code up to Heroku with the command, `git push heroku main`. If we had just typed `git push origin main` the code would have been pushed to GitHub, not Heroku. Adding heroku to the command sends the code to Heroku.

Shell

```
(.venv) $ git push heroku main
Enumerating objects: 339, done.
Counting objects: 100% (339/339), done.
Delta compression using up to 10 threads
Compressing objects: 100% (333/333), done.
Writing objects: 100% (339/339), 798.15 KiB | 14.25 MiB/s, done.
Total 339 (delta 39), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (39/39), done.
remote: Updated 569 paths from 2ebafa9
remote: Compressing source files... done.
remote: Building source:
remote:
remote: -----> Building on the Heroku-22 stack
...
remote:      https://afternoon-wave-82807-b672795cd97e.herokuapp.com/
        deployed to Heroku
```



헤로쿠 대시보드 업데이트된 설정

이제 코드를 헤로쿠에 푸시할 시간입니다. 명령어 `git push heroku main` 을 사용하세요. 만약 `git push origin main`을 입력했다면 코드는 헤로쿠가 아닌 깃허브로 푸시되었을 것입니다. 명령어에 `heroku`를 추가하면 코드를 헤로쿠로 보냅니다.

셸

```
(.venv) $ git push heroku main 객체 열
거: 339, 완료.
객체 수 세기: 100% (339/339), 완료. 델타 압축을 위
해 최대 10개의 스레드 사용 객체 압축: 100%
(333/333), 완료.
객체 쓰기: 100% (339/339), 798.15 KiB | 14.25 MiB/s, 완료. 총 339 (델타 39),
재사용 0 (델타 0), 패키지 재사용 0 (0에서) 원격: 델타 해결: 100% (39/39), 완
료.
원격: 2ebafa9에서 569 경로 업데이트됨 원격: 소스
파일 압축 중... 완료. 원격: 소스 빌드 중:

원격:
원격: -----> Heroku-22 스택에서 빌드 중...

원격: https://afternoon-wave-82807-b672795cd97e.herokuapp.com/
Heroku에 배포됨
```

```
remote:
remote: Verifying deploy... done.
To https://git.heroku.com/afternoon-wave-82807.git
* [new branch]      main -> main
```

This command generates a lot of output from Heroku and might take a while the first time. We are pushing the code to Heroku, and it is rebuilding a production version of our Django project on its servers. You'll see that it installs each item from our `requirements.txt` file, among other actions.

The last step is starting a [Dyno](#), Heroku's term for our app's lightweight container. We need at least one to be running to make our website live. If we start to see a spike in traffic, we could add more dynos to our project, and Heroku will handle all the infrastructure. For a small project like this, I recommend the [Basic Dyno](#), which is \$0.01 per hour with a maximum of \$7 per month.

We will spin up one dyno using the Heroku CLI, but dynos can also be managed via the web interface. The general syntax for the CLI is to start with `heroku`, `ps` is a command that prefixes many commands affecting dynos, `ps:scale` is used to increase the number of dynos running a process. Therefore, the command below tells Heroku to run one dyno for our website.

Shell

```
(.venv) $ heroku ps:scale web=1
Scaling dynos... done, now running web at 1:Basic
```

The total cost for our project—if we let it run all the time—is \$12 per month: \$5/month for the Postgres database and \$7/month for the Dyno. Heroku bills per hour so you can always deploy the website and take it down after a few days, which should cost only cents.

We're done! The last step is to confirm our app is live and online. If you use the command `heroku open`, your web browser will open a new tab with the URL of your app:

Shell

```
(.venv) $ heroku open
```

```
remote:
remote: 배포 확인 중... 완료.
https://git.heroku.com/afternoon-wave-82807.git로 전송됨
* [새 브랜치] main -> main
```

이 명령은 Heroku에서 많은 출력을 생성하며 처음 실행할 때는 시간이 걸릴 수 있습니다. 우리는 코드를 Heroku에 푸시하고 있으며, Heroku의 서버에서 Django 프로젝트의 프로덕션 버전을 재구성하고 있습니다. requirements.txt 파일의 각 항목을 설치하는 것을 포함하여 여러 작업을 수행하는 것을 볼 수 있습니다.

마지막 단계는 Dyno를 시작하는 것입니다. Dyno는 우리의 앱의 경량 컨테이너에 대한 Heroku의 용어입니다. 웹사이트를 실시간으로 운영하기 위해서는 최소한 하나가 실행 중이어야 합니다. 트래픽이 급증하기 시작하면 프로젝트에 더 많은 Dyno를 추가할 수 있으며, Heroku가 모든 인프라를 처리합니다. 이런 작은 프로젝트에는 시간당 \$0.01, 월 최대 \$7인 Basic Dyno를 추천합니다.

Heroku CLI를 사용하여 하나의 Dyno를 시작할 것이지만, Dyno는 웹 인터페이스를 통해서도 관리할 수 있습니다. CLI의 일반 구문은 heroku로 시작하며, ps는 Dyno에 영향을 미치는 많은 명령의 접두사 역할을 하는 명령입니다. ps:scale은 프로세스를 실행하는 Dyno의 수를 늘리는 데 사용됩니다. 따라서 아래 명령은 Heroku에 웹사이트를 위해 하나의 Dyno를 실행하라고 지시합니다.

Shell

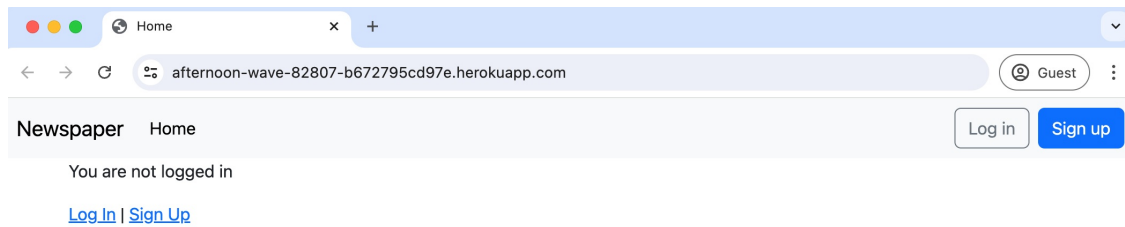
```
(. venv) $ heroku ps:scale web=1
Dyno 크기 조정 중... 완료, 이제 1:Basic에서 웹 실행 중
```

우리 프로젝트의 총 비용은 – 항상 실행하게 두면 – 월 \$12입니다: Postgres 데이터베이스에 대해 월 \$5, Dyno에 대해 월 \$7입니다. Heroku는 시간당 청구하므로 웹사이트를 배포한 후 며칠 후에 내릴 수 있으며, 이는 몇 센트만 비용이 들 것입니다.

우리는 끝났습니다! 마지막 단계는 우리의 앱이 라이브 상태인지 확인하는 것입니다. heroku open 명령어를 사용하면 웹 브라우저가 앱의 URL이 있는 새 탭을 엽니다:

셸

```
(. venv) $ heroku open
```



Production Homepage

The Newspaper website is live, but you'll quickly see some problems if you try it out. For one thing, there are no articles or comments! That's because we still need to configure the production PostgreSQL database running on Heroku. Let's do that now. To run Django commands on Heroku instead of locally, we use the prefix `heroku run`. So to migrate our database with initial settings, we run the following command:

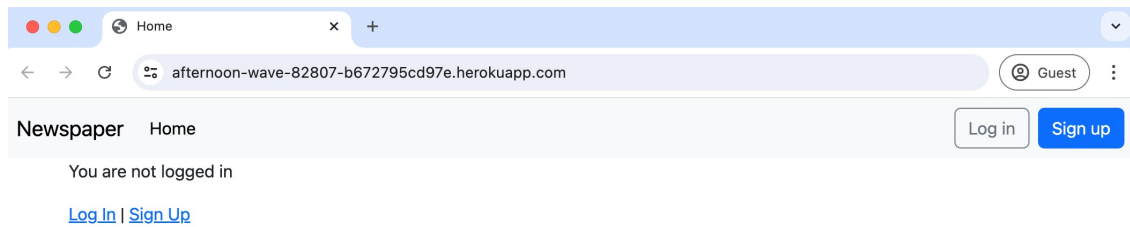
Shell

```
(.venv) $ heroku run python manage.py migrate
Running python manage.py migrate on ● afternoon-wave-82807... up, run.2790 (Basic)
Operations to perform:
  Apply all migrations: accounts, admin, articles, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0001_initial... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying accounts.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying articles.0001_initial... OK
  Applying articles.0002_comment... OK
  Applying sessions.0001_initial... OK
```

Now, create a superuser account to access the admin.

Shell

```
(.venv) $ heroku run python manage.py createsuperuser
Running python manage.py createsuperuser on ● afternoon-wave-82807... up, run.7422
```



생산 홈페이지

신문 웹사이트가 활성화되었지만, 사용해보면 몇 가지 문제가 발생하는 것을 금방 알 수 있습니다. 우선, 기사나 댓글이 없습니다! 이는 Heroku에서 실행 중인 생산 PostgreSQL 데이터베이스를 아직 구성해야 하기 때문입니다. 지금 그 작업을 해봅시다. 로컬이 아닌 Heroku에서 Django 명령을 실행하기 위해서는 접두어 `heroku run`을 사용합니다. 따라서 초기 설정으로 데이터베이스를 마이그레이션하기 위해 다음 명령을 실행합니다:

셸

```
(.venv) $ heroku run python manage.py migrate
```

● afternoon-wave-82807에서 python manage.py migrate 실행 중... 실행 중, run.2790 (기본) 수행할 작업:

모든 마이그레이션 적용: accounts, admin, articles, auth, contenttypes, sessions 마이그레이션
실행 중:

```
contenttypes.0001_initial 적용 중... OK
contenttypes.0002_remove_content_type_name 적용 중...
OKauth.0001_initial 적용 중... OK
auth.0002_alter_permission_name_max_length 적용 중...
OKauth.0003_alter_user_email_max_length 적용 중...
OKauth.0004_alter_user_username_opts 적용 중... OK
auth.0005_alter_user_last_login_null 적용 중...
OKauth.0006_require_contenttypes_0002 적용 중... OK
auth.0007_alter_validators_add_error_messages 적용 중...
OKauth.0008_alter_user_username_max_length 적용 중...
OKauth.0009_alter_user_last_name_max_length 적용 중...
OKauth.0010_alter_group_name_max_length 적용 중...
OKauth.0011_update_proxy_permissions 적용 중... OK
auth.0012_alter_user_first_name_max_length 적용 중...
OKaccounts.0001_initial 적용 중... OK
admin.0001_initial 적용 중... OK
admin.0002_logentry_remove_auto_add 적용 중... OK
admin.0003_logentry_add_action_flag_choices 적용 중...
OKarticles.0001_initial 적용 중... OK
articles.0002_comment 적용 중... OK
sessions.0001_initial 적용 중... OK
```

이제 관리에 접근할 수 있는 슈퍼유저 계정을 생성하세요.

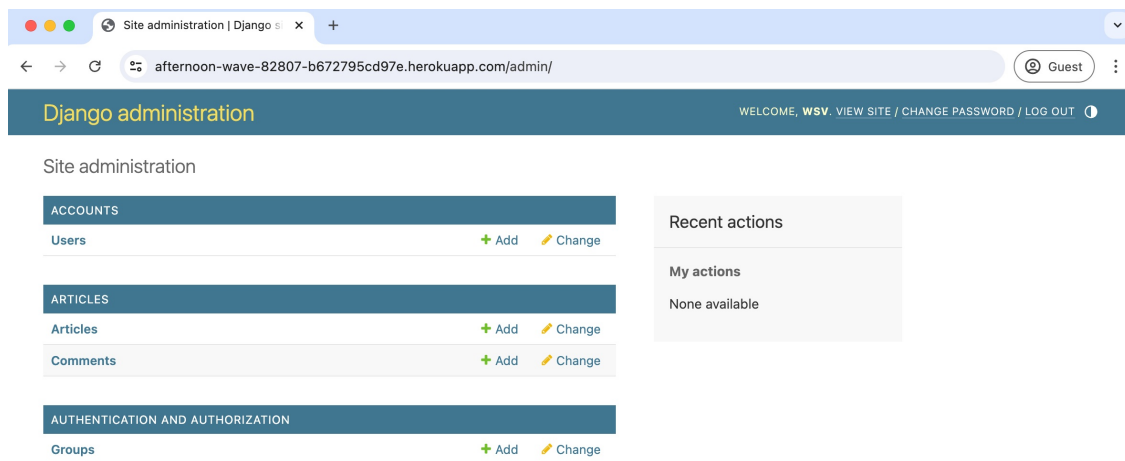
셸

```
(.venv) $ heroku run python manage.py createsuperuser
```

Running python manage.py createsuperuser on ● afternoon-wave-82807... up, run.7422

(Basic)
Username: wsv
Email address: will@learndjango.com
Password:
Password (again):
Superuser created successfully.

Navigate to the admin section of your deployed website, log in with your superuser credentials, and add some articles and comments.



Production Admin Dashboard

They will then be displayed on the live website. You can also create user accounts and confirm that the user authentication flow works correctly by resetting your password.

For future updates to the production website, the pattern is as follows:

- make local code changes and save them with Git
- use `git push origin main` to deploy them to GitHub
- use `git push heroku main` to push the code to Heroku

If you want to remove a hosted website, log into your [Heroku dashboard](#) and click on the app name. Click on the Settings link in the navigation bar at the top, then scroll down to the bottom of the page under Delete App and click the “Delete App...” button. You will be asked to type in your full app name one more time to confirm that you want to permanently delete it.

(기본)사용자

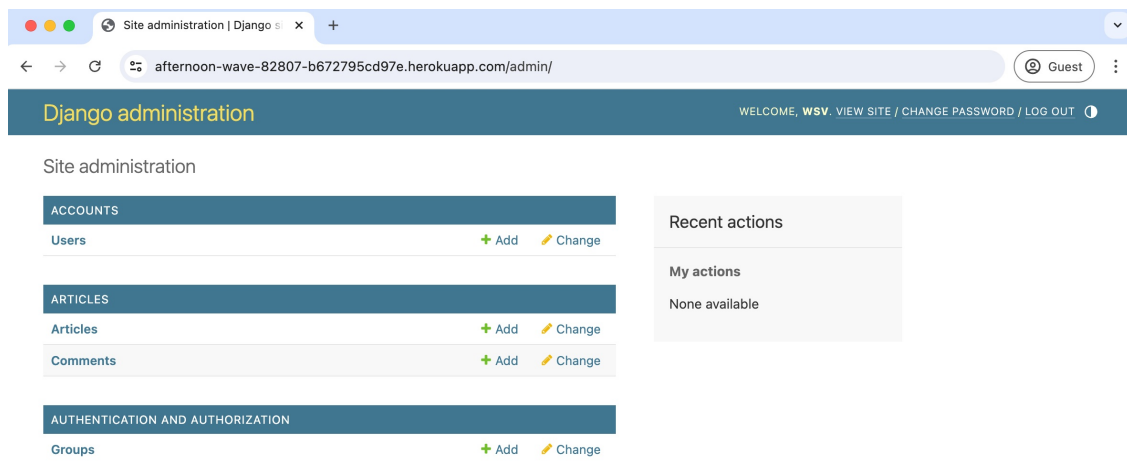
이름 : wsv

이메일 주소 : will@learndjango.com비밀번호 :

비밀번호(다시 입력) :

슈퍼유저가 성공적으로 생성되었습니다.

배포된 웹사이트의 관리자 섹션으로 이동하여 슈퍼유저 자격 증명으로 로그인하고 일부 기사와 댓글을 추가하세요.



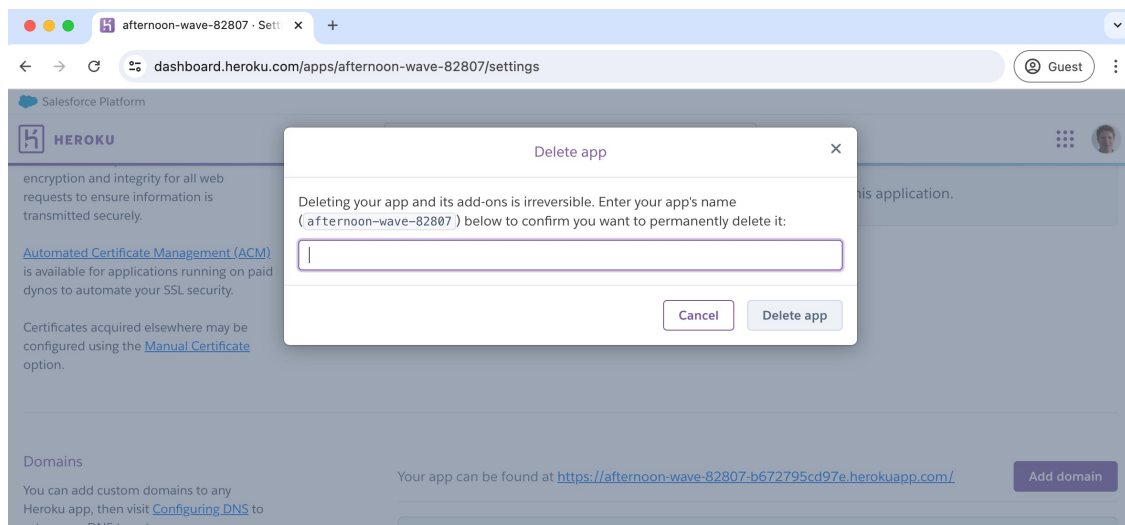
프로덕션 관리자 대시보드

그들은 라이브 웹사이트에 표시됩니다. 또한 사용자 계정을 생성하고 비밀번호를 재설정하여 사용자 인증 흐름이 올바르게 작동하는지 확인할 수 있습니다.

프로덕션 웹사이트에 대한 향후 업데이트는 다음과 같은 패턴입니다:

- 로컬 코드 변경을 하고 Git으로 저장한 후 git push
- origin main을 사용하여 GitHub에 배포하고 git push
- heroku main을 사용하여 코드를 Heroku에 푸시합니다.

호스팅된 웹사이트를 제거하려면 Heroku 대시보드에 로그인하고 앱 이름을 클릭하세요. 상단의 탐색 모음에서 설정 링크를 클릭한 다음 페이지 하단으로 스크롤하여 앱 삭제 아래에서 “앱 삭제...” 버튼을 클릭하세요. 영구적으로 삭제하려는 앱 이름을 한 번 더 입력하라는 메시지가 표시됩니다.



Heroku Delete App

Another tip is that you can type `Ctrl + d` to exit the Heroku CLI at any time.

Additional Security Steps

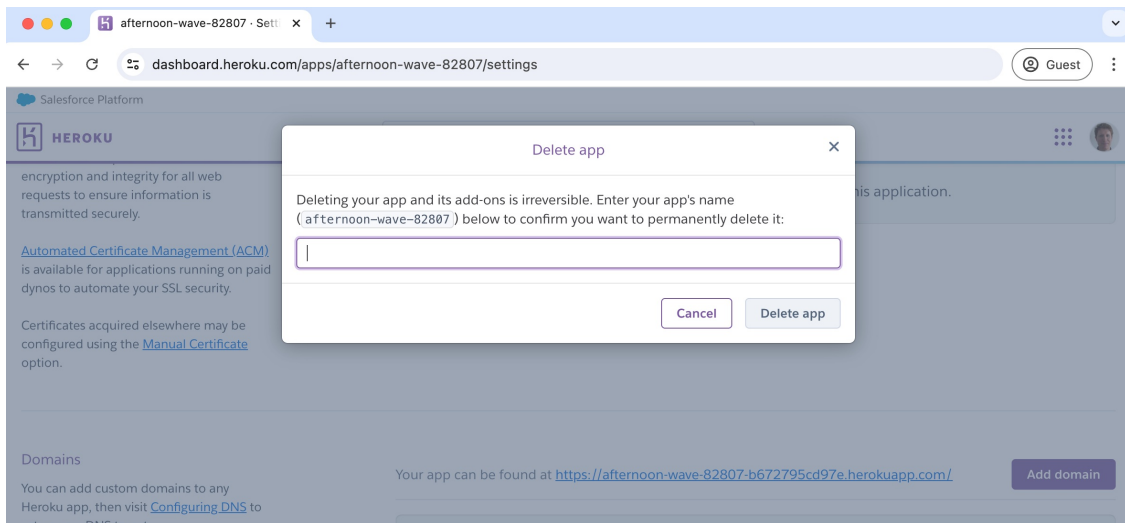
There is an almost infinite list of additional security procedures to secure a production website. Our production checklist covers the basics, but there are more if you want to take the additional steps.

First, update `ALLOWED_HOSTS` and `CSRF_TRUSTED_ORIGINS` to use the exact production URL for your project.

Next, you would run Django's management command, which runs several [automated checks](#) around deployment. To run that command you would prefix `heroku run` so it would be `heroku run python manage.py check --deploy`. You now know how to reference the Django docs and update your local and production environment variables to make them pass.

Conclusion

We just covered a lot of new material, so you will likely feel overwhelmed. That's normal. There are many steps involved in configuring a website for proper deployment, and the good news is that this same list of production settings will hold true for almost every Django project. Don't worry about memorizing all the steps; use the deployment checklist!



Heroku 앱 삭제

또 다른 팁은 언제든지 Heroku CLI를 종료하기 위해 `Ctrl + d`를 입력할 수 있다는 것입니다.

추가 보안 단계

생산 웹사이트를 보호하기 위한 추가 보안 절차의 목록은 거의 무한합니다. 우리의 생산 체크리스트는 기본 사항을 다루지만, 추가 단계를 원하신다면 더 많은 절차가 있습니다.

먼저, `ALLOWED_HOSTS`와 `CSRF_TRUSTED_ORIGINS`를 프로젝트의 정확한 생산 URL로 업데이트하세요.

다음으로, Django의 관리 명령을 실행해야 합니다. 이 명령은 배포와 관련된 여러 자동 검사를 실행합니다. 해당 명령을 실행하려면 `heroku run`을 접두사로 붙여서 `heroku run python manage.py check --deploy`로 실행합니다. 이제 Django 문서를 참조하고 로컬 및 생산 환경 변수를 업데이트하여 통과시키는 방법을 알게 되었습니다.

결론

우리는 많은 새로운 내용을 다루었으므로 아마도 압도당할 것입니다. 그건 정상입니다. 웹사이트를 적절하게 배포하기 위해 구성하는 데는 많은 단계가 포함되며, 좋은 소식은 이 동일한 생산 설정 목록이 거의 모든 Django 프로젝트에 적용된다는 것입니다. 모든 단계를 암기하는 것에 대해 걱정하지 마세요; 배포 체크리스트를 사용하세요!

The other big stumbling block for newcomers is becoming comfortable with the difference between local and production environments. You will likely forget to push code changes into production and spend minutes or hours wondering why the change isn't live on your site. Or even worse, you'll change your local SQLite database and expect them to magically appear in the production PostgreSQL database. It's part of the learning process, but Django makes it much smoother than it otherwise would be. You know enough to confidently deploy any Django project online with a PaaS.

신규 사용자에게 또 다른 큰 장애물은 로컬 환경과 프로덕션 환경의 차이에 익숙해지는 것입니다. 코드 변경 사항을 프로덕션에 푸시하는 것을 잊어버리고 변경 사항이 사이트에 실시간으로 반영되지 않는 이유를 몇 분 또는 몇 시간 동안 고민하게 될 것입니다. 또는 더 나쁜 경우, 로컬 SQLite 데이터베이스를 변경하고 그것이 프로덕션 PostgreSQL 데이터베이스에 마법처럼 나타나기를 기대할 것입니다. 이것은 학습 과정의 일부이지만, Django는 그렇지 않았을 경우보다 훨씬 더 원활하게 만들어 줍니다. 당신은 PaaS를 사용하여 온라인에 Django 프로젝트를 자신 있게 배포할 수 있을 만큼 충분히 알고 있습니다.

Chapter 17: Conclusion

Congratulations on finishing *Django for Beginners*! Starting from scratch, we've built six different web applications from scratch and covered Django's major features: templates, views, URLs, users, models, security, testing, and deployment. You now have the knowledge to build modern websites with Django.

As with any new skill, practicing and applying what you've just learned is important. The CRUD (Create-Read-Update-Delete) functionality in our *Blog* and *Newspaper* sites is commonplace in many other web applications. For example, can you make a Todo List web application? An Instagram or Facebook clone? You already have all the tools you need. When starting out, the best approach is to build as many small projects as possible, incrementally add complexity, and research new things.

Learning Resources

As you become more comfortable with Django and web development in general, you'll find the [official Django documentation](#) and [source code](#) increasingly valuable. I refer to both on an almost daily basis. There is also the [official Django forum](#), a great albeit underutilized resource for Django-specific questions.

To stay current with the latest Django news, the [Django News Newsletter](#) is a free weekly newsletter with all the latest news, events, articles, tutorials, and projects. If you prefer the audio format, [Django Chat](#) is a biweekly podcast I co-host with Django Fellow Carlton Gibson, where we interview leading developers and provide deep dives on various Django topics.

If you want an all-in-one source of free tutorials and additional courses that cover APIs, Docker, testing, and other topics in more depth please check out [LearnDjango.com](#), a comprehensive learning website I run focused exclusively on Django.

3rd Party Packages

제17장: 결론

Django for Beginners를 마치신 것을 축하드립니다! 처음부터 시작하여, 우리는 여섯 개의 서로 다른 웹 애플리케이션을 처음부터 만들었고 Django의 주요 기능인 템플릿, 뷰, URL, 사용자, 모델, 보안, 테스트 및 배포를 다루었습니다. 이제 Django로 현대적인 웹사이트를 구축할 수 있는 지식을 갖추게 되었습니다.

새로운 기술과 마찬가지로, 방금 배운 것을 연습하고 적용하는 것이 중요합니다. 우리의 블로그와 신문 사이트에서의 CRUD(생성-읽기-업데이트-삭제) 기능은 많은 다른 웹 애플리케이션에서 일반적입니다. 예를 들어, 할 일 목록 웹 애플리케이션을 만들 수 있나요? 인스타그램이나 페이스북 클론은요? 필요한 모든 도구를 이미 갖추고 있습니다. 시작할 때는 가능한 한 많은 작은 프로젝트를 만들고 점진적으로 복잡성을 추가하며 새로운 것을 연구하는 것이 가장 좋은 접근 방식입니다.

학습 자료

Django와 웹 개발에 점점 더 익숙해지면, 공식 Django 문서와 소스 코드가 점점 더 유용하다는 것을 알게 될 것입니다. [저는 거의 매일 두 가지 모두를 참조합니다.](#) 또한 공식 Django 포럼이 있으며, [Django 특정 질문에 대한 훌륭한](#) 하지만 활용도가 낮은 자원입니다.

최신 Django 뉴스에 대한 정보를 얻으려면, [Django News Newsletter](#)는 최신 뉴스, 이벤트, 기사, 튜토리얼 및 프로젝트가 포함된 무료 주간 뉴스레터입니다. 오디오 형식을 선호하신다면, [Django Chat](#)은 제가 Django Fellow Carlton Gibson과 공동 진행하는 격주 팟캐스트로, 우리는 주요 개발자들을 인터뷰하고 다양한 Django 주제에 대해 깊이 있는 논의를 제공합니다.

API, Docker, 테스트 및 기타 주제를 더 깊이 다루는 무료 튜토리얼과 추가 과정의 올인원 소스를 원하신다면, Django에만 집중한 포괄적인 학습 웹사이트인 [LearnDjango.com](#)을 확인해 보세요.

서드파티 패키지

As we've seen in this book, third-party packages are a vital part of the Django ecosystem, especially regarding deployment or improvements around user registration. It's common for a professional Django website to rely on dozens of such packages.

However, a word of caution is in order: don't mindlessly install and use third-party packages because it saves a small amount of time. Every additional package introduces another dependency, another risk that its maintainer won't fix every bug or keep up to date with the latest version of Django. Take the time to understand what it is doing.

If you'd like to view more packages, the [Django Packages](#) website is a comprehensive resource of over 4,000 available third-party packages. A more curated option, the [awesome-django](#) repo, which I run with the current maintainer of Django Packages, is worth a look. And if you need help starting a new project quickly, I've long maintained a free starter project, [DjangoX](#), that comes with the latest version of Django, built-in user authentication, and more to jumpstart any new projects.

Python Books

Django is, ultimately, just Python, so if your Python skills could improve, I recommend Eric Matthes's [Python Crash Course](#). For intermediate to advanced developers, [Fluent Python](#) and [Effective Python](#) are worthy of additional study.

Feedback

If you purchased this book on Amazon, please leave an honest review. Your review will have an enormous impact on book sales and help me continue to teach Django full-time.

Finally, I'd love to hear your thoughts about the book. It is a constant work in progress, and the detailed feedback I receive from readers helps me continue to improve it. I try to respond to every email at will@learndjango.com.

Thank you for reading the book. Good luck on your journey with Django!

이 책에서 보았듯이, 서드파티 패키지는 Django 생태계의 중요한 부분이며, 특히 배포나 사용자 등록 개선과 관련하여 그렇습니다. 전문 Django 웹사이트가 수십 개의 이러한 패키지에 의존하는 것은 일반적입니다.

그러나 주의할 점이 있습니다: 시간을 조금 절약한다고 해서 서드파티 패키지를 무작정 설치하고 사용하지 마십시오. 추가 패키지마다 또 다른 의존성이 생기고, 유지 관리자가 모든 버그를 수정하지 않거나 최신 Django 버전과 동기화하지 않을 위험이 있습니다. 그것이 무엇을 하는지 이해하는 데 시간을 투자하십시오.

더 많은 패키지를 보고 싶다면, [Django Packages](#) 웹사이트는 4,000개 이상의 사용 가능한 서드파티 패키지에 대한 포괄적인 리소스입니다. 더 선별된 옵션인 [awesome-django](#) 레포는 제가 Django Packages의 현재 유지 관리자와 함께 운영하는 것으로, 살펴볼 가치가 있습니다. 그리고 새로운 프로젝트를 빠르게 시작하는 데 도움이 필요하다면, 최신 Django 버전, 내장 사용자 인증 및 기타 기능이 포함된 무료 스타터 프로젝트인 DjangoX를 오랫동안 유지해왔습니다.

파이썬 서적

결국 Django는 단지 파이썬이므로, 파이썬 기술을 향상시키고 싶다면 Eric Matthes의 [Python Crash Course](#)를 추천합니다. 중급에서 고급 개발자를 위해 [Fluent Python](#)과 [Effective Python](#)은 추가 학습할 가치가 있습니다.

피드백

이 책을 Amazon에서 구매하셨다면, 솔직한 리뷰를 남겨주세요. 귀하의 리뷰는 책 판매에 엄청난 영향을 미치고 제가 Django를 전업으로 계속 가르치는 데 도움이 됩니다.

마지막으로, 이 책에 대한 귀하의 생각을 듣고 싶습니다. 이는 지속적으로 진행 중인 작업이며, 독자들로부터 받는 상세한 피드백이 제가 계속 개선하는 데 도움이 됩니다. 저는 will@learndjango.com으로 오는 모든 이메일에 응답하려고 노력합니다.

책을 읽어주셔서 감사합니다. Django와 함께하는 여정에 행운이 있기를 바랍니다!